Program No: 01	**Date:** 05-12-2025

Program Title: Design a responsive webpage using HTML5 semantic elements and CSS.

```
<!--
Responsive Design
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Git Essentials | Cheat Sheet</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
    href="https://fonts.googleapis.com/css2?
family=JetBrains+Mono:wght@400;700&family=Inter:wght@400;600&display=swap"
    rel="stylesheet">
  <style>
    :root {
      --primary: 14 89% 53%;
      --secondary: 220 15% 20%;
      --surface-1: 0 0% 98%;
      --surface-2: 0 0% 100%;
      --text-1: 220 15% 10%;
      --text-2: 220 10% 40%;
      --code-bg: 220 15% 94%;
      --shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1);
      --font-body: 'Inter', system-ui, sans-serif;
      --font-mono: 'JetBrains Mono', monospace;
    }

    *,
    *::before,
    *::after {
```

```
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: var(--font-body);
    background-color: hsl(var(--surface-1));
    color: hsl(var(--text-1));
    line-height: 1.5;
    height: 100vh;
    display: flex;
    flex-direction: column;
    overflow: hidden;
}

a {
    color: inherit;
    text-decoration: none;
    transition: color 0.2s;
}

a:hover {
    color: hsl(var(--primary));
}

h1,
h2,
h3 {
    line-height: 1.1;
    margin-bottom: 0.75rem;
    letter-spacing: -0.02em;
}

h1 {
    font-size: clamp(2rem, 3vw, 2.5rem);
    color: hsl(var(--text-1));
}

code {
    font-family: var(--font-mono);
```

```
    background: hsl(var(--code-bg));
    padding: 0.2em 0.4em;
    border-radius: 4px;
    font-size: 0.9em;
    color: #d03d3d;
}

.container {
    width: min(94%, 1200px);
    margin-inline: auto;
    height: 100%;
    display: flex;
    flex-direction: column;
}

header {
    padding-block: 1rem;
    background-color: hsl(var(--surface-2));
    border-bottom: 1px solid rgba(0, 0, 0, 0.1);
    flex-shrink: 0;
}

.header-inner {
    display: flex;
    justify-content: space-between;
    align-items: center;
    height: auto;
    flex-direction: row;
}

.logo {
    font-weight: 800;
    font-size: 1.5rem;
    display: flex;
    align-items: center;
    gap: 0.5rem;
}

.logo span {
    color: hsl(var(--primary));
}
```

```
nav ul {
    list-style: none;
    display: flex;
    gap: 1.5rem;
    font-weight: 500;
    font-size: 0.95rem;
}

main {
    flex: 1;
    display: grid;
    grid-template-columns: 1.8fr 1.2fr;
    grid-template-rows: auto 1fr;
    gap: 1.5rem;
    padding-block: 1.5rem;
    min-height: 0;
}

.hero {
    grid-column: 1 / -1;
    background: hsl(var(--secondary));
    color: white;
    padding: 2rem;
    border-radius: 12px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    box-shadow: var(--shadow);
}

.hero h1 {
    color: white;
    margin-bottom: 0.5rem;
    font-size: 2rem;
}

.hero p {
    opacity: 0.9;
    max-width: 600px;
}
```

```
.btn {
    background: hsl(var(--primary));
    color: white;
    padding: 0.6em 1.2em;
    border-radius: 6px;
    font-weight: 600;
    font-size: 0.9rem;
    border: 1px solid transparent;
}

section {
    background: hsl(var(--surface-2));
    padding: 1.5rem;
    border-radius: 12px;
    border: 1px solid rgba(0, 0, 0, 0.05);
    box-shadow: var(--shadow);
    overflow-y: auto;
}

.command-list {
    display: grid;
    gap: 1rem;
}

.command-item {
    display: flex;
    flex-direction: column;
    gap: 0.25rem;
    padding-bottom: 0.75rem;
    border-bottom: 1px dashed rgba(0, 0, 0, 0.1);
}

.command-item:last-child {
    border-bottom: none;
}

.command-item strong {
    font-family: var(--font-mono);
    color: hsl(var(--text-1));
    font-size: 1em;
```

```
}

.command-item span {
  font-size: 0.9rem;
  color: hsl(var(--text-2));
}

aside h3 {
  color: hsl(var(--primary));
  border-bottom: 2px solid hsl(var(--surface-1));
  padding-bottom: 0.5rem;
}

aside ul {
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 0.8rem;
  margin-top: 1rem;
}

aside li {
  display: flex;
  gap: 0.5rem;
  align-items: baseline;
  font-size: 0.9rem;
}

aside li::before {
  content: '✓';
  color: hsl(var(--primary));
  font-weight: bold;
}

footer {
  text-align: center;
  padding: 1rem;
  font-size: 0.85rem;
  color: hsl(var(--text-2));
  border-top: 1px solid rgba(0, 0, 0, 0.05);
  margin-top: auto;
```

```
        flex-shrink: 0;
      }

      @media (max-width:800px) {
        body {
          overflow-y: auto;
          height: auto;
        }

        main {
          grid-template-columns: 1fr;
          display: flex;
          flex-direction: column;
        }

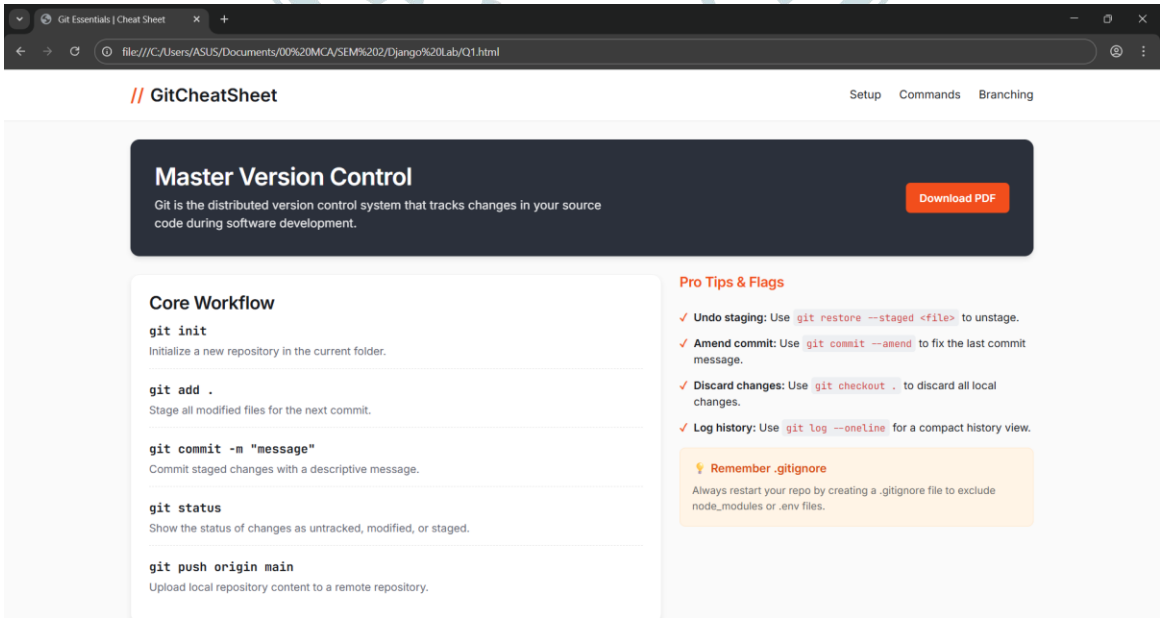
        .hero {
          flex-direction: column;
          text-align: center;
          gap: 1rem;
        }
      }
    </style>
  </head>

  <body>
    <header role="banner">
      <div class="container" style="flex-direction:row;height:auto;">
        <div class="header-inner" style="width:100%;">
          <div class="logo"><span>//</span> GitCheatSheet</div>
          <nav role="navigation">
            <ul>
              <li><a href="#">Setup</a></li>
              <li><a href="#">Commands</a></li>
              <li><a href="#">Branching</a></li>
            </ul>
          </nav>
        </div>
      </div>
    </header>
    <div class="container">
      <main role="main">
```

```
<article class="hero">
  <div>
    <h1>Master Version Control</h1>
    <p>Git is the distributed version control system that
       tracks changes in your source code during
      software development.</p>
  </div>
  <a href="#" class="btn">Download PDF</a>
</article>
<section aria-labelledby="core-cmds">
  <h2 id="core-cmds">Core Workflow</h2>
  <div class="command-list">
    <div class="command-item">
      <strong>git init</strong>
      <span>Initialize a new repository in the current folder.</span>
    </div>
    <div class="command-item">
      <strong>git add .</strong>
      <span>Stage all modified files for the next commit.</span>
    </div>
    <div class="command-item">
      <strong>git commit -m "message"</strong>
      <span>Commit staged changes with a descriptive message.</span>
    </div>
    <div class="command-item">
      <strong>git status</strong>
      <span>Show the status of changes as untracked, modified, or staged.</span>
    </div>
    <div class="command-item">
      <strong>git push origin main</strong>
      <span>Upload local repository content to a remote repository.</span>
    </div>
  </div>
</section>
<aside role="complementary">
  <h3>Pro Tips & Flags</h3>
  <ul>
    <li><span><b>Undo staging:</b>
       Use <code>git restore --staged &lt;file&gt;</code> to unstage.</span>
    </li>
    <li><span><b>Amend commit:</b>
```

```
            Use <code>git commit --amend</code> to fix the last commit
             message.</span></li>
          <li><span><b>Discard changes:</b>
            Use <code>git checkout .</code> to discard all local
             changes.</span></li>
          <li><span><b>Log history:</b>
            Use <code>git log --oneline</code> for a compact history view.</span>
          </li>
        </ul>
        <div
          style="margin-top:1.0rem;padding:1rem;
            background:#fff4e6;border-radius:8px;border:1px solid #ffe8cc;">
          <h4 style="color:#d9480f;margin-bottom:0.5rem;
            font-size:0.95rem;">💡 Remember .gitignore</h4>
          <p style="font-size:0.85rem;color:#666;">
            Always restart your repo by creating a .gitignore file to
           exclude node_modules or .env files.</p>
        </div>
      </aside>
    </main>
  </div>
</body>
</html>
```

Output



Program No: 02	**Date:** 05-12-2025

Program Title: Implement CSS animations and transitions on webpage elements

```
<!--
CSS Animations
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Mentor Dashboard</title>
  <style>
    body {
      margin: 0;
      font-family: "Segoe UI", Arial, sans-serif;
      background: #eef2f6;
      color: #2c3e50;
      height: 100vh;
      overflow: hidden;
    }

    header {
      background: #ffffff;
      padding: 18px 30px;
      font-size: 26px;
      font-weight: 600;
      box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
      animation: slideDown 1s ease;
    }

    @keyframes slideDown {
      0% {
        transform: translateY(-60px);
        opacity: 0;
      }
```

```
    100% {
      transform: translateY(0);
      opacity: 1;
    }
}

.container {
    display: flex;
    height: 85vh;
}

.sidebar {
    width: 240px;
    background: #ffffff;
    padding: 20px;
    box-shadow: 2px 0 8px rgba(0, 0, 0, 0.05);
    animation: fadeIn 2s;
}

.menu {
    padding: 12px;
    margin-bottom: 12px;
    border-radius: 6px;
    cursor: pointer;
    transition: all 0.3s ease;
}

.menu:hover {
    background: #dce3ea;
    transform: translateX(8px);
}

.content {
    flex: 1;
    padding: 30px;
    animation: fadeIn 2.5s;
}

@keyframes fadeIn {
    from {
```

```
        opacity: 0;
        transform: translateY(20px);
    }

    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.cards {
    display: flex;
    gap: 25px;
    margin-top: 20px;
}

.card {
    background: #ffffff;
    padding: 20px;
    width: 220px;
    border-radius: 8px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.08);
    transition: all 0.4s ease;
}

.card:hover {
    transform: translateY(-12px);
    box-shadow: 0 12px 20px rgba(0, 0, 0, 0.18);
}

button {
    margin-top: 30px;
    padding: 12px 22px;
    border: none;
    background: #5a7fa6;
    color: white;
    border-radius: 6px;
    font-size: 15px;
    cursor: pointer;
    transition: all 0.3s ease;
}
```

```
      button:hover {
        background: #466585;
        transform: scale(1.08);
      }

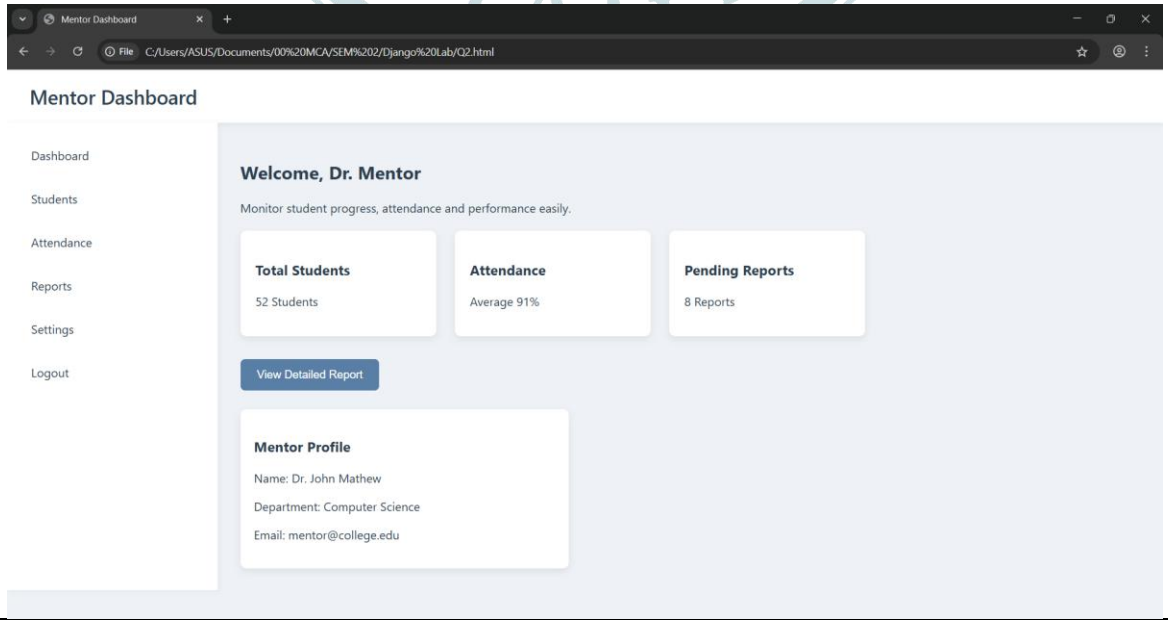
      .profile {
        margin-top: 25px;
        background: white;
        padding: 18px;
        width: 400px;
        border-radius: 8px;
        box-shadow: 0 4px 10px rgba(0, 0, 0, 0.08);
        transition: all 0.4s ease;
      }

      .profile:hover {
        transform: translateX(10px);
        box-shadow: 0 12px 20px rgba(0, 0, 0, 0.18);
      }
    </style>
</head>

<body>
    <header>
      Mentor Dashboard
    </header>
    <div class="container">
      <div class="sidebar">
        <div class="menu">Dashboard</div>
        <div class="menu">Students</div>
        <div class="menu">Attendance</div>
        <div class="menu">Reports</div>
        <div class="menu">Settings</div>
        <div class="menu">Logout</div>
      </div>
      <div class="content">
        <h2>Welcome, Dr. Mentor</h2>
        <p>Monitor student progress, attendance and performance easily.</p>
        <div class="cards">
          <div class="card">
            <h3>Total Students</h3>
```

```
            <p>52 Students</p>
          </div>
          <div class="card">
            <h3>Attendance</h3>
            <p>Average 91%</p>
          </div>
          <div class="card">
            <h3>Pending Reports</h3>
            <p>8 Reports</p>
          </div>
        </div>
        <button>
          View Detailed Report
        </button>
        <div class="profile">
          <h3>Mentor Profile</h3>
          <p>Name: Dr. John Mathew</p>
          <p>Department: Computer Science</p>
          <p>Email: mentor@college.edu</p>
        </div>
      </div>
    </div>
</body>
</html>
```

Output



Program No: 03	**Date:** 05-12-2025

Program Title: Create a webpage demonstrating CSS Flexbox layouts.

```
<!--
Flexbox Layout
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>FoodExpress Flexbox Demo</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #f6f8fa;
      height: 100vh;
      display: flex;
      flex-direction: column;
    }

    header {
      background: #ffffff;
      padding: 15px 25px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
    }

    .brand {
      font-size: 24px;
      font-weight: 600;
      color: #e67e22;
    }
```

```
.menu {
   display: flex;
   gap: 25px;
}

.menu a {
   text-decoration: none;
   color: #2c3e50;
   font-weight: 500;
}

.main {
   flex: 1;
   display: flex;
   gap: 20px;
   padding: 20px;
}

.left {
   flex: 2;
   display: flex;
   flex-direction: column;
   gap: 20px;
}

.food-list {
   display: flex;
   gap: 20px;
}

.food {
   flex: 1;
   background: white;
   padding: 15px;
   border-radius: 8px;
   box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
   display: flex;
   flex-direction: column;
   justify-content: space-between;
}
```

```
.food h3 {
    margin: 0;
}

.food button {
    margin-top: 10px;
    padding: 8px;
    border: none;
    background: #e67e22;
    color: white;
    border-radius: 5px;
}

.offer {
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
    display: flex;
    justify-content: space-between;
    align-items: center;
}

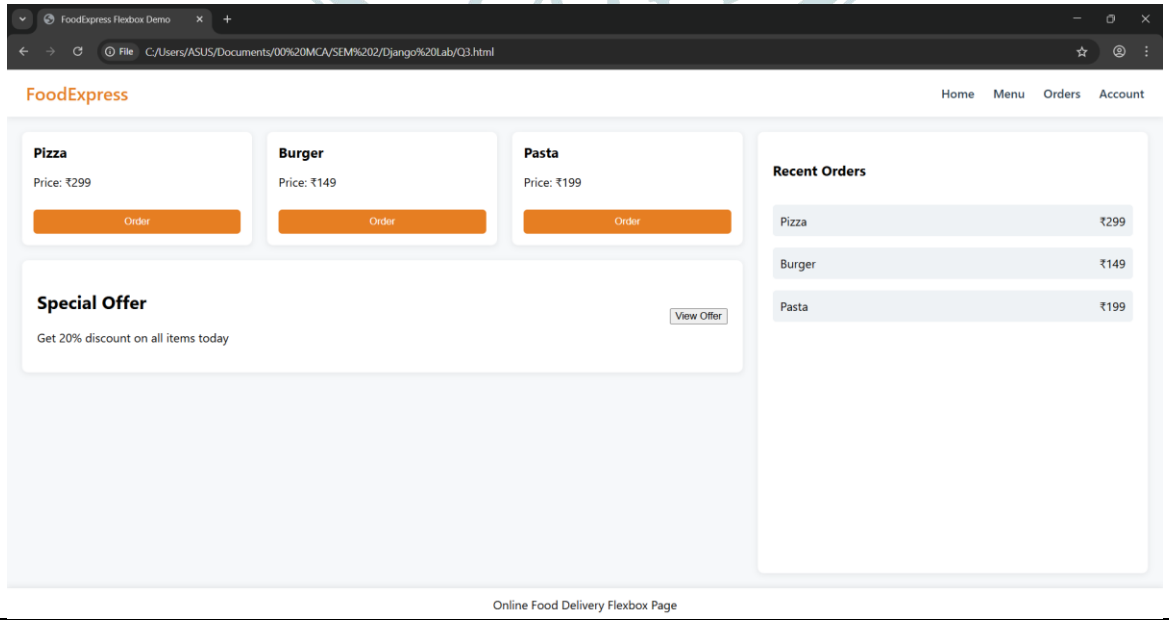
.right {
    flex: 1;
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
    display: flex;
    flex-direction: column;
    gap: 15px;
}

.order {
    display: flex;
    justify-content: space-between;
    background: #eef2f5;
    padding: 10px;
    border-radius: 5px;
}
```

```
      footer {
        background: white;
        text-align: center;
        padding: 10px;
        box-shadow: 0 -2px 8px rgba(0, 0, 0, 0.08);
      }
    </style>
</head>
<body>
    <header>
      <div class="brand">FoodExpress</div>
      <div class="menu">
        <a href="#">Home</a>
        <a href="#">Menu</a>
        <a href="#">Orders</a>
        <a href="#">Account</a>
      </div>
    </header>
    <div class="main">
      <div class="left">
        <div class="food-list">
          <div class="food">
            <h3>Pizza</h3>
            <p>Price: ₹299</p>
            <button>Order</button>
          </div>
          <div class="food">
            <h3>Burger</h3>
            <p>Price: ₹149</p>
            <button>Order</button>
          </div>
          <div class="food">
            <h3>Pasta</h3>
            <p>Price: ₹199</p>
            <button>Order</button>
          </div>
        </div>
        <div class="offer">
          <div>
            <h2>Special Offer</h2>
            <p>Get 20% discount on all items today</p>
```

```
        <button>View Offer</button>
      </div>
    </div>
    <div class="right">
      <h3>Recent Orders</h3>
      <div class="order">
        <span>Pizza</span>
        <span>₹299</span>
      </div>
      <div class="order">
        <span>Burger</span>
        <span>₹149</span>
      </div>
      <div class="order">
        <span>Pasta</span>
        <span>₹199</span>
      </div>
    </div>
  </div>
  <footer>
    Online Food Delivery Flexbox Page
  </footer>
</body>
</html>
```

Output



Program No: 04	**Date:** 05-12-2025

Program Title: Design a pricing table layout using HTML and CSS.

```
<!--
Pricing Table
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Pricing Table</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #f4f7fb;
      height: 100vh;
      display: flex;
      flex-direction: column;
      justify-content: center;
      align-items: center;
    }

    h1 {
      margin-bottom: 20px;
      color: #2c3e50;
    }

    .pricing {
      display: flex;
      gap: 25px;
    }

    .plan {
      background: white;
```

```
    width: 220px;
    padding: 25px;
    border-radius: 10px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
    display: flex;
    flex-direction: column;
    align-items: center;
}

.plan h2 {
    margin: 0;
    color: #34495e;
}

.price {
    font-size: 28px;
    margin: 15px 0;
    color: #27ae60;
    font-weight: bold;
}

.features {
    list-style: none;
    padding: 0;
    margin: 0;
    width: 100%;
}

.features li {
    padding: 8px;
    text-align: center;
    border-bottom: 1px solid #ecf0f1;
}

.features li:last-child {
    border: none;
}

button {
    margin-top: 15px;
    padding: 10px;
```

```
        width: 100%;
        border: none;
        background: #2980b9;
        color: white;
        border-radius: 6px;
        font-size: 15px;
      }

      .highlight {
        transform: scale(1.08);
        background: #2980b9;
        color: white;
      }

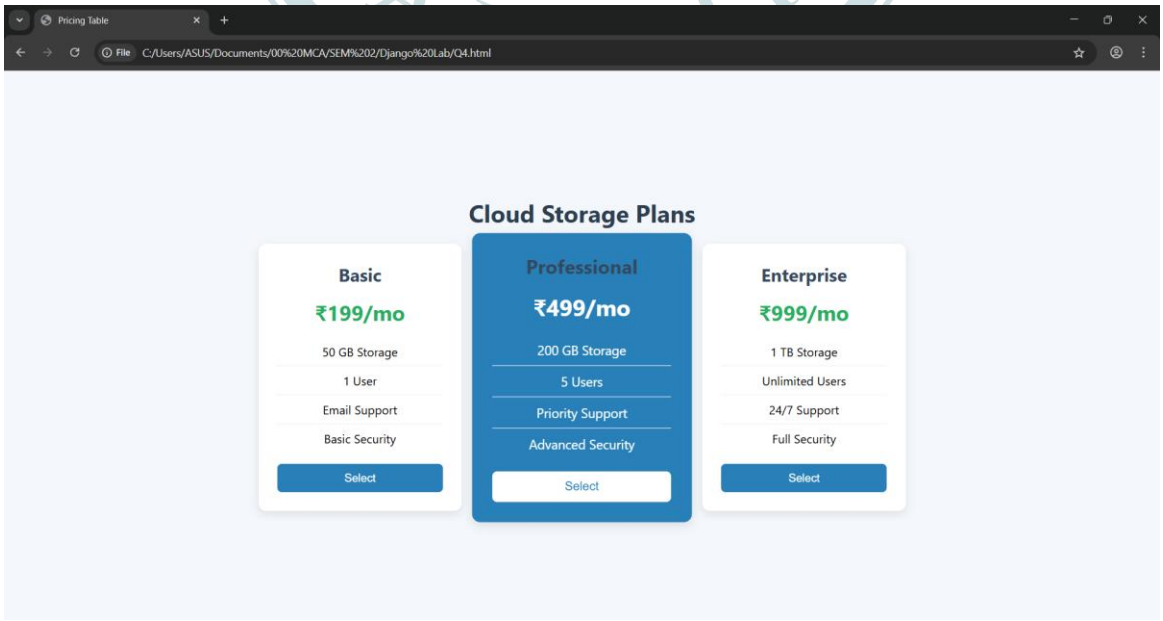
      .highlight .price {
        color: white;
      }

      .highlight button {
        background: white;
        color: #2980b9;
      }
    </style>
</head>

<body>
    <h1>Cloud Storage Plans</h1>
    <div class="pricing">
      <div class="plan">
        <h2>Basic</h2>
        <div class="price">₹199/mo</div>
        <ul class="features">
          <li>50 GB Storage</li>
          <li>1 User</li>
          <li>Email Support</li>
          <li>Basic Security</li>
        </ul>
        <button>Select</button>
      </div>
      <div class="plan highlight">
        <h2>Professional</h2>
```

```
        <div class="price">₹499/mo</div>
        <ul class="features">
          <li>200 GB Storage</li>
          <li>5 Users</li>
          <li>Priority Support</li>
          <li>Advanced Security</li>
        </ul>
        <button>Select</button>
      </div>
      <div class="plan">
        <h2>Enterprise</h2>
        <div class="price">₹999/mo</div>
        <ul class="features">
          <li>1 TB Storage</li>
          <li>Unlimited Users</li>
          <li>24/7 Support</li>
          <li>Full Security</li>
        </ul>
        <button>Select</button>
      </div>
    </div>
</body>
</html>
```

Output



Program No: 05	**Date:** 12-12-2025

Program Title: Design a multi-section landing page using HTML and CSS.

```
<!--
Landing Page
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>TravelGo Landing Page</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #f3f6f9;
      color: #2c3e50;
    }

    header {
      background: white;
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 15px 40px;
      box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
    }

    .logo {
      font-size: 24px;
      font-weight: bold;
      color: #3498db;
    }

    nav {
```

```
    display: flex;
    gap: 25px;
}

nav a {
    text-decoration: none;
    color: #2c3e50;
    font-weight: 500;
}

.hero {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 60px 40px;
    background: #3498db;
    color: white;
}

.hero-text {
    max-width: 500px;
}

.hero-text h1 {
    margin: 0;
    font-size: 36px;
}

.hero-text p {
    margin: 15px 0;
}

.hero-text button {
    padding: 12px 20px;
    border: none;
    background: white;
    color: #3498db;
    border-radius: 6px;
    font-size: 16px;
}
```

```
.services {
    padding: 40px;
    background: #ffffff;
}

.services h2 {
    text-align: center;
    margin-bottom: 25px;
}

.service-box {
    display: flex;
    gap: 20px;
}

.service {
    flex: 1;
    background: #ecf0f1;
    padding: 20px;
    border-radius: 8px;
    text-align: center;
}

.about {
    display: flex;
    gap: 30px;
    padding: 40px;
    align-items: center;
}

.about-text {
    flex: 1;
}

.about-box {
    flex: 1;
    background: #3498db;
    color: white;
    padding: 30px;
    border-radius: 8px;
}
```

```
    .contact {
      background: #2c3e50;
      color: white;
      padding: 30px;
      text-align: center;
    }

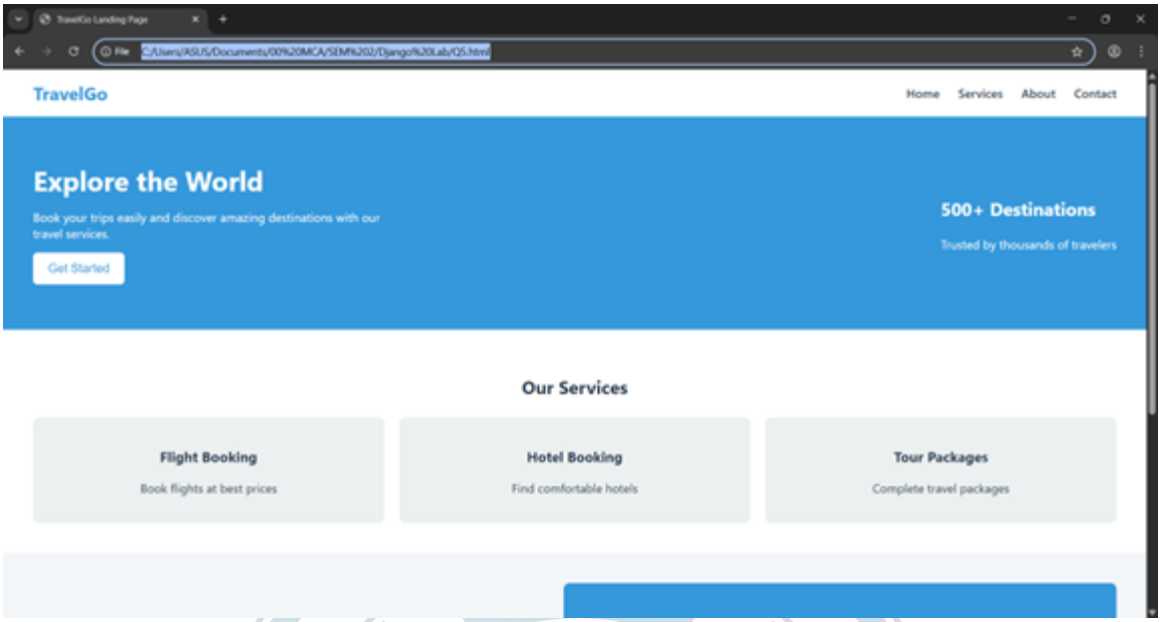
    .contact button {
      margin-top: 10px;
      padding: 10px 18px;
      border: none;
      background: #3498db;
      color: white;
      border-radius: 5px;
    }

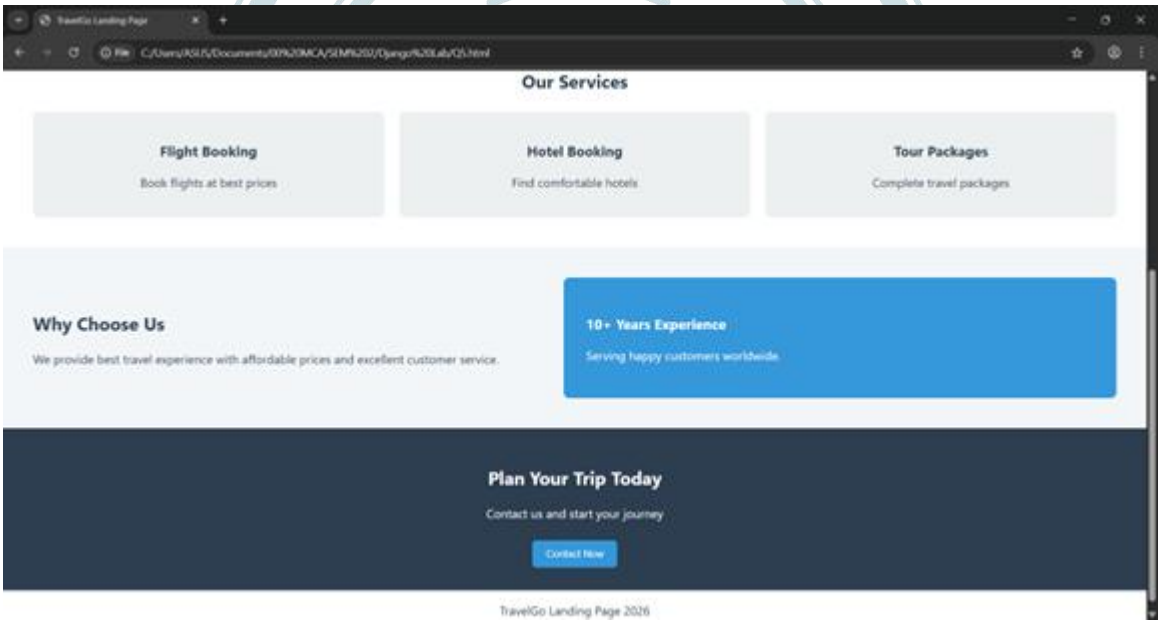
    footer {
      background: white;
      text-align: center;
      padding: 15px;
    }
  </style>
</head>

<body>
  <header>
    <div class="logo">TravelGo</div>
    <nav>
      <a href="#">Home</a>
      <a href="#">Services</a>
      <a href="#">About</a>
      <a href="#">Contact</a>
    </nav>
  </header>
  <section class="hero">
    <div class="hero-text">
      <h1>Explore the World</h1>
      <p>Book your trips easily and discover amazing destinations with our travel services.</p>
      <button>Get Started</button>
    </div>
      <h2>500+ Destinations</h2>
```

```
        <p>Trusted by thousands of travelers</p>
      </div>
    </section>
    <section class="services">
      <h2>Our Services</h2>
      <div class="service-box">
        <div class="service">
          <h3>Flight Booking</h3>
          <p>Book flights at best prices</p>
        </div>
        <div class="service">
          <h3>Hotel Booking</h3>
          <p>Find comfortable hotels</p>
        </div>
        <div class="service">
          <h3>Tour Packages</h3>
          <p>Complete travel packages</p>
        </div>
      </div>
    </section>
    <section class="about">
      <div class="about-text">
        <h2>Why Choose Us</h2>
        <p>We provide best travel experience with affordable prices and excellent customer
service.</p>
      </div>
      <div class="about-box">
        <h3>10+ Years Experience</h3>
        <p>Serving happy customers worldwide.</p>
      </div>
    </section>
    <section class="contact">
      <h2>Plan Your Trip Today</h2>
      <p>Contact us and start your journey</p>
      <button>Contact Now</button>
    </section>
    <footer>
      TravelGo Landing Page 2026
    </footer>
</body>
</html>
```

Output


TravelGo
Home
Services
About
Contact
Explore the World
Book your trips easily and discover amazing destinations with our travel services.
Get Started
500+ Destinations
Trusted by thousands of travelers
Our Services
Flight Booking
Book flights at best prices
Hotel Booking
Find comfortable hotels
Tour Packages
Complete travel packages


Our Services
Flight Booking
Book flights at best prices
Hotel Booking
Find comfortable hotels
Tour Packages
Complete travel packages
Why Choose Us
We provide best travel experience with affordable prices and excellent customer service.
10+ Years Experience
Serving happy customers worldwide
Plan Your Trip Today
Contact us and start your journey
Contact Now
TravelGo Landing Page 2026

Program No: 06	**Date:** 12-12-2025

Program Title: Write a JavaScript program to validate a registration form.

```
<!--
Form Validation
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Registration Form Validation</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #eef2f7;
      height: 100vh;
      display: flex;
    }

    .form-box {
      background: white;
      padding: 25px;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
      width: 320px;
    }

    input {
      width: 100%;
      padding: 10px;
      margin-top: 8px;
      margin-bottom: 12px;
      box-sizing:border-box;
    }
```

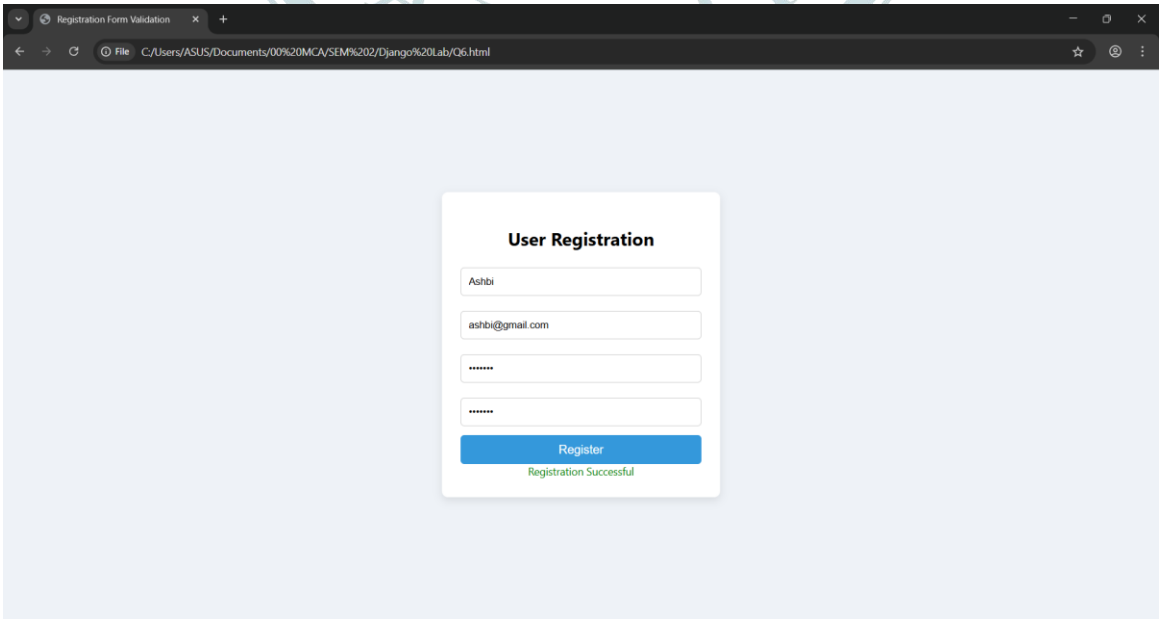
```
    button {
      width: 100%;
      padding: 10px;
      border: none;
      background: #3498db;
      color: white;
      border-radius: 5px;
      font-size: 16px;
    }

    .error {
      color: red;
      font-size: 14px;
      text-align: center;
    }

    .success {
      color: green;
      font-size: 14px;
      text-align: center;
    }
  </style>
</head>
<body>
  <div class="form-box">
    <h2>User Registration</h2>
    <form onsubmit="return validateForm()">
      <input type="text" id="name" placeholder="Full Name">
      <input type="email" id="email" placeholder="Email">
      <input type="password" id="password" placeholder="Password">
      <input type="password" id="confirm" placeholder="Confirm Password">
      <button type="submit">Register</button>
      <div id="message"></div>
    </form>
  </div>
  <script>
    function validateForm() {
      var name = document.getElementById("name").value;
      var email = document.getElementById("email").value;
      var password = document.getElementById("password").value;
      var confirm = document.getElementById("confirm").value;
```

```
        var message = document.getElementById("message");
        if (name == "" || email == "" || password == "" || confirm == "") {
            message.innerHTML = "All fields are required";
            message.className = "error";
            return false;
        }
        if (password.length < 6) {
            message.innerHTML = "Password must be at least 6 characters";
            message.className = "error";
            return false;
        }
        if (password != confirm) {
            message.innerHTML = "Passwords do not match";
            message.className = "error";
            return false;
        }
        message.innerHTML = "Registration Successful";
        message.className = "success";
        return false;
    }
  </script>
</body>
</html>
```

Output



Program No: 07	**Date:** 12-12-2025

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Data Filtering
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dynamic Search Filter</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #f0f3f7;
      height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
    }

    .container {
      background: white;
      padding: 25px;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
      width: 400px;
    }

    input {
      width: 100%;
      padding: 10px;
      margin-bottom: 15px;
      border: 1px solid #ccc;
      border-radius: 5px;
```

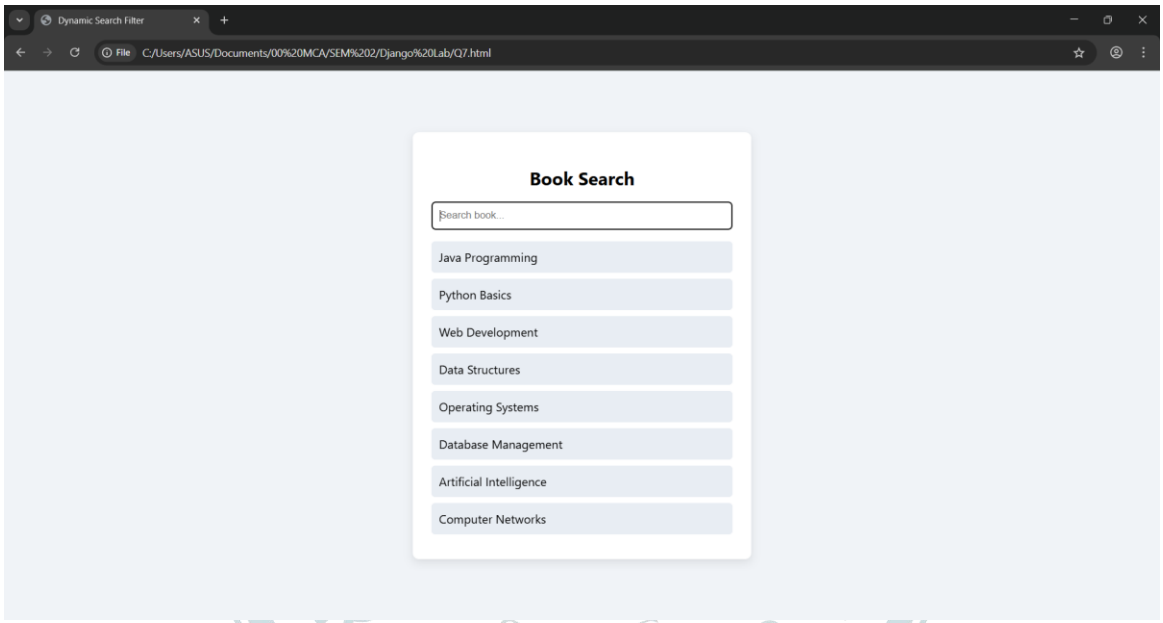
```
        box-sizing: border-box;
    }

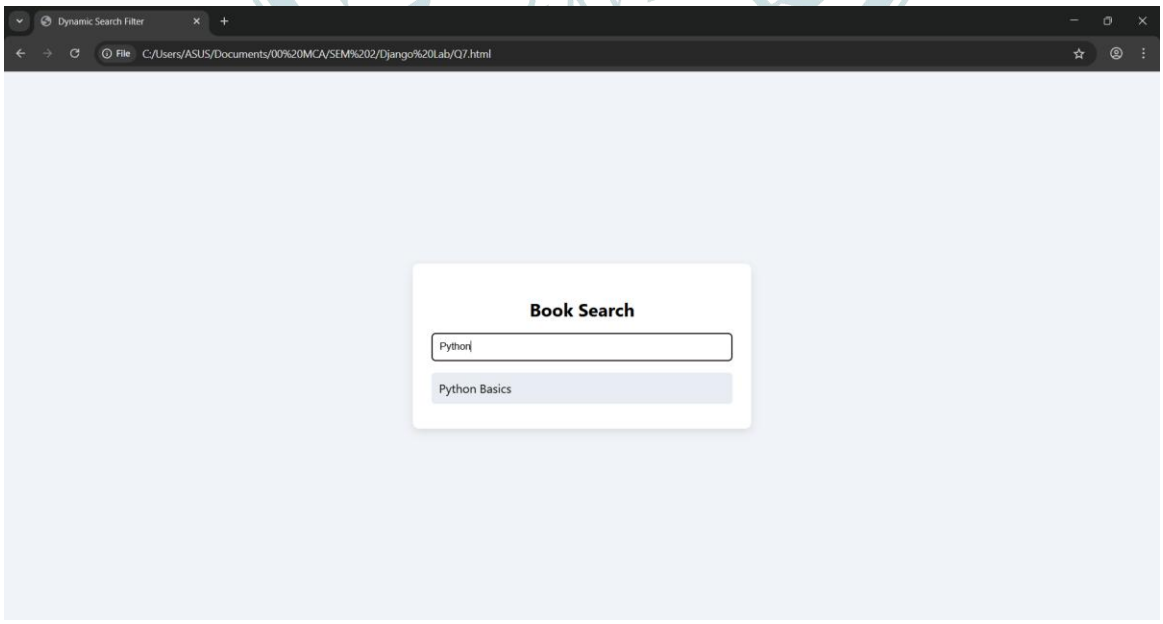
    ul {
        list-style: none;
        padding: 0;
        margin: 0;
    }

    li {
        padding: 10px;
        margin-bottom: 8px;
        background: #e8edf3;
        border-radius: 5px;
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>Book Search</h2>
    <input type="text" id="search" placeholder="Search book..." onkeyup="filterList()">
    <ul id="bookList">
        <li>Java Programming</li>
        <li>Python Basics</li>
        <li>Web Development</li>
        <li>Data Structures</li>
        <li>Operating Systems</li>
        <li>Database Management</li>
        <li>Artificial Intelligence</li>
        <li>Computer Networks</li>
    </ul>
  </div>
  <script>
    function filterList() {
        var input = document.getElementById("search").value.toLowerCase();
        var ul = document.getElementById("bookList");
        var li = ul.getElementsByTagName("li");
        for (var i = 0; i < li.length; i++) {
            var text = li[i].innerText.toLowerCase();
            if (text.includes(input)) {
                li[i].style.display = "";
```

```
            } else {
                li[i].style.display = "none";
            }
        }
    }
  </script>
</body>
</html>
```

Output





Program No: 08	**Date:** 12-12-2025

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Image Slideshow
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Image Slideshow Grid</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #eef1f5;
      height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
    }

    .gallery {
      display: grid;
      grid-template-columns: 1fr;
      grid-template-rows: auto 60px;
      background: white;
      padding: 15px;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
      width: 500px;
    }

    .slideshow {
      grid-row: 1;
```

```
        position: relative;
    }

    .slideshow img {
        width: 100%;
        height: 300px;
        object-fit: cover;
        border-radius: 6px;
    }

    .controls {
        grid-row: 2;
        display: grid;
        grid-template-columns: 1fr 1fr;
        gap: 10px;
        margin-top: 10px;
    }

    button {
        padding: 10px;
        border: none;
        background: #2e86c1;
        color: white;
        border-radius: 5px;
        font-size: 16px;
    }

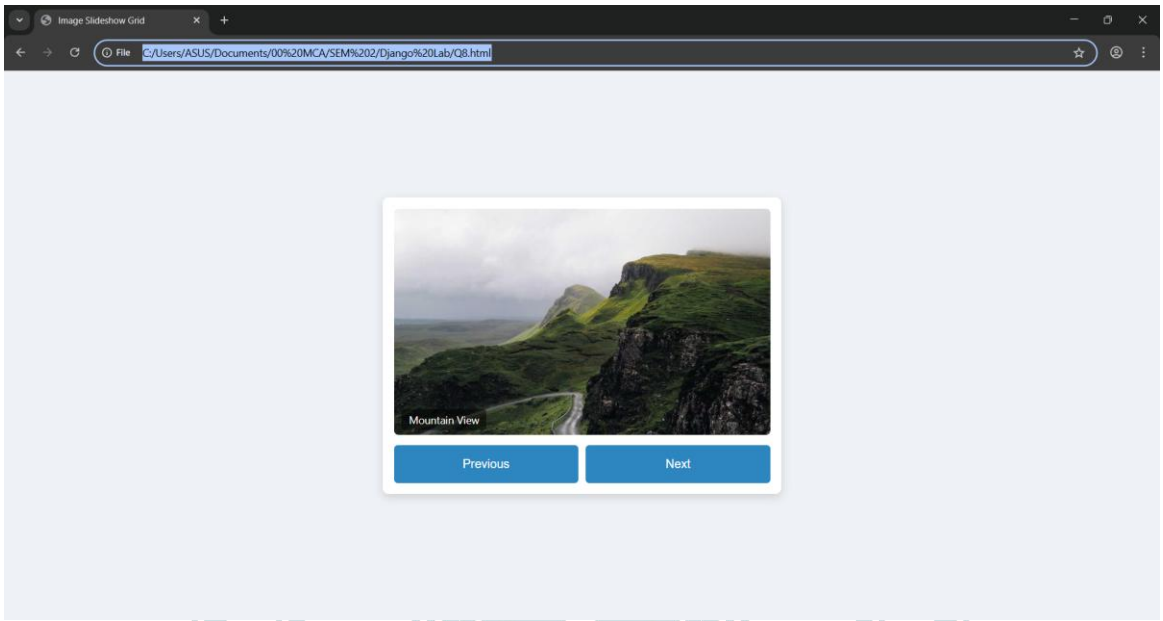
    .title {
        position: absolute;
        bottom: 10px;
        left: 10px;
        background: rgba(0, 0, 0, 0.5);
        color: white;
        padding: 5px 10px;
        border-radius: 4px;
        font-size: 14px;
    }
  </style>
</head>

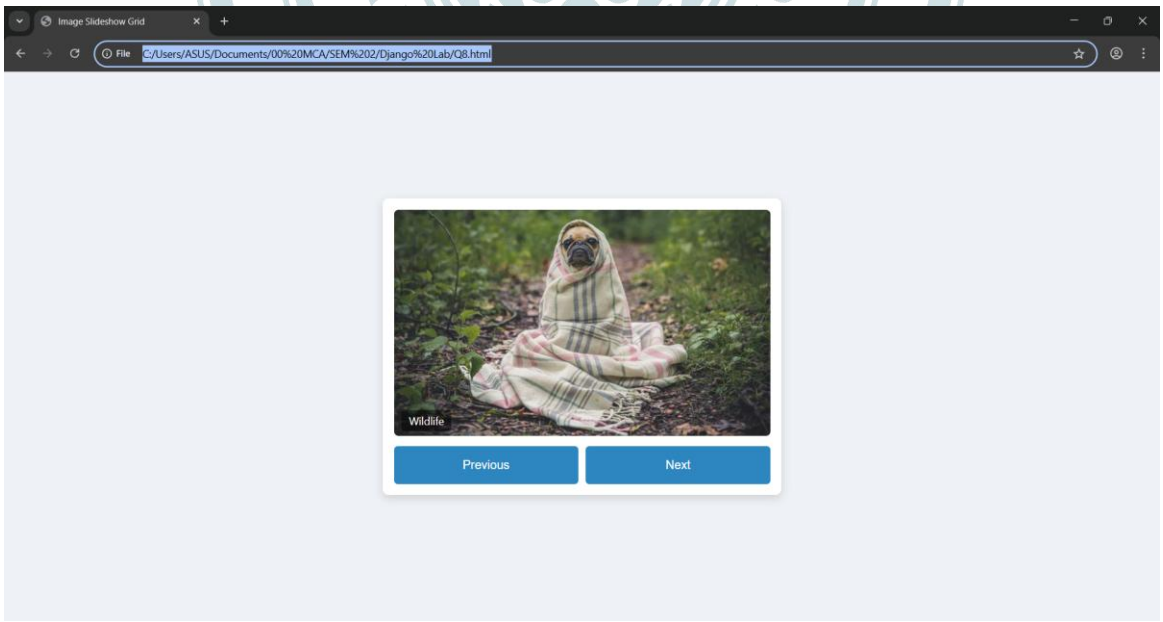
<body>
```

```
<div class="gallery">
  <div class="slideshow">
    <img id="slide" src="https://picsum.photos/id/1018/600/400">
    <div class="title" id="caption">Mountain View</div>
  </div>
  <div class="controls">
    <button onclick="prevSlide()">Previous</button>
    <button onclick="nextSlide()">Next</button>
  </div>
</div>
<script>
  var images = [
    "https://picsum.photos/id/1018/600/400",
    "https://picsum.photos/id/1025/600/400",
    "https://picsum.photos/id/1039/600/400",
    "https://picsum.photos/id/1043/600/400"
  ];
  var captions = [
    "Mountain View",
    "Wildlife",
    "Forest Path",
    "Beach Scene"
  ];
  var index = 0;
  function showSlide() {
    document.getElementById("slide").src = images[index];
    document.getElementById("caption").innerHTML = captions[index];
  }
  function nextSlide() {
    index++;
    if (index >= images.length) {
      index = 0;
    }
    showSlide();
  }
  function prevSlide() {
    index--;
    if (index < 0) {
      index = images.length - 1;
    }
    showSlide();
```

```
    }
    setInterval(nextSlide, 3000);
  </script>
</body>
</html>
```

Output





Program No: 09	**Date:** 19-12-2025

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Digital Clock
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Digital Clock</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #0f172a;
      height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      color: white;
    }

    .clock-box {
      background: #1e293b;
      padding: 30px 50px;
      border-radius: 10px;
      box-shadow: 0 4px 15px rgba(0, 0, 0, 0.3);
      text-align: center;
    }

    #time {
      font-size: 48px;
      letter-spacing: 3px;
      margin-bottom: 10px;
```

```
    }

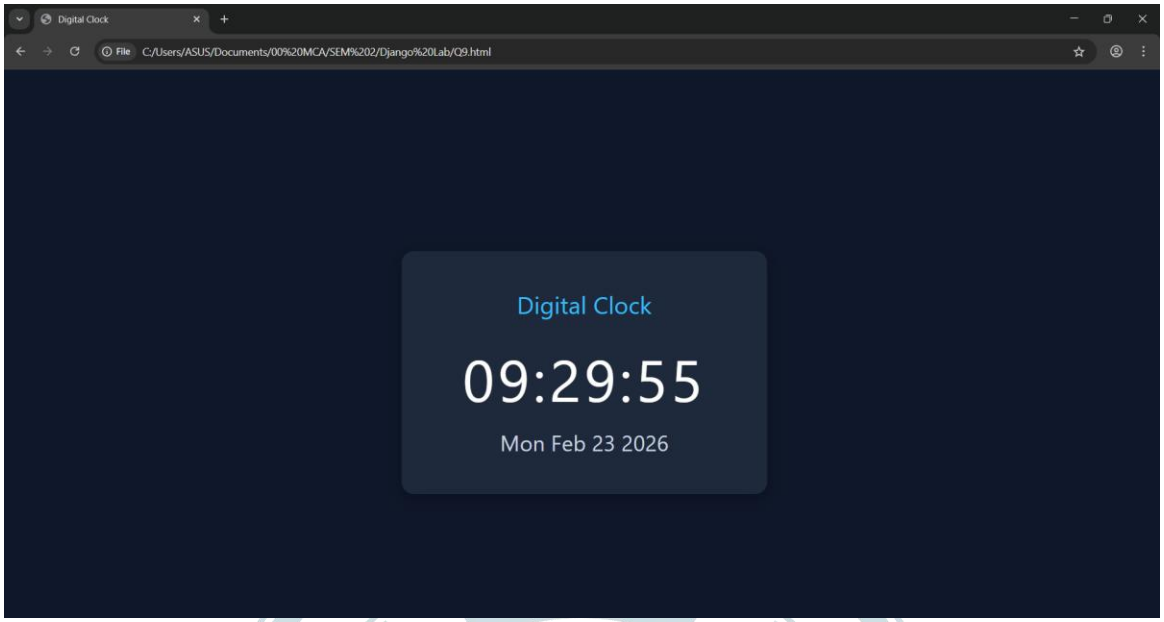
    #date {
      font-size: 18px;
      color: #cbd5e1;
    }

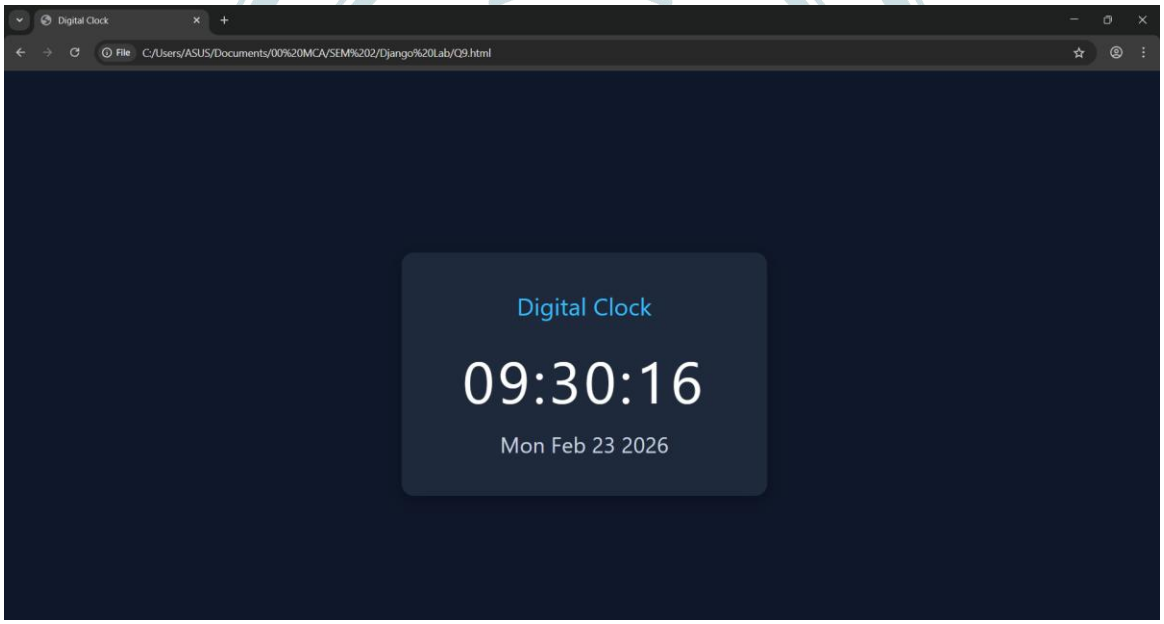
    .title {
      margin-bottom: 15px;
      font-size: 20px;
      color: #38bdf8;
    }
  </style>
</head>

<body>
  <div class="clock-box">
    <div class="title">Digital Clock</div>
    <div id="time"></div>
    <div id="date"></div>
  </div>
  <script>
    function updateClock() {
      var now = new Date();
      var h = now.getHours();
      var m = now.getMinutes();
      var s = now.getSeconds();
      var d = now.toDateString();
      if (h < 10) { h = "0" + h; }
      if (m < 10) { m = "0" + m; }
      if (s < 10) { s = "0" + s; }
      document.getElementById("time").innerHTML = h + ":" + m + ":" + s;
      document.getElementById("date").innerHTML = d;
    }
    setInterval(updateClock, 1000);
    updateClock();
  </script>
</body>

</html>
```

Output


Digital Clock
File C:/Users/ASUS/Documents/00%20MCA/SEM%202/Django%20Lab/Q9.html
Digital Clock
09:29:55
Mon Feb 23 2026


Digital Clock
File C:/Users/ASUS/Documents/00%20MCA/SEM%202/Django%20Lab/Q9.html
Digital Clock
09:30:16
Mon Feb 23 2026

Program No: 10	**Date:** 19-12-2025

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Dropdown Rendering
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Dropdown Content Rendering</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #f1f5f9;
      height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
    }

    .container {
      background: white;
      padding: 25px;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
      width: 400px;
      text-align: center;
    }

    h2 {
      margin-bottom: 15px;
    }
```

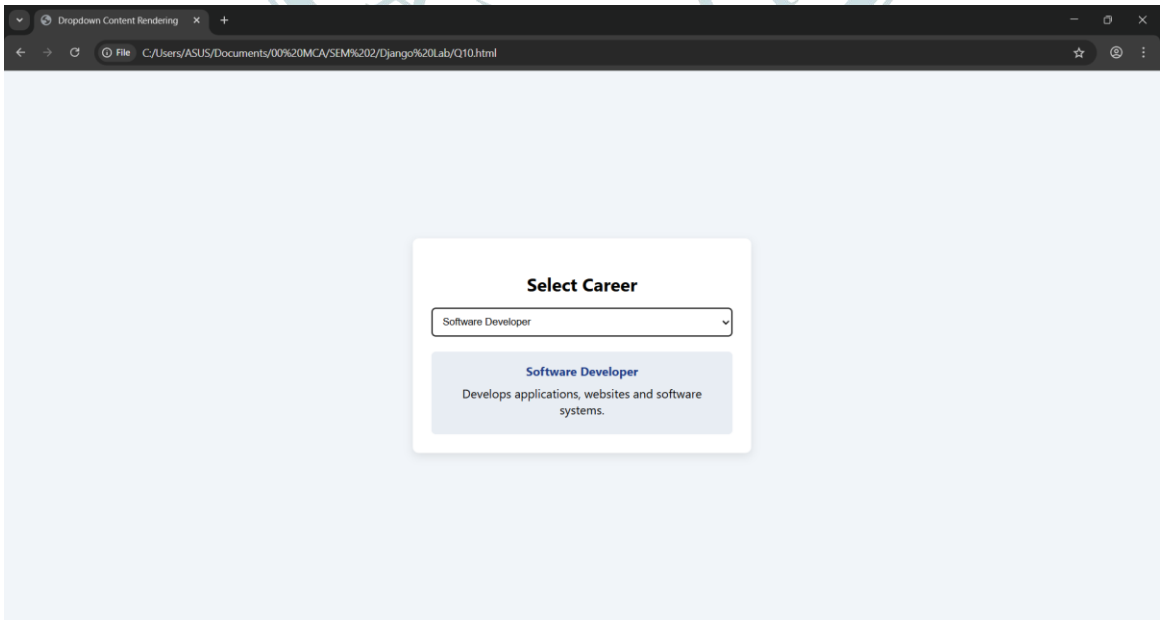
```
    select {
      width: 100%;
      padding: 10px;
      border: 1px solid #ccc;
      border-radius: 5px;
      margin-bottom: 20px;
    }

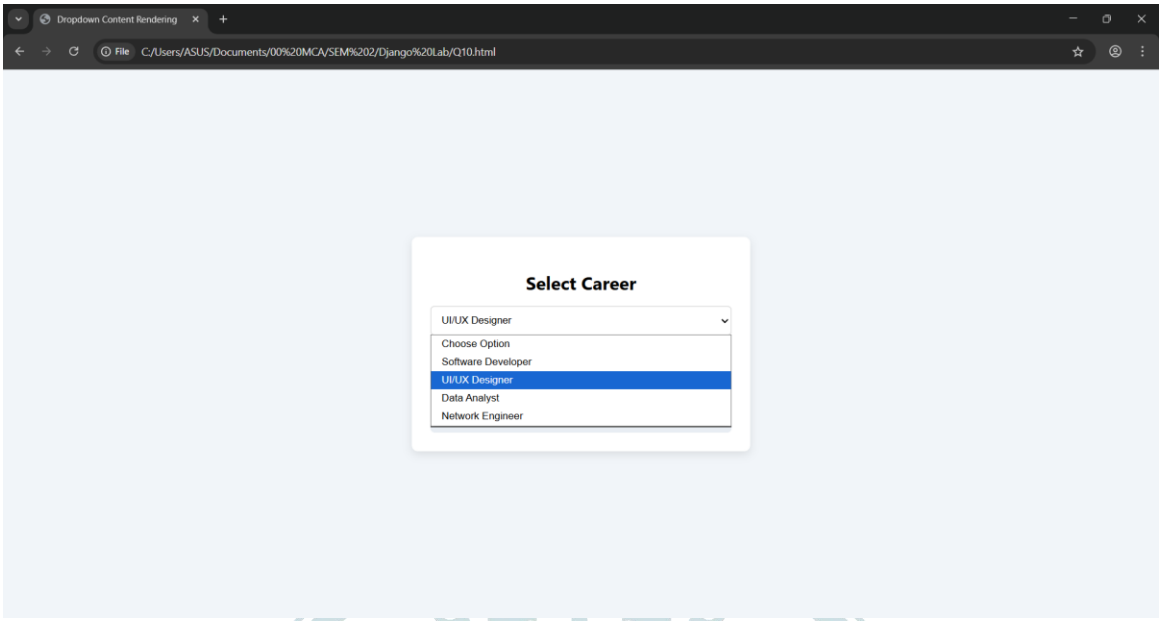
    .content {
      background: #e8edf3;
      padding: 15px;
      border-radius: 6px;
      min-height: 80px;
    }

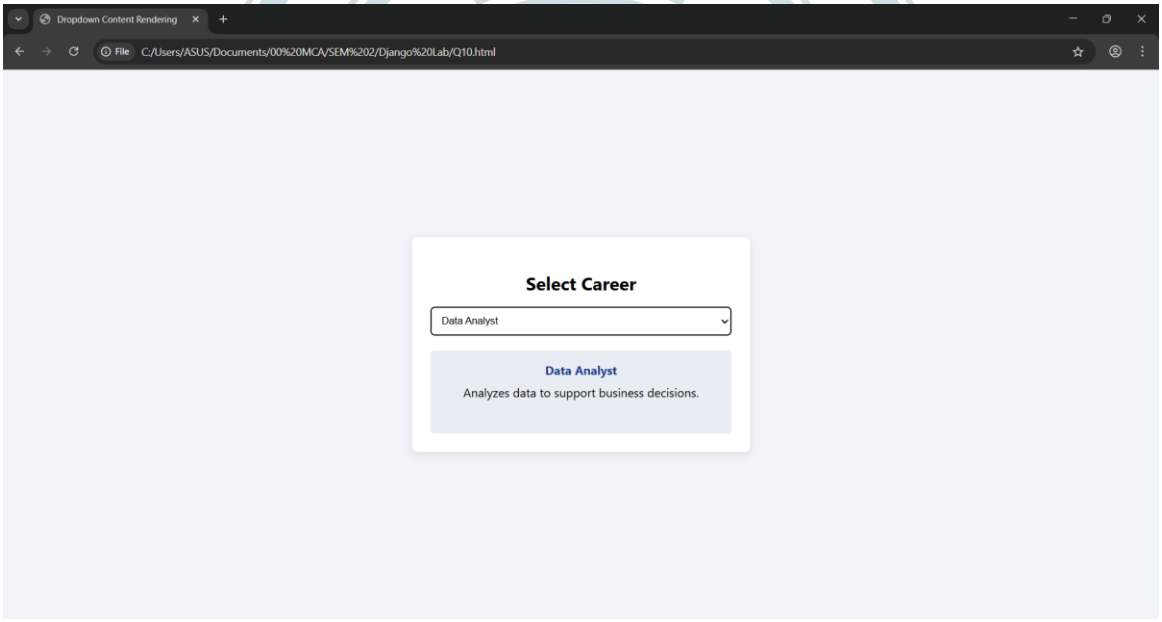
    .title {
      font-weight: bold;
      margin-bottom: 8px;
      color: #1e3a8a;
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>Select Career</h2>
    <select onchange="showContent()" id="career">
      <option value="">Choose Option</option>
      <option value="developer">Software Developer</option>
      <option value="designer">UI/UX Designer</option>
      <option value="analyst">Data Analyst</option>
      <option value="network">Network Engineer</option>
    </select>
    <div class="content" id="output">
      Please select a career to view details.
    </div>
  </div>
  <script>
    function showContent() {
      var value = document.getElementById("career").value;
      var text = "";
      if (value == "developer") {
```

```
            text = "<div class='title'>Software Developer</div>Develops applications, websites and software systems.";
        }
        else if (value == "designer") {
            text = "<div class='title'>UI/UX Designer</div>Designs user interfaces and improves user experience.";
        }
        else if (value == "analyst") {
            text = "<div class='title'>Data Analyst</div>Analyzes data to support business decisions.";
        }
        else if (value == "network") {
            text = "<div class='title'>Network Engineer</div>Manages and maintains computer networks.";
        }
        else {
            text = "Please select a career to view details.";
        }
        document.getElementById("output").innerHTML = text;
    }
  </script>
</body>
</html>
```

Output




Dropdown Content Rendering
File C:/Users/ASUS/Documents/00%20MCA/SEM%202/Django%20Lab/Q10.html
Select Career
UI/UX Designer
Choose Option
Software Developer
UI/UX Designer
Data Analyst
Network Engineer


Dropdown Content Rendering
File C:/Users/ASUS/Documents/00%20MCA/SEM%202/Django%20Lab/Q10.html
Select Career
Data Analyst
Data Analyst
Analyzes data to support business decisions.

Program No: 11	**Date:** 19-12-2025

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
JSON Display
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Student Details JSON</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      background: #eef3f7;
      height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
    }

    .container {
      background: white;
      padding: 25px;
      border-radius: 8px;
      box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
      width: 350px;
      text-align: center;
    }

    h2 {
      margin-bottom: 15px;
    }
```

```
    table {
      width: 100%;
      border-collapse: collapse;
    }

    td {
      padding: 10px;
      border-bottom: 1px solid #ddd;
      text-align: left;
    }

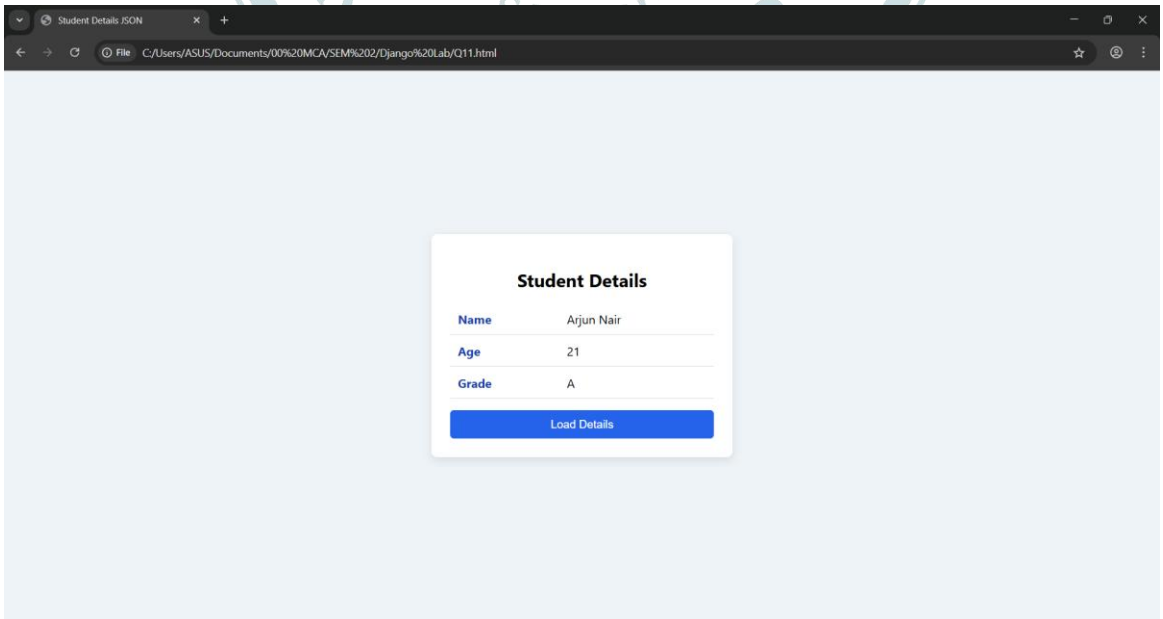
    .label {
      font-weight: bold;
      color: #1e40af;
    }

    button {
      margin-top: 15px;
      padding: 10px;
      width: 100%;
      border: none;
      background: #2563eb;
      color: white;
      border-radius: 5px;
      font-size: 15px;
    }
  </style>
</head>

<body>
  <div class="container">
    <h2>Student Details</h2>
    <table>
      <tr>
        <td class="label">Name</td>
        <td id="name"></td>
      </tr>
      <tr>
        <td class="label">Age</td>
        <td id="age"></td>
      </tr>
```

```
        <tr>
          <td class="label">Grade</td>
          <td id="grade"></td>
        </tr>
      </table>
      <button onclick="showStudent()">Load Details</button>
    </div>
    <script>
      function showStudent() {
        var student = {
          "name": "Arjun Nair",
          "age": 21,
          "grade": "A"
        };
        document.getElementById("name").innerHTML = student.name;
        document.getElementById("age").innerHTML = student.age;
        document.getElementById("grade").innerHTML = student.grade;
      }
    </script>
</body>
</html>
```

Output



Program No: 12	**Date:** 09-01-2026

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Weather App
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Live Weather</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      background: linear-gradient(135deg, #4facfe, #00f2fe);
      color: #ffffff;
    }

    .card {
      background: rgba(255, 255, 255, 0.15);
      backdrop-filter: blur(10px);
      padding: 30px;
      border-radius: 12px;
      width: 380px;
      box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);
      text-align: center;
    }

    .title {
      font-size: 24px;
```

```
    margin-bottom: 20px;
    font-weight: 600;
}

.search {
    display: flex;
    gap: 10px;
    margin-bottom: 20px;
}

.search input {
    flex: 1;
    padding: 10px;
    border: none;
    border-radius: 6px;
    outline: none;
}

.search button {
    padding: 10px 15px;
    border: none;
    border-radius: 6px;
    background: #ffffff;
    color: #0984e3;
    font-weight: 600;
}

.weather {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 15px;
}

.box {
    background: rgba(255, 255, 255, 0.2);
    padding: 12px;
    border-radius: 8px;
}

.label {
    font-size: 13px;
```

```
      opacity: 0.8;
    }

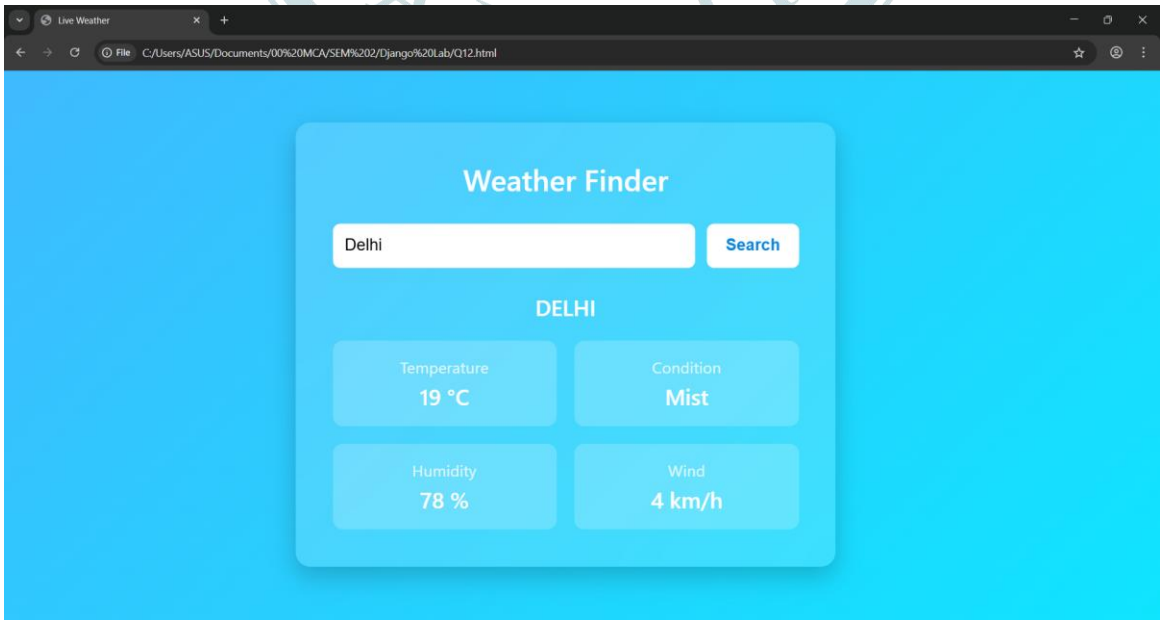
    .value {
      font-size: 18px;
      font-weight: 600;
      margin-top: 4px;
    }

    .city {
      margin-bottom: 15px;
      font-size: 18px;
      font-weight: 500;
    }
  </style>
</head>

<body>
  <div class="card">
    <div class="title">Weather Finder</div>
    <div class="search">
      <input type="text" id="city" placeholder="Enter city">
      <button onclick="getWeather()">Search</button>
    </div>
    <div class="city" id="cityName">--</div>
    <div class="weather">
      <div class="box">
        <div class="label">Temperature</div>
        <div class="value" id="temp">--</div>
      </div>
      <div class="box">
        <div class="label">Condition</div>
        <div class="value" id="condition">--</div>
      </div>
      <div class="box">
        <div class="label">Humidity</div>
        <div class="value" id="humidity">--</div>
      </div>
      <div class="box">
        <div class="label">Wind</div>
        <div class="value" id="wind">--</div>
```

```
        </div>
      </div>
    </div>
    <script>
      function getWeather() {
        var city = document.getElementById("city").value;
        document.getElementById("cityName").innerHTML = city;
        fetch("https://wttr.in/" + city + "?format=j1")
          .then(r => r.json())
          .then(d => {
            document.getElementById("temp").innerHTML =
                d.current_condition[0].temp_C + " °C";
            document.getElementById("condition").innerHTML =
                d.current_condition[0].weatherDesc[0].value;
            document.getElementById("humidity").innerHTML =
                d.current_condition[0].humidity + " %";
            document.getElementById("wind").innerHTML =
                d.current_condition[0].windspeedKmph + " km/h";
          });
      }
    </script>
</body>
</html>
```

Output



Program No: 13	**Date:** 09-01-2026

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Theme Changer
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Control Panel Theme</title>
  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial, sans-serif;
      transition: all 0.4s;
    }

    .panel {
      width: 420px;
      margin: 80px auto;
      padding: 25px;
      border-radius: 12px;
      box-shadow: 0 8px 20px rgba(0, 0, 0, 0.15);
      text-align: center;
    }

    .title {
      font-size: 24px;
      margin-bottom: 10px;
      font-weight: 600;
    }

    .subtitle {
      font-size: 14px;
```

```
    margin-bottom: 20px;
    opacity: 0.7;
}

.colors {
    display: flex;
    justify-content: center;
    gap: 15px;
    margin-bottom: 25px;
}

.colors div {
    width: 35px;
    height: 35px;
    border-radius: 50%;
    cursor: pointer;
    border: 3px solid white;
    box-shadow: 0 0 0 2px rgba(0, 0, 0, 0.2);
}

.preview {
    padding: 20px;
    border-radius: 8px;
    margin-top: 10px;
    font-size: 16px;
}

.default {
    background: #f1f5f9;
    color: #0f172a;
}

.default .panel {
    background: white;
}

.default .preview {
    background: #e2e8f0;
}
```

LEARN • SERVE • EXCEL RAJAGIRI

```
    .night {
      background: #020617;
      color: #e2e8f0;
    }

    .night .panel {
      background: #0f172a;
    }

    .night .preview {
      background: #1e293b;
    }

    .sunset {
      background: #fff7ed;
      color: #7c2d12;
    }

    .sunset .panel {
      background: #ffedd5;
    }

    .sunset .preview {
      background: #fed7aa;
    }

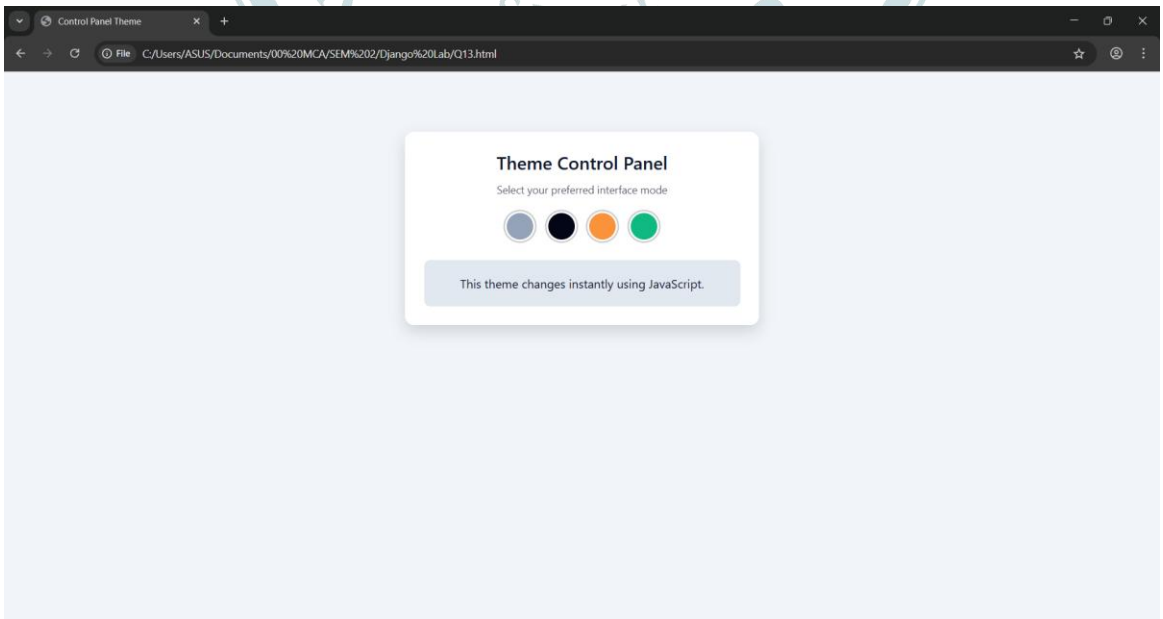
    .forest {
      background: #ecfdf5;
      color: #064e3b;
    }

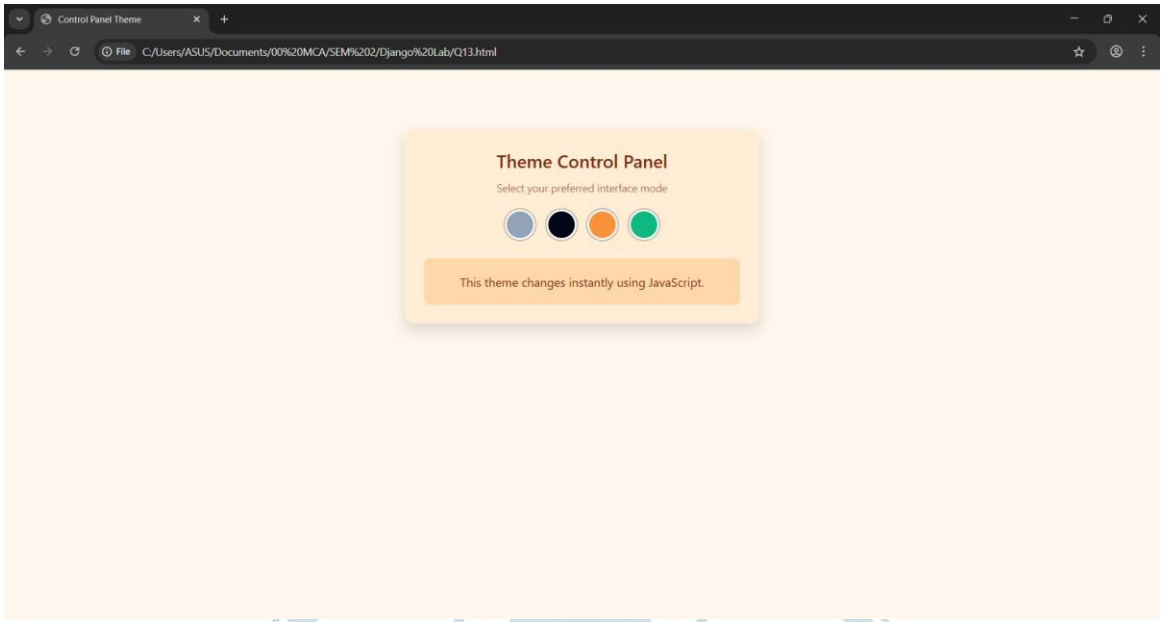
    .forest .panel {
      background: #d1fae5;
    }

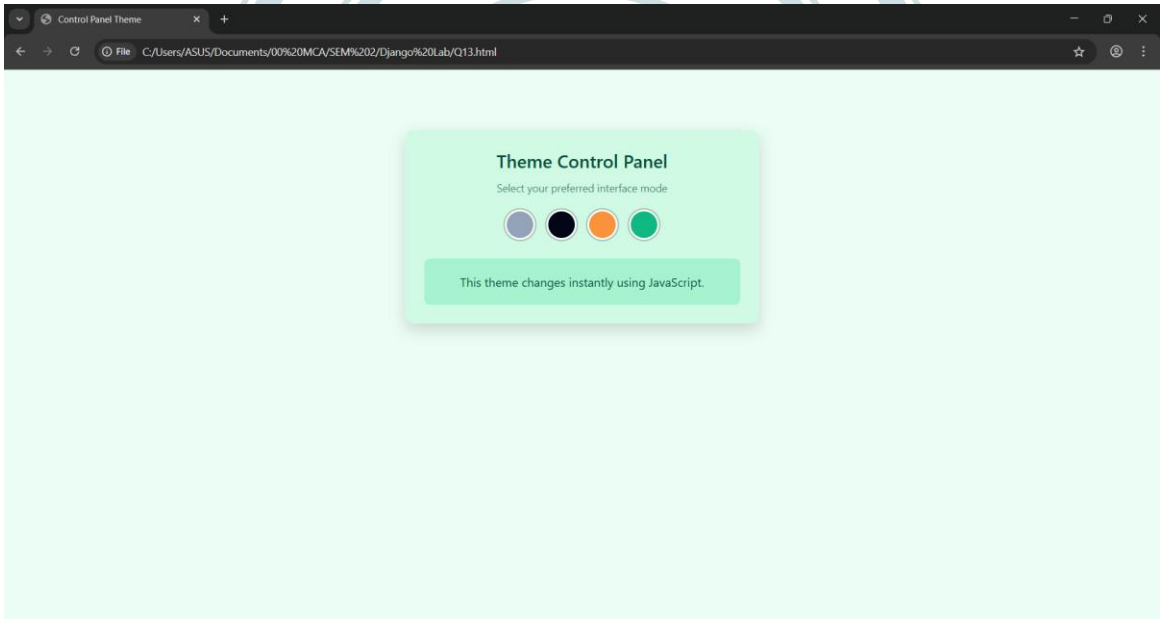
    .forest .preview {
      background: #a7f3d0;
    }
  </style>
</head>
```

```
<body class="default">
  <div class="panel">
    <div class="title">Theme Control Panel</div>
    <div class="subtitle">Select your preferred interface mode</div>
    <div class="colors">
      <div style="background:#94a3b8" onclick="setTheme('default')"></div>
      <div style="background:#020617" onclick="setTheme('night')"></div>
      <div style="background:#fb923c" onclick="setTheme('sunset')"></div>
      <div style="background:#10b981" onclick="setTheme('forest')"></div>
    </div>
    <div class="preview">
      This theme changes instantly using JavaScript.
    </div>
  </div>
  <script>
    function setTheme(t) {
      document.body.className = t;
    }
  </script>
</body>
</html>
```

Output




Control Panel Theme
File C:/Users/ASUS/Documents/00%20MCA/SEM%202/Django%20Lab/Q13.html
Theme Control Panel
Select your preferred interface mode
This theme changes instantly using JavaScript.


Control Panel Theme
File C:/Users/ASUS/Documents/00%20MCA/SEM%202/Django%20Lab/Q13.html
Theme Control Panel
Select your preferred interface mode
This theme changes instantly using JavaScript.

Program No: 14	**Date:** 09-01-2026

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Bootstrap Forms
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Bootstrap Auth Page</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      min-height: 100vh;
      background: linear-gradient(135deg, #667eea, #764ba2);
      display: flex;
      align-items: center;
    }

    .auth-box {
      background: white;
      border-radius: 12px;
      overflow: hidden;
      box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
    }

    .left-side {
      background: linear-gradient(135deg, #4facfe, #00f2fe);
      color: white;
      display: flex;
      flex-direction: column;
      justify-content: center;
```

```
        align-items: center;
        padding: 40px;
        height: 100%;
      }

      .left-side h2 {
        font-weight: 700;
      }

      .form-control {
        border-radius: 8px;
      }

      .btn-custom {
        border-radius: 8px;
        padding: 10px;
        font-weight: 600;
      }

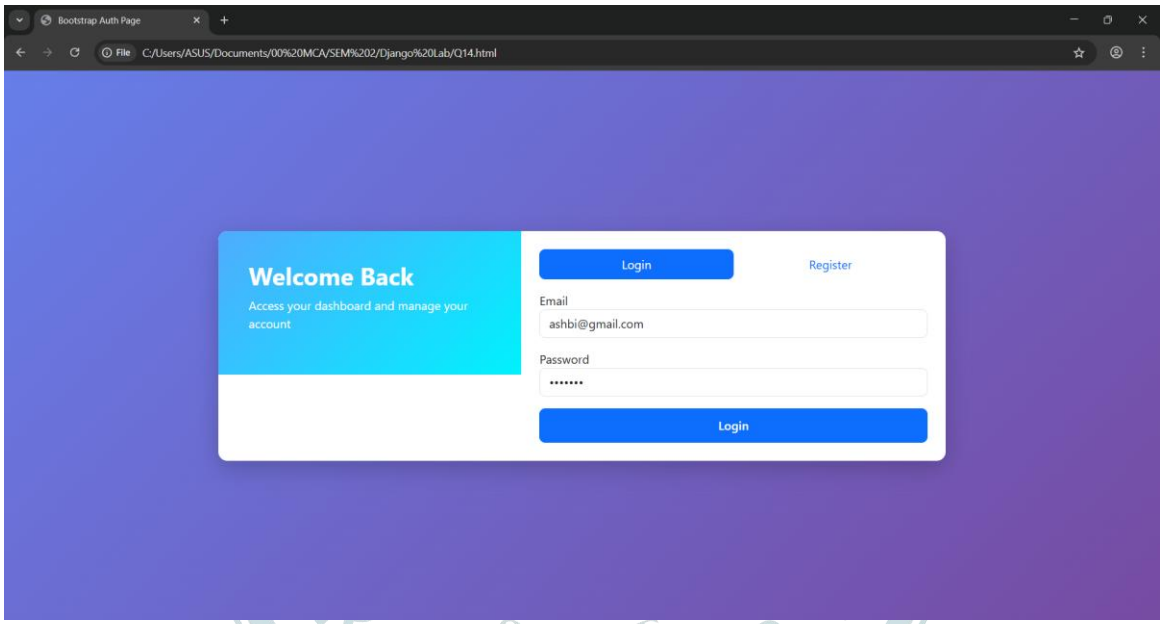
      .nav-pills .nav-link {
        border-radius: 8px;
      }
    </style>
</head>

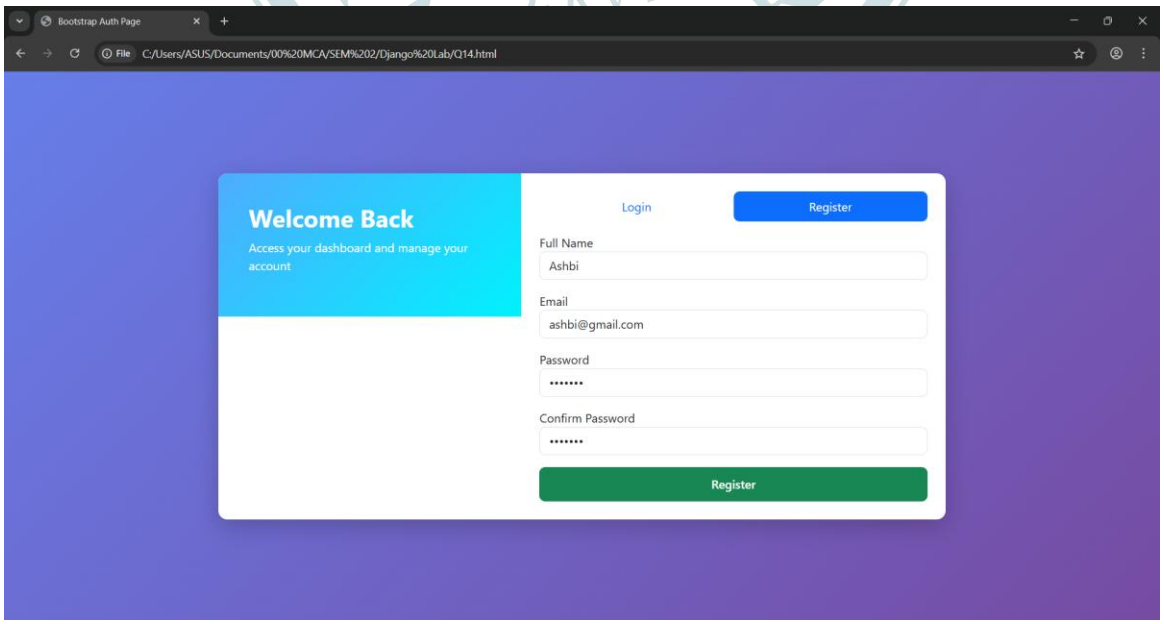
<body>
    <div class="container">
      <div class="row justify-content-center">
        <div class="col-lg-9">
          <div class="auth-box row g-0">
            <div class="col-md-5 d-none d-md-flex left-side">
              <div>
                <h2>Welcome Back</h2>
                <p>Access your dashboard and manage your account</p>
              </div>
            </div>
            <div class="col-md-7 p-4">
              <ul class="nav nav-pills nav-justified mb-3">
                <li class="nav-item">
                  <button class="nav-link active" data-bs-toggle="pill"
                    data-bs-target="#login">Login</button>
```

```
            </li>
            <li class="nav-item">
              <button class="nav-link" data-bs-toggle="pill"
                data-bs-target="#register">Register</button>
            </li>
          </ul>
          <div class="tab-content">
            <div class="tab-pane fade show active" id="login">
              <div class="mb-3">
                <label>Email</label>
                <input type="email" class="form-control">
              </div>
              <div class="mb-3">
                <label>Password</label>
                <input type="password" class="form-control">
              </div>
              <div class="d-grid">
                <button class="btn btn-primary btn-custom">Login</button>
              </div>
            </div>
            <div class="tab-pane fade" id="register">
              <div class="mb-3">
                <label>Full Name</label>
                <input type="text" class="form-control">
              </div>
              <div class="mb-3">
                <label>Email</label>
                <input type="email" class="form-control">
              </div>
              <div class="mb-3">
                <label>Password</label>
                <input type="password" class="form-control">
              </div>
              <div class="mb-3">
                <label>Confirm Password</label>
                <input type="password" class="form-control">
              </div>
              <div class="d-grid">
                <button class="btn btn-success btn-custom">Register</button>
              </div>
            </div>
```

```
        </div>
      </div>
    </div>
  </div>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Output





Program No: 15	**Date:** 09-01-2026

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Material Design
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Material Dashboard</title>
  <link
href="https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;500&display=swap"
rel="stylesheet">
  <link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet">
  <link href="https://unpkg.com/material-components-web@latest/dist/material-components-
web.min.css" rel="stylesheet">

  <style>
    body {
      margin: 0;
      font-family: Roboto;
      background: #f4f6fb;
    }

    .mdc-top-app-bar {
      background: #6750a4;
      position: static;
    }

    .profile {
      text-align: center;
      padding: 15px;
      background: white;
```

```
    margin: 15px;
    border-radius: 10px;
}

.profile img {
    width: 70px;
    border-radius: 50%;
    margin-bottom: 5px;
}

.stats {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 15px;
    padding: 0 15px;
    margin-bottom: 15px;
}

.stat-card {
    padding: 15px;
    text-align: center;
    background: white;
    border-radius: 10px;
}

.stat-number {
    font-size: 22px;
    font-weight: 500;
    color: #6750a4;
}
.container {
    padding: 0 15px 15px 15px;
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 15px;
}

.mdc-card {
    border-radius: 12px;
    overflow: hidden;
}
```

```
    .card-img {
      height: 130px;
      background-size: cover;
      background-position: center;
    }

    .card-content {
      padding: 12px;
    }

    .card-title {
      font-size: 16px;
      font-weight: 500;
      margin-bottom: 5px;
    }

    .card-text {
      font-size: 13px;
      color: #666;
      margin-bottom: 10px;
    }

    .footer {
      text-align: center;
      padding: 10px;
      background: #e0e0e0;
      font-size: 14px;
    }
  </style>
</head>

<body>
  <header class="mdc-top-app-bar">
    <div class="mdc-top-app-bar__row">
      <section class="mdc-top-app-bar__section">
        <span class="mdc-top-app-bar__title">
          Material Dashboard
        </span>
      </section>
    </div>
```

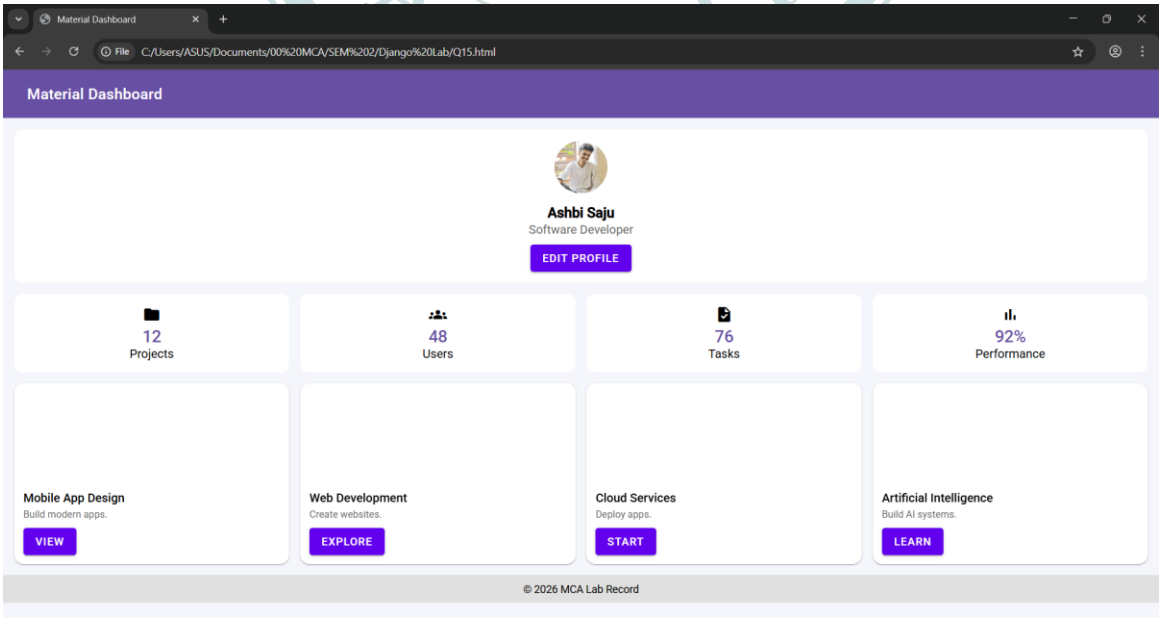
LEARN • SERVE • EXCEL RAJAGIRI

```
  </header>
  <div class="profile">
    <img src="c:\Users\ASUS\Downloads\IMG.jpg" alt="Profile Picture">
    <h3>Ashbi Saju</h3>
    <p>Software Developer</p>
    <button class="mdc-button mdc-button--raised">
      <span class="mdc-button__label">Edit Profile</span>
    </button>
  </div>
  <div class="stats">
    <div class="stat-card">
      <span class="material-icons">folder</span>
      <div class="stat-number">12</div>
      Projects
    </div>
    <div class="stat-card">
      <span class="material-icons">groups</span>
      <div class="stat-number">48</div>
      Users
    </div>
    <div class="stat-card">
      <span class="material-icons">task</span>
      <div class="stat-number">76</div>
      Tasks
    </div>
    <div class="stat-card">
      <span class="material-icons">bar_chart</span>
      <div class="stat-number">92%</div>
      Performance
    </div>
  </div>
  <div class="container">
    <div class="mdc-card">
      <div class="card-img" style="background-
image:url('https://source.unsplash.com/400x200/?mobile')">
      </div>
      <div class="card-content">
        <div class="card-title">
          Mobile App Design
        </div>
        <div class="card-text">
```

```
            Build modern apps.
          </div>
          <button class="mdc-button mdc-button--raised">
            <span class="mdc-button__label">View</span>
          </button>
        </div>
      </div>
      <div class="mdc-card">
        <div class="card-img" style="background-
image:url('https://source.unsplash.com/400x200/?website')">
        </div>
        <div class="card-content">
          <div class="card-title">
            Web Development
          </div>
          <div class="card-text">
            Create websites.
          </div>
          <button class="mdc-button mdc-button--raised">
            <span class="mdc-button__label">Explore</span>
          </button>
        </div>
      </div>
      <div class="mdc-card">
        <div class="card-img" style="background-
image:url('https://source.unsplash.com/400x200/?cloud')">
        </div>
        <div class="card-content">
          <div class="card-title">
            Cloud Services
          </div>
          <div class="card-text">
            Deploy apps.
          </div>
          <button class="mdc-button mdc-button--raised">
            <span class="mdc-button__label">Start</span>
          </button>
      </div>
      <div class="mdc-card">
        <div class="card-img" style="background-
image:url('https://source.unsplash.com/400x200/?ai')">
```

```
        </div>
        <div class="card-content">
          <div class="card-title">
            Artificial Intelligence
          </div>
          <div class="card-text">
            Build AI systems.
          </div>
          <button class="mdc-button mdc-button--raised">
            <span class="mdc-button__label">Learn</span>
          </button>
        </div>
      </div>
    <div class="footer">
      © 2026 MCA Lab Record
    </div>
    <script src="https://unpkg.com/material-components-web@latest/dist/material-components-web.min.js"></script>
    <script>
      mdc.autoInit();
    </script>
</body>
</html>
```

Output



Program No: 16	**Date:** 16-01-2026

Program Title: Program to display two numbers using command line arguments, add those numbers, and display the result.

```
<!--
Professional UI
@ Ashbi Saju
Roll no: 16
Date: 17-11-2025
-->

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Pro UI - Bootstrap + Material</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link href="https://unpkg.com/material-components-web@latest/dist/material-components-
web.min.css" rel="stylesheet">
  <link
href="https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;500&display=swap"
rel="stylesheet">
  <style>
    body {
      margin: 0;
      font-family: Roboto, sans-serif;
      background: #f4f6fb;
    }

    .navbar {
      background: white;
      box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
    }

    .navbar-brand {
      font-weight: 500;
      color: #5f63f2;
    }
```

```
.layout {
  display: flex;
  min-height: calc(100vh - 56px);
}

.sidebar {
  width: 240px;
  background: #ffffff;
  padding: 20px;
  box-shadow: 2px 0 8px rgba(0, 0, 0, 0.05);
}

.sidebar .mdc-button {
  width: 100%;
  justify-content: flex-start;
  margin-bottom: 12px;
  border-radius: 10px;
}

.main {
  flex: 1;
  padding: 25px;
}

.mdc-card {
  padding: 20px;
  border-radius: 16px;
  height: 100%;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
  display: flex;
  flex-direction: column;
  justify-content: space-between;
}

.stat {
  font-size: 28px;
  font-weight: 500;
  color: #5f63f2;
}
```

```
      .label {
        color: #6c757d;
        font-size: 14px;
      }

      .action-card {
        background: linear-gradient(135deg, #5f63f2, #7a7ff6);
        color: white;
      }

      .action-card .mdc-button {
        background: white;
        color: #5f63f2;
        border-radius: 20px;
      }

      .footer {
        text-align: center;
        padding: 10px;
        background: white;
        box-shadow: 0 -2px 6px rgba(0, 0, 0, 0.08);
      }
    </style>
</head>

<body>

    <nav class="navbar navbar-expand-lg px-4">
      <a class="navbar-brand" href="#">ProDashboard</a>
      <div class="ms-auto">
        <button class="btn btn-outline-primary">Profile</button>
      </div>
    </nav>

    <div class="layout">

      <div class="sidebar">
        <button class="mdc-button mdc-button--raised">
          <span class="mdc-button__label">Overview</span>
        </button>
        <button class="mdc-button mdc-button--outlined">
```

```
      <span class="mdc-button__label">Customers</span>
    </button>
    <button class="mdc-button mdc-button--outlined">
      <span class="mdc-button__label">Orders</span>
    </button>
    <button class="mdc-button mdc-button--outlined">
      <span class="mdc-button__label">Analytics</span>
    </button>
    <button class="mdc-button mdc-button--outlined">
      <span class="mdc-button__label">Settings</span>
    </button>
  </div>

  <div class="main">

    <div class="row g-4">

      <div class="col-md-4">
        <div class="mdc-card">
          <div>
            <div class="stat">1,250</div>
            <div class="label">Total Users</div>
          </div>
          <button class="mdc-button mdc-button--outlined">
            <span class="mdc-button__label">View Users</span>
          </button>
        </div>
      </div>

      <div class="col-md-4">
        <div class="mdc-card">
          <div>
            <div class="stat">$8,430</div>
            <div class="label">Revenue</div>
          </div>
          <button class="mdc-button mdc-button--outlined">
            <span class="mdc-button__label">View Report</span>
          </button>
        </div>
      </div>
```

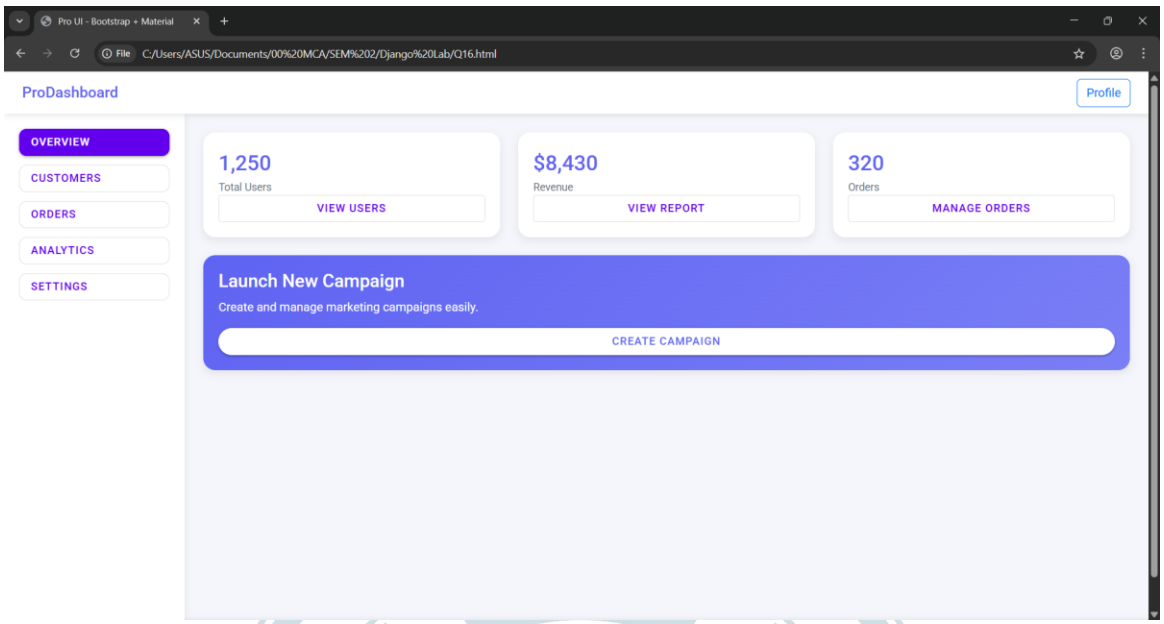
```
          <div class="col-md-4">
            <div class="mdc-card">
              <div>
                <div class="stat">320</div>
                <div class="label">Orders</div>
              </div>
              <button class="mdc-button mdc-button--outlined">
                <span class="mdc-button__label">Manage Orders</span>
              </button>
            </div>
          </div>

          <div class="col-md-12">
            <div class="mdc-card action-card">
              <div>
                <h4>Launch New Campaign</h4>
                <p>Create and manage marketing campaigns easily.</p>
              </div>
              <button class="mdc-button mdc-button--raised">
                <span class="mdc-button__label">Create Campaign</span>
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>
    <div class="footer">
      Professional UI using Bootstrap 5 and Material Design
    </div>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
    <script src="https://unpkg.com/material-components-web@latest/dist/material-
components-web.min.js"></script>
    <script>
      mdc.autoInit();
    </script>

</body>
</html>
```

Output



LEARN • SERVE • EXCEL
RAJAGIRI

Program No: 17	**Date:** 16-01-2026

Program Title: Use Bootstrap 5 for layout and responsive design. Use at least three Material Design components (Cards, Buttons, Forms, Shadows,
etc.). Implement Login and Registration pages for user and admin with proper validation. Use Local Storage for all database operations (Create, Read, Update, Delete). Dynamically render products using JavaScript.
Include the following pages: Home Page, Login Page, Registration Page, Product Listing Page, Add Product Page, Edit Product Page, Cart Page

```
<!--
Bootstrap Product App
@ Ashbi Saju
Roll no: 16
Date: 16-01-2026
-->
```

index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>OnShopee — Home</title>

  <!-- Bootstrap 5 -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet"/>

  <!-- Material Icons -->
  <link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet">

  <!-- Custom CSS (Dark + Neon) -->
  <link rel="stylesheet" href="css/style.css" />
</head>
<body>
  <!-- Navbar (kept same IDs/classes) -->
  <nav class="navbar navbar-expand-lg navbar-light bg-light shadow-sm">
    <div class="container">
      <a class="navbar-brand" href="index.html">OnShopee</a>
      <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#nav"
aria-controls="nav" aria-expanded="false">
        <span class="navbar-toggler-icon"></span>
      </button>

      <div id="nav" class="collapse navbar-collapse">
```

```
      <ul class="navbar-nav ms-auto align-items-lg-center">
        <li class="nav-item"><a class="nav-link" href="index.html">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="shop.html">Shop</a></li>
        <li class="nav-item d-none" id="addProductNav"><a class="nav-link" href="add-
product.html">
Add Product</a></li>
        <li class="nav-item"><a class="nav-link" href="cart.html">Cart</a></li>
        <li class="nav-item" id="authLinks"><!-- auth links injected by JS --></li>
      </ul>
    </div>
  </div>
 </nav>

 <!-- Hero -->
 <header class="py-5 hero">
  <div class="container text-center">
   <h1 class="display-5">Welcome to OnShopee</h1>
   <p class="lead">A demo shop built with Bootstrap and LocalStorage</p>
   <a id="startShoppingBtn" class="btn material-btn btn-lg">Start Shopping</a>
  </div>
 </header>

 <!-- Categories -->
 <main class="container my-5">
  <section>
   <h2 class="mb-4">Top Categories</h2>

   <div class="row g-3">
    <div class="col-md-4">
     <div class="card material-card p-3">
      <div class="card-body">
       <img src="https://images.unsplash.com/photo-1566793474285-
2decf0fc182a?fm=jpg&q=60&w=3000&ixlib=rb-
4.1.0&ixid=M3wxMjA3fDB8MHxzZWFyY2h8Mnx8cGhvbmUlMj
BhY2Nlc3NvcmllcxlbnwwfHwwfHx8MA%3D%3D" class="card-img-top" alt="Electronics">
        <h5 class="card-title">Electronics</h5>
        <p class="card-text">Phones, accessories, and gadgets.</p>
        <a href="shop.html" class="btn material-btn">Explore</a>
      </div>
     </div>
    </div>

    <div class="col-md-4">
     <div class="card material-card p-3">
      <div class="card-body">
       <img src="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTGgq73
zcVWmUmUZPlw8YF_QZoRdlXprndBFA&s" class="card-img-top"  alt="Clothing">
```

```
          <br>
          <br>
          <h5 class="card-title">Clothing</h5>
          <p class="card-text">Latest trends & comfortable wear.</p>
          <a href="shop.html" class="btn material-btn">Explore</a>
         </div>
        </div>
       </div>

       <div class="col-md-4">
        <div class="card material-card p-3">
         <div class="card-body">
          <img src="https://www.orangetree.in/cdn/shop/articles/Living-and-
dining_1_1.jpg?v=1741062042" class="card-img-top" alt="Home & Living">
          <h5 class="card-title">Home &amp;amp; Living</h5>
          <p class="card-text">Decor and daily essentials.</p>
          <a href="shop.html" class="btn material-btn">Explore</a>
         </div>
        </div>
       </div>
     </div>
   </section>
  </main>

  <!-- Footer -->
  <footer class="text-center py-4 bg-light">
   <small>&copy; <span id="year"></span> MyShop</small>
  </footer>

  <!-- Snackbar -->
  <div id="snackbar"></div>

  <!-- Scripts (kept order) -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
  <script src="js/main.js"></script>
  <script src="js/auth.js"></script>

  <script>
   document.getElementById('year').textContent = new Date().getFullYear();
   document.getElementById('startShoppingBtn').addEventListener('click', () => location.href =
'shop.html');
  </script>
</body>
</html>
```

main.js

```
// js/main.js
// show/hide nav items based on login state

function renderNavbar() {
  const navbar = document.getElementById("navbar");
  if (!navbar) return;

  const user = JSON.parse(localStorage.getItem("loggedInUser"));
  let html = `
  <nav class="navbar navbar-expand-lg navbar-light bg-light px-3">
   <a class="navbar-brand" href="index.html">Online Shop</a>
   <div class="collapse navbar-collapse">
    <ul class="navbar-nav ms-auto">
  `;

  if (user) {
    // Logged in
    html += `<li class="nav-item"><a class="nav-link" href="shop.html">Shop</a></li>`;

    if (user.isAdmin) {
      // Admin sees Add Product
      html += `<li class="nav-item"><a class="nav-link" href="add-product.html">Add
Product</a></li>`;
    } else {
      // Regular user sees Cart
      html += `<li class="nav-item"><a class="nav-link" href="cart.html">Cart</a></li>`;
    }

    html += `<li class="nav-item"><a class="nav-link" href="#" onclick="logoutUser()">Logout
(${user.name})</a></li>`;
  } else {
    // Not logged in
    html += `
      <li class="nav-item"><a class="nav-link" href="login.html">Login</a></li>
      <li class="nav-item"><a class="nav-link" href="register.html">Register</a></li>
    `;
  }

  html += `</ul></div></nav>`;
  navbar.innerHTML = html;
}

// Run on page load
document.addEventListener("DOMContentLoaded", renderNavbar);
```

```
function updateNavbar(){
  const logged = JSON.parse(localStorage.getItem('loggedInUser') || 'null');
  const authLinks = document.getElementById('authLinks');
  const addNav = document.getElementById('addProductNav');

  if(!authLinks) return;

  if(logged){
    authLinks.innerHTML = `
      <span class="nav-link">Hi, ${escapeHtml(logged.name)}</span>
      <a class="nav-link" href="#" id="logoutBtn">Logout</a>
    `;
    document.getElementById('logoutBtn').addEventListener('click', (e) => {
      e.preventDefault();
      localStorage.removeItem('loggedInUser');
      location.href = 'login.html';
    });
    if(addNav) addNav.classList.remove('d-none');
  } else {
    authLinks.innerHTML = `
      <a class="nav-link" href="login.html">Login</a>
      <a class="nav-link" href="register.html">Register</a>
    `;
    if(addNav) addNav.classList.add('d-none');
  }
}

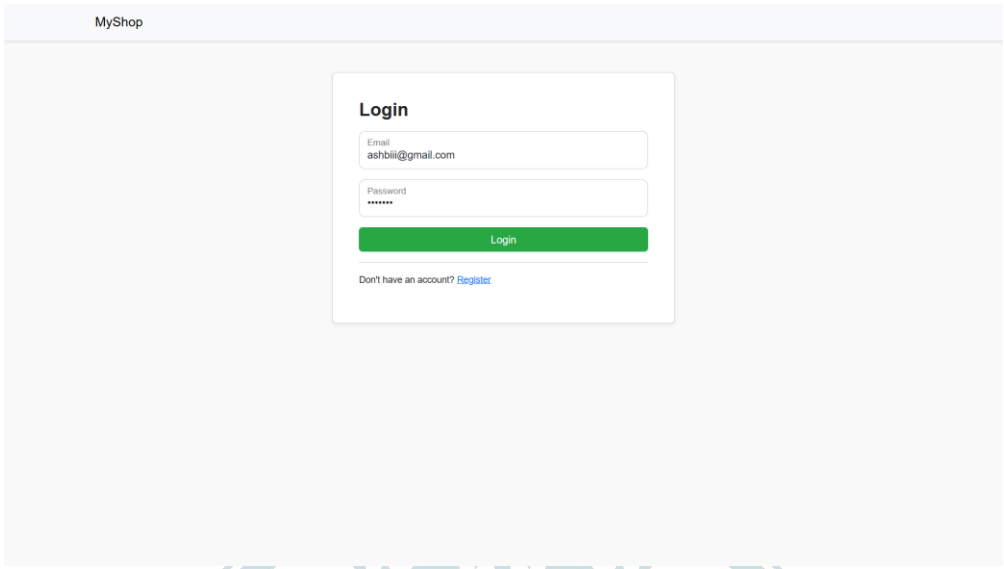
function checkLogin() {
let user = JSON.parse(localStorage.getItem("loggedInUser"));
if (!user) window.location.href = "login.html";
}

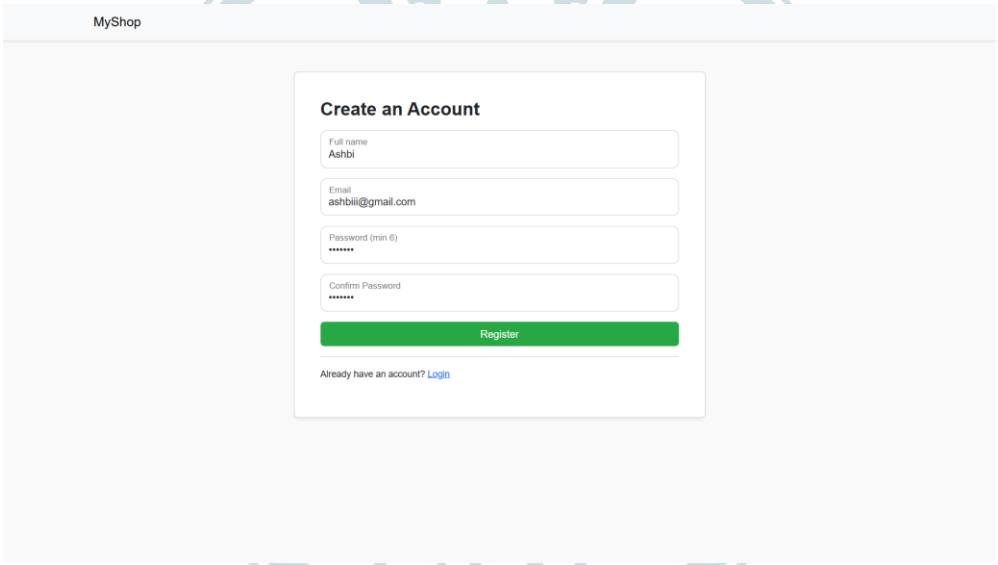
function logout() {
localStorage.removeItem("loggedInUser");
window.location.href = "login.html";
}

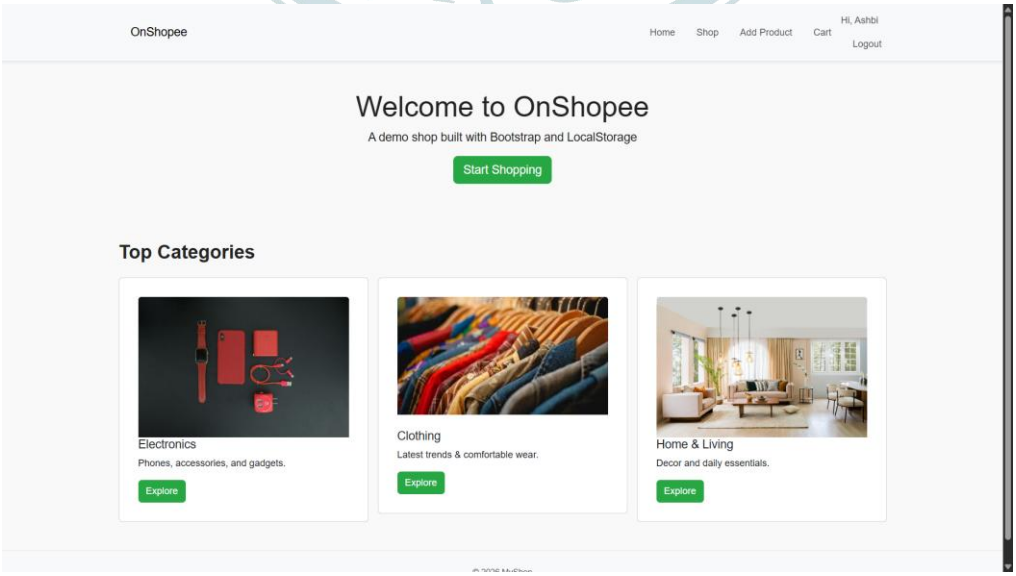
// basic escape
function escapeHtml(s){ return String(s).replaceAll('<','&lt;').replaceAll('>','&gt;'); }

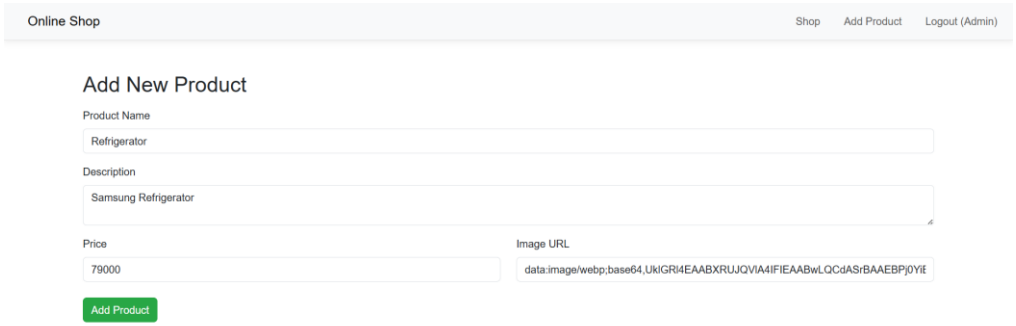
// run on load
document.addEventListener('DOMContentLoaded', updateNavbar);
```

Output

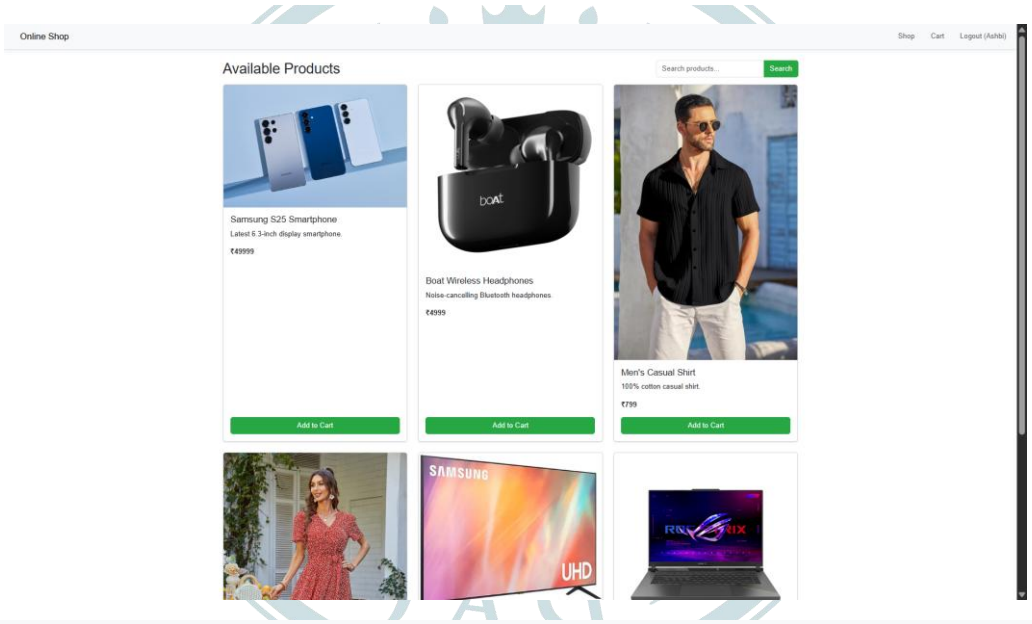

MyShop
Login
Email
ashbiii@gmail.com
Password
Login
Don't have an account? Register

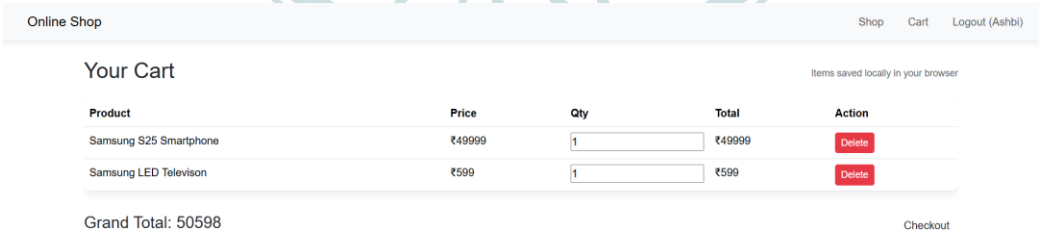

MyShop
Create an Account
Full name
Ashbi
Email
ashbiii@gmail.com
Password (min 6)
Confirm Password
Register
Already have an account? Login


OnShopee
Home
Shop
Add Product
Cart
Hi, Ashbi
Logout
Welcome to OnShopee
A demo shop built with Bootstrap and LocalStorage
Start Shopping
Top Categories
Electronics
Phones, accessories, and gadgets.
Explore
Clothing
Latest trends & comfortable wear.
Explore
Home & Living
Decor and daily essentials.
Explore


Online Shop
Shop
Add Product
Logout (Admin)
Add New Product
Product Name
Refrigerator
Description
Samsung Refrigerator
Price
79000
Image URL

Add Product


Online Shop
Shop
Cart
Logout (Ashbi)
Available Products
Search products...
Search
Samsung S25 Smartphone
Latest 6.3-inch display smartphone.
₹49999
Add to Cart
Boat Wireless Headphones
Noise-cancelling Bluetooth headphones
₹4999
Add to Cart
Men's Casual Shirt
100% cotton casual shirt.
₹799
Add to Cart
SAMSUNG
UHD


Online Shop
Shop
Cart
Logout (Ashbi)
Your Cart
Items saved locally in your browser
Product
Price
Qty
Total
Action
Samsung S25 Smartphone
₹49999
1
₹49999
Delete
Samsung LED Televison
₹599
1
₹599
Delete
Grand Total: 50598
Checkout

Program No: 18	**Date:** 16-01-2026

Program Title: Create a calculator in flask using ajax.

```
<!--
AJAX Flask Calculator
@ Ashbi Saju
Roll no: 16
Date: 16-01-2026
-->

from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/calculate', methods=['POST'])
def calculate():
    data = request.get_json()
    num1 = float(data.get('num1', 0))
    num2 = float(data.get('num2', 0))
    operator = data.get('operator')

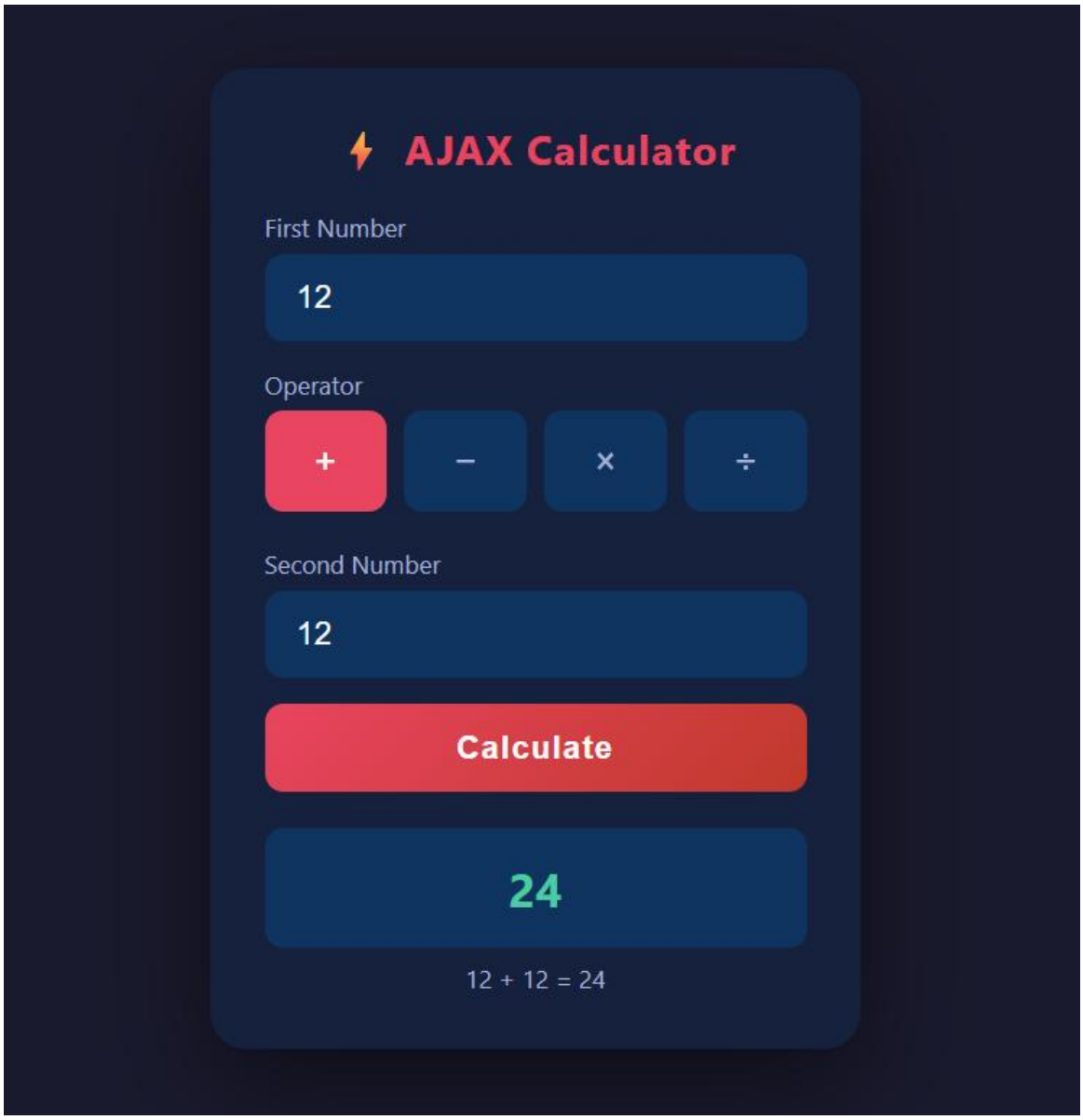
    try:
        if operator == '+':
            result = num1 + num2
        elif operator == '-':
            result = num1 - num2
        elif operator == '*':
            result = num1 * num2
        elif operator == '/':
            if num2 == 0:
                return jsonify({'error': 'Division by zero!'})
            result = num1 / num2
        else:
            return jsonify({'error': 'Invalid operator!'})

        if result == int(result):
            result = int(result)

        return jsonify({'result': result})
    except Exception as e:
        return jsonify({'error': str(e)})
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Output



Program No: 19	**Date:** 16-01-2026

Program Title: Create a suggestion form using ajax in flask.

```
<!--
AJAX Suggestion Form
@ Ashbi Saju
Roll no: 16
Date: 16-01-2026
-->

from flask import Flask, render_template, request, jsonify
from datetime import datetime

app = Flask(__name__)

# In-memory store for suggestions
suggestions = []

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/submit', methods=['POST'])
def submit():
    data = request.get_json()

    name = data.get('name', '').strip()
    email = data.get('email', '').strip()
    category = data.get('category', '').strip()
    message = data.get('message', '').strip()

    # Validation
    if not name:
        return jsonify({'success': False, 'error': 'Name is required.'})
    if not email or '@' not in email:
        return jsonify({'success': False, 'error': 'A valid email is required.'})
    if not message:
        return jsonify({'success': False, 'error': 'Suggestion message cannot be empty.'})

    suggestion = {
        'id': len(suggestions) + 1,
        'name': name,
        'email': email,
        'category': category if category else 'General',
        'message': message,
```

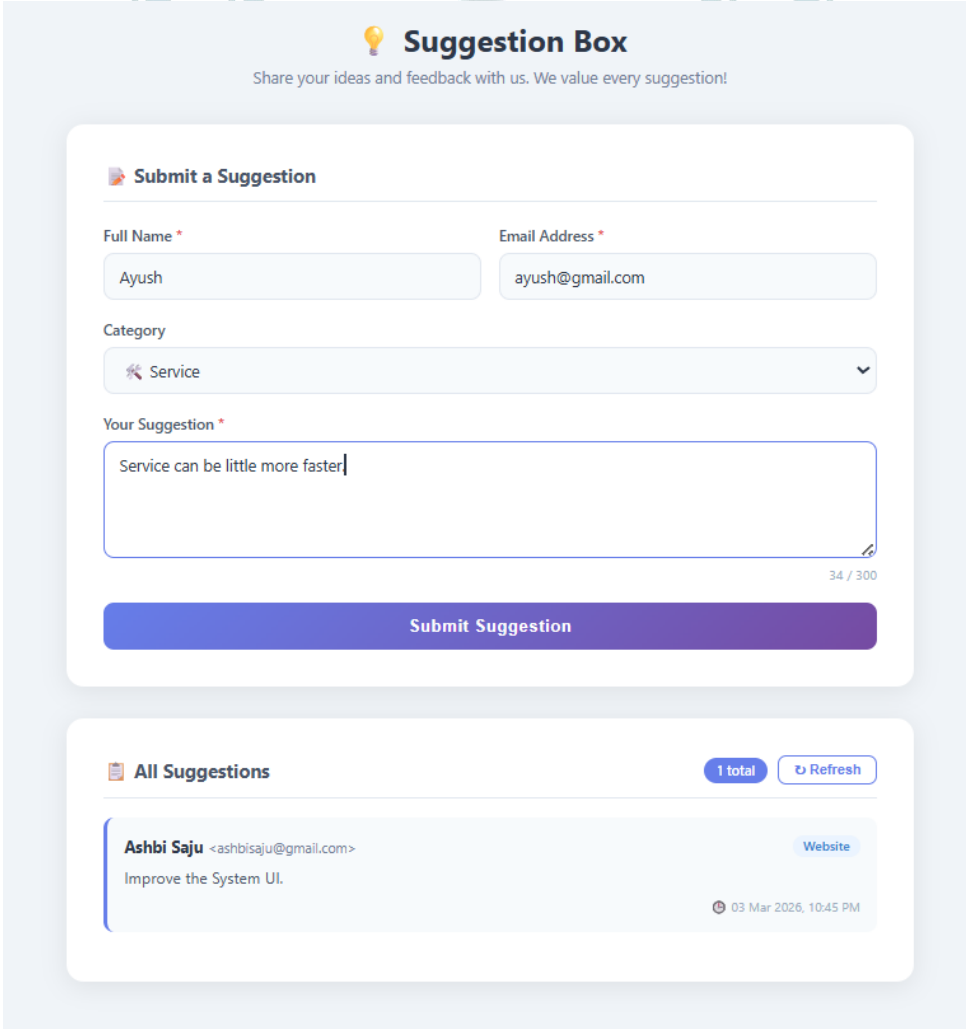
```
        'timestamp': datetime.now().strftime('%d %b %Y, %I:%M %p')
    }
    suggestions.append(suggestion)

    return jsonify({
        'success': True,
        'message': f'Thank you, {name}! Your suggestion has been submitted.',
        'total': len(suggestions)
    })

@app.route('/suggestions', methods=['GET'])
def get_suggestions():
    return jsonify({'suggestions': suggestions[::-1]})  # newest first

if __name__ == '__main__':
    app.run(debug=True)
```

Output



Program No: 20	**Date:** 23-01-2026

Program Title: Create a Django project and display Hello World on the browser.

```
<!--
Django Hello World
@ Ashbi Saju
Roll no: 16
Date: 23-01-2026
-->
```

urls.py

```
from django.contrib import admin
from django.urls import path
from . import views # import views

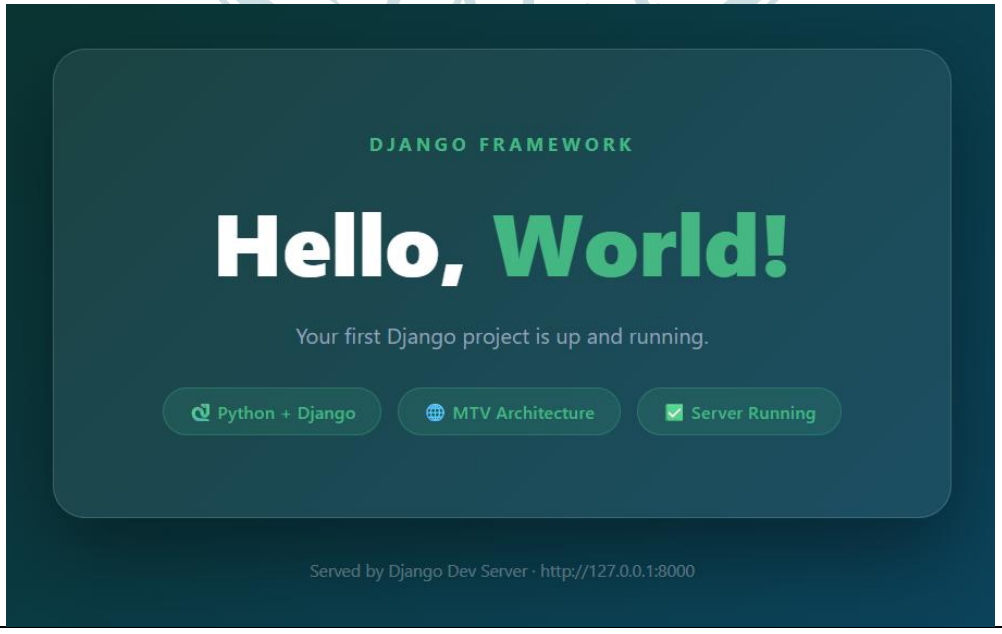
urlpatterns = [
path('admin/', admin.site.urls),
path('', views.hello, name='hello'),
```

views.py

```
from django.shortcuts import render

def hello(request):
    return render(request, 'myapp/index.html')
```

Output



Program No: 21	**Date:** 23-01-2026

Program Title: Create a Django app and configure it in the project.

```
<!--
Django App Setup
@ Ashbi Saju
Roll no: 16
Date: 23-01-2026
-->
```

urls.py

```
from django.urls import path
from . import views

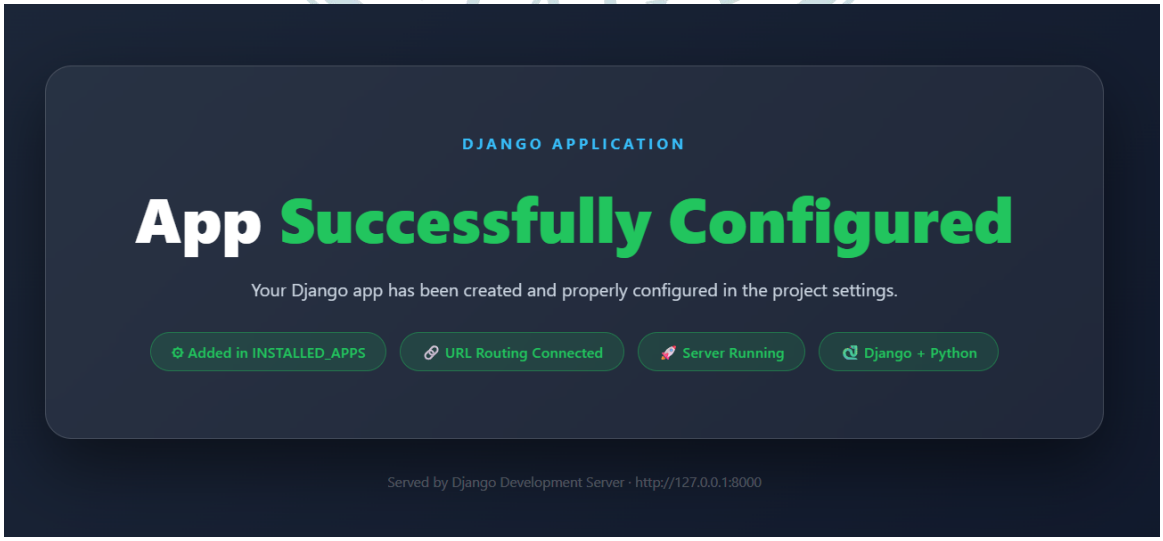
urlpatterns = [
    path('', views.hello, name='hello'),
]
```

views.py

```
from django.shortcuts import render

def hello(request):
    return render(request, 'myapp/index.html')
```

Output



Program No: 22	**Date:** 23-01-2026

Program Title: Create a Django model for Student and perform migrations. Register models in the Django admin panel

```
<!--
Student Model Migration
@ Ashbi Saju
Roll no: 16
Date: 23-01-2026
-->
```

urls.py

```
from django.urls import path

from . import views

urlpatterns = [
    path('', views.student_list, name='student_list'),
]
```

views.py

```
from django.shortcuts import render

from .models import Student

def student_list(request):
    students = Student.objects.all()
    return render(request, 'studentapp/student_list.html', {'students': students})
```

admins.py

```
from django.contrib import admin

from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display  = ('roll_no', 'name', 'branch', 'year')
    list_filter   = ('branch', 'year')
    search_fields = ('name', 'roll_no')
```

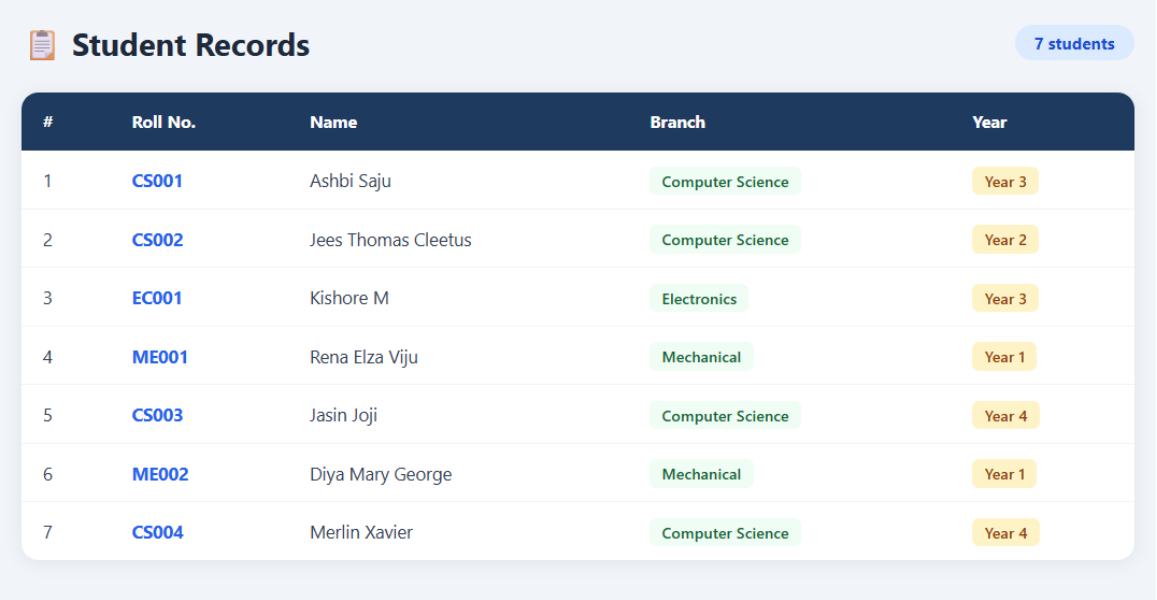
models.py

```
from django.db import models

class Student(models.Model):
    name    = models.CharField(max_length=100)
    roll_no = models.CharField(max_length=20, unique=True)
    branch  = models.CharField(max_length=50)
    year    = models.IntegerField()

    def __str__(self):
        return f"{self.name} ({self.roll_no})"
```

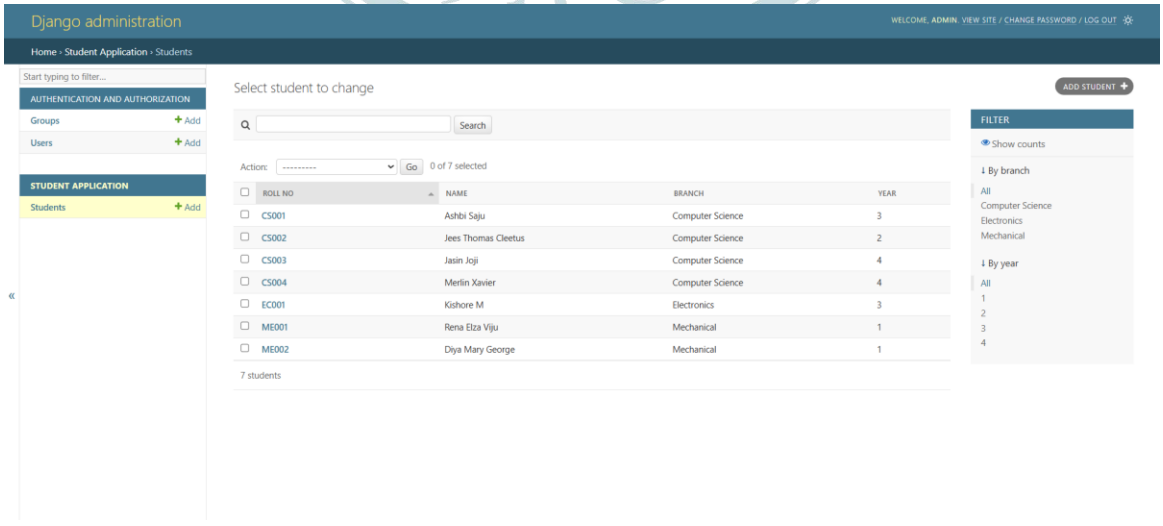
Output



Student Records

7 students

#	Roll No.	Name	Branch	Year
1	CS001	Ashbi Saju	Computer Science	Year 3
2	CS002	Jees Thomas Cleetus	Computer Science	Year 2
3	EC001	Kishore M	Electronics	Year 3
4	ME001	Rena Elza Viju	Mechanical	Year 1
5	CS003	Jasin Joji	Computer Science	Year 4
6	ME002	Diya Mary George	Mechanical	Year 1
7	CS004	Merlin Xavier	Computer Science	Year 4



Program No: 23	**Date:** 23-01-2026

Program Title: Perform CRUD operations using Django ORM.

```
<!--
Django ORM CRUD
@ Ashbi Saju
Roll no: 16
Date: 23-01-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_student_list, name='home'),
    path('student/', views.student_student_list,   name='student_list'),
    path('student/add/', views.student_student_create, name='student_create'),
    path('student/<int:pk>/', views.student_student_detail, name='student_detail'),
    path('student/<int:pk>/edit/', views.student_student_edit,   name='student_edit'),
    path('student/<int:pk>/delete/', views.student_student_delete, name='student_delete'),
]
```

views.py

```
from django.shortcuts import render, get_object_or_404, redirect
from .models import Student

def student_student_list(request):
    students = Student.objects.all()
    return render(request, 'students/student_list.html', {'students': students})

def student_student_detail(request, pk):
    student = get_object_or_404(Student, pk=pk)
    return render(request, 'students/student_detail.html', {'student': student})

def student_student_create(request):
    if request.method == 'POST':
        Student.objects.create(
            name    = request.POST['name'],
            roll_no = request.POST['roll_no'],
            branch  = request.POST['branch'],
            year    = request.POST['year'],
            email   = request.POST.get('email', ''),
```

```
            phone   = request.POST.get('phone', ''),
        )
        return redirect('student_list')
    return render(request, 'students/student_form.html',
    {'action': 'Add', 'view_type': 'STUDENT'})

def student_student_edit(request, pk):
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        student.name    = request.POST['name']
        student.roll_no = request.POST['roll_no']
        student.branch  = request.POST['branch']
        student.year    = request.POST['year']
        student.email   = request.POST.get('email', '')
        student.phone   = request.POST.get('phone', '')
        student.save()
        return redirect('student_list')
    return render(request, 'students/student_form.html',
    {'student': student, 'action': 'Edit', 'view_type': 'STUDENT'})

def student_student_delete(request, pk):
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        student.delete()
        return redirect('student_list')
    return render(request, 'students/student_confirm_delete.html',
    {'student': student, 'view_type': 'STUDENT'})
```

admins.py

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display   = ('roll_no', 'name', 'branch', 'year', 'email', 'phone')
    list_filter    = ('branch', 'year')
    search_fields  = ('name', 'roll_no', 'email')
    ordering       = ('roll_no',)
```

models.py

```
from django.db import models

BRANCH_CHOICES = [
    ('CSE', 'Computer Science'),
```

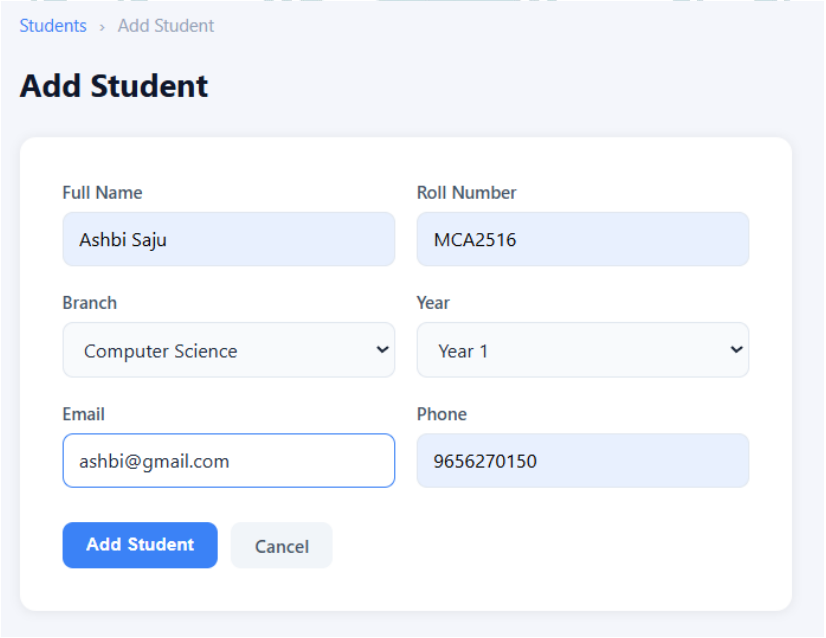
```
    ('ECE', 'Electronics'),
    ('ME',  'Mechanical'),
    ('CE',  'Civil'),
]

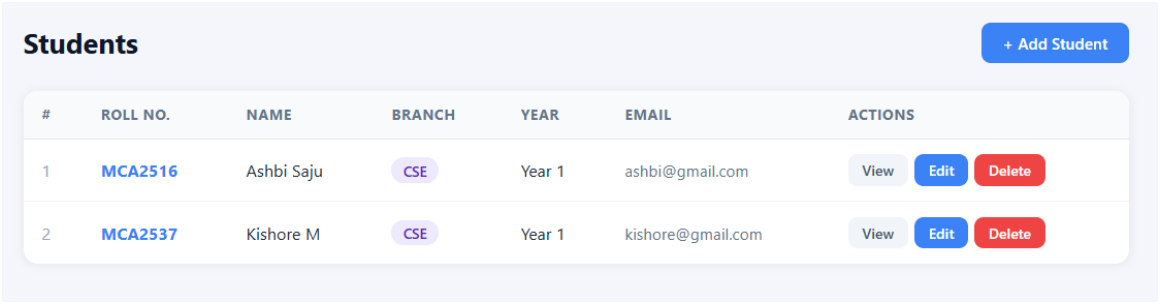
class Student(models.Model):
    name     = models.CharField(max_length=100)
    roll_no  = models.CharField(max_length=20, unique=True)
    branch   = models.CharField(max_length=10, choices=BRANCH_CHOICES)
    year     = models.IntegerField()
    email    = models.EmailField(blank=True)
    phone    = models.CharField(max_length=15, blank=True)

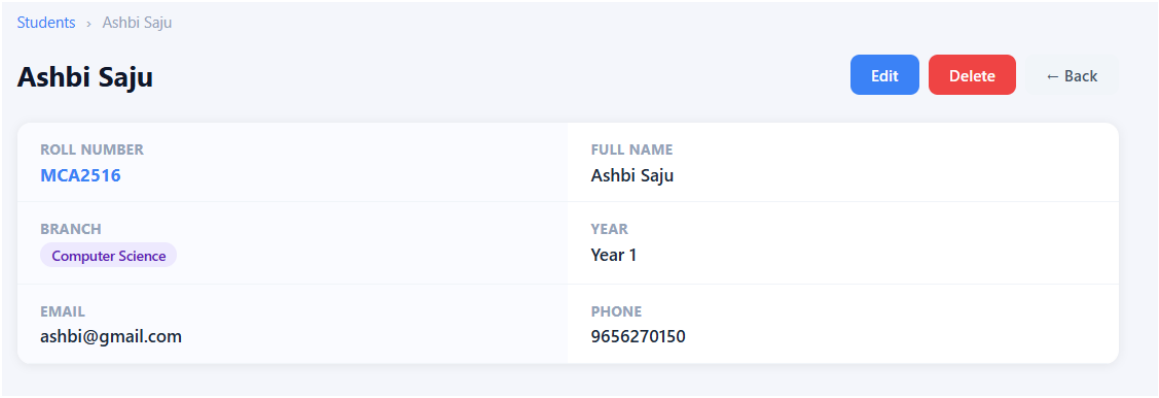
    def __str__(self):
        return f"{self.name} ({self.roll_no})"

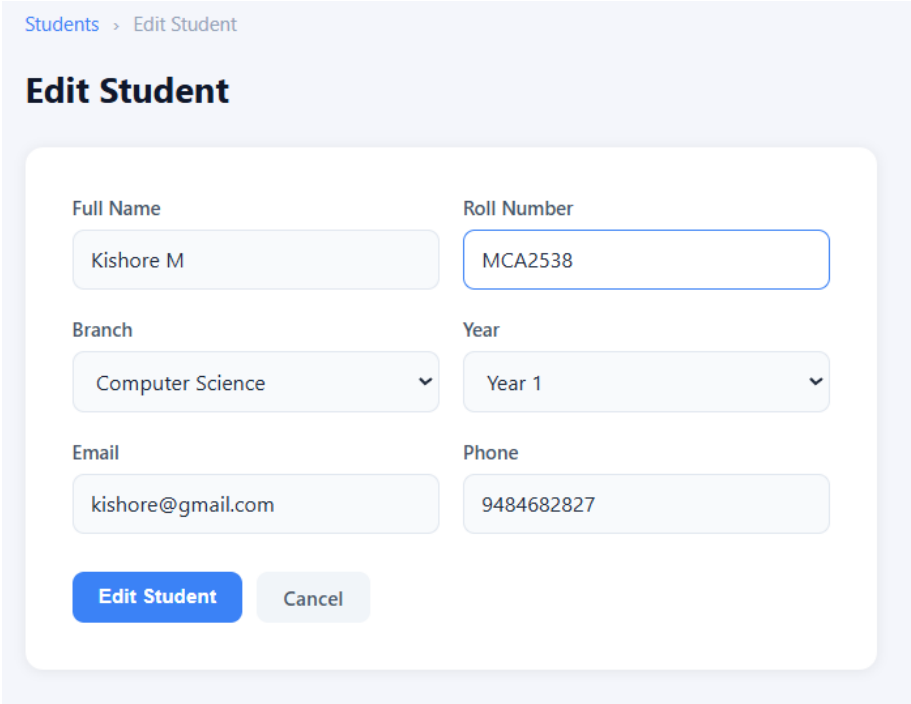
    class Meta:
        ordering = ['roll_no']
```

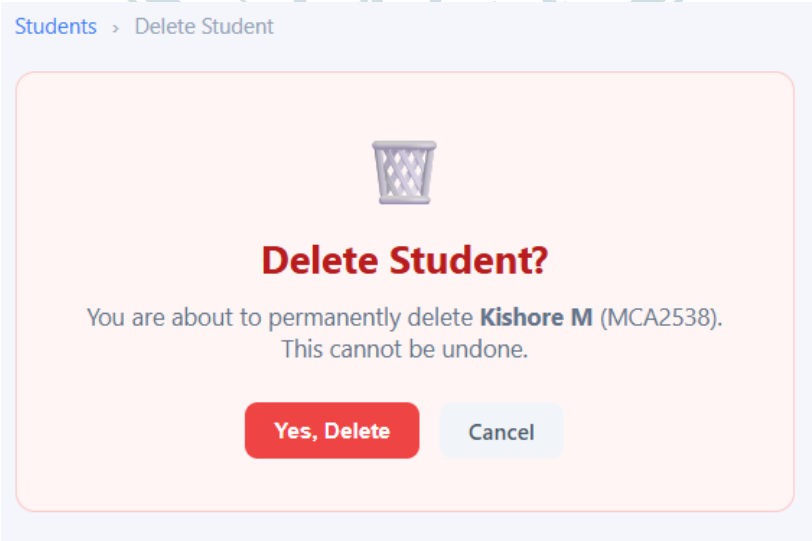
Output






Students › Ashbi Saju
Ashbi Saju
Edit
Delete
← Back
ROLL NUMBER
MCA2516
FULL NAME
Ashbi Saju
BRANCH
Computer Science
YEAR
Year 1
EMAIL
ashbi@gmail.com
PHONE
9656270150


Students › Edit Student
Edit Student
Full Name
Kishore M
Roll Number
MCA2538
Branch
Computer Science
Year
Year 1
Email
kishore@gmail.com
Phone
9484682827
Edit Student
Cancel


Students › Delete Student
Delete Student?
You are about to permanently delete Kishore M (MCA2538).
This cannot be undone.
Yes, Delete
Cancel

Program No: 24	**Date:** 30-01-2026

Program Title: Create function-based views in Django.

```
<!--
Function-Based Views
@ Ashbi Saju
Roll no: 16
Date: 30-01-2026
-->
```

urls.py

```
from django.urls import path
from .views import function_view

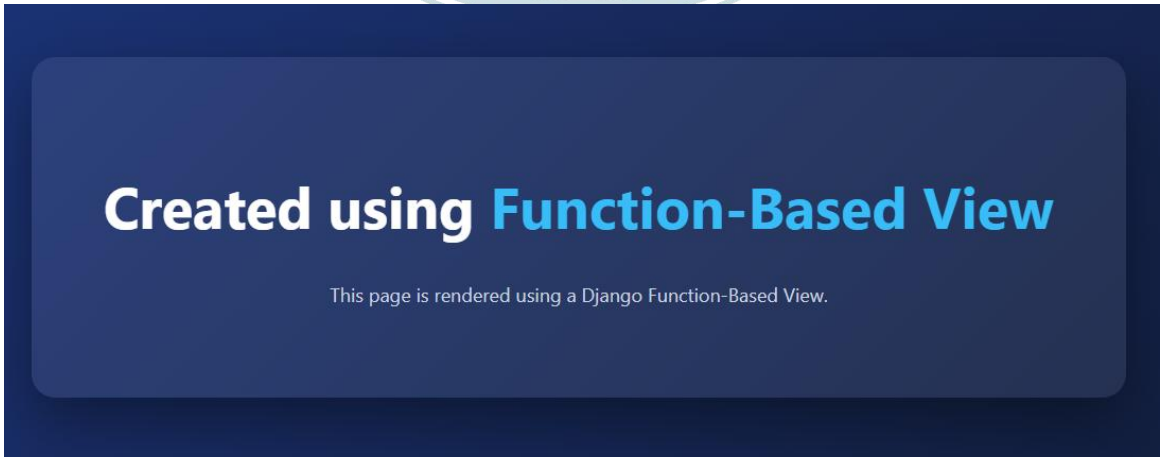
urlpatterns = [
    path('', function_view, name='function_view'),
]
```

views.py

```
from django.shortcuts import render

# Function-Based View
def function_view(request):
    return render(request, 'function_view.html')
```

Output



Program No: 25	**Date:** 30-01-2026

Program Title: Create class-based views in Django.

```
<!--
Class-Based Views
@ Ashbi Saju
Roll no: 16
Date: 30-01-2026
-->
```

urls.py

```
from django.urls import path
from .views import ClassBasedHelloView

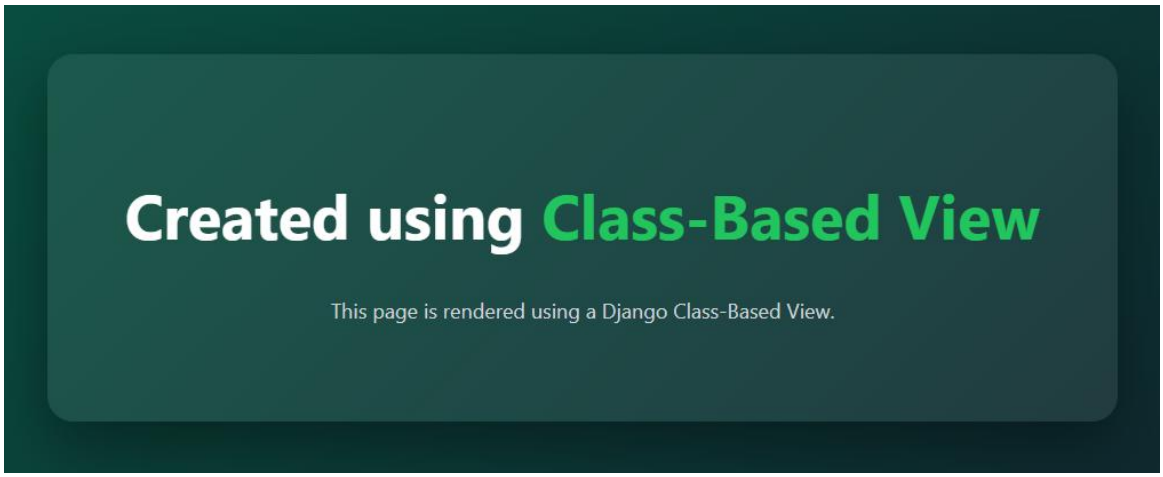
urlpatterns = [
    path('', ClassBasedHelloView.as_view(), name='class_view'),
]
```

views.py

```
from django.shortcuts import render
from django.views import View

# Class-Based View
class ClassBasedHelloView(View):
    def get(self, request):
        return render(request, 'class_view.html')
```

Output



Program No: 26	**Date:** 30-01-2026

Program Title: Configure dynamic URL routing in Django.

```
<!--
Dynamic URL Routing
@ Ashbi Saju
Roll no: 16
Date: 30-01-2026
-->
```

urls.py

```
from django.urls import path
from .views import greet

urlpatterns = [
    path('greet/<str:name>/', greet, name='greet'),
]
```

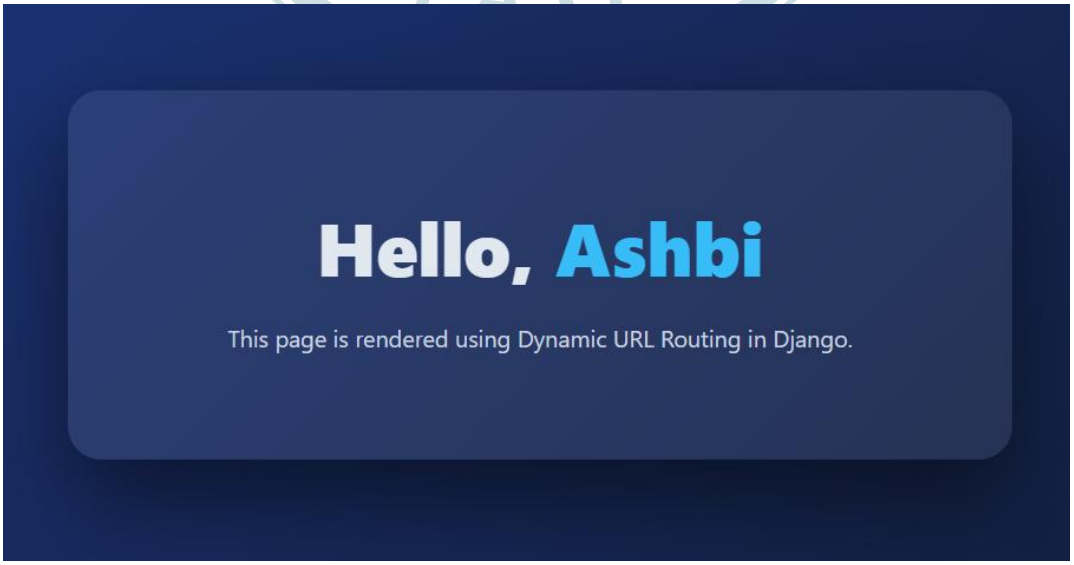
views.py

```
from django.shortcuts import render

def greet(request, name):
    return render(request, 'greetapp/greeting.html', {'name': name})
```

Output

`http://127.0.0.1:8000/greet/Ashbi/`



Program No: 27	**Date:** 30-01-2026

Program Title: Create a Django login and logout system Create a Django login and logout system. Implement dark mode and light mode toggle using HTML and CSS.

```
<!--
Django Auth System
@ Ashbi Saju
Roll no: 16
Date: 30-01-2026
-->
```

urls.py

```
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views
from .forms import CustomLoginForm

urlpatterns = [
    path('', views.dashboard, name='dashboard'),
    path('login/', auth_views.LoginView.as_view(
        template_name='accounts/login.html',
        authentication_form=CustomLoginForm
    ), name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
]
```

views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
    return render(request, 'accounts/dashboard.html')
```

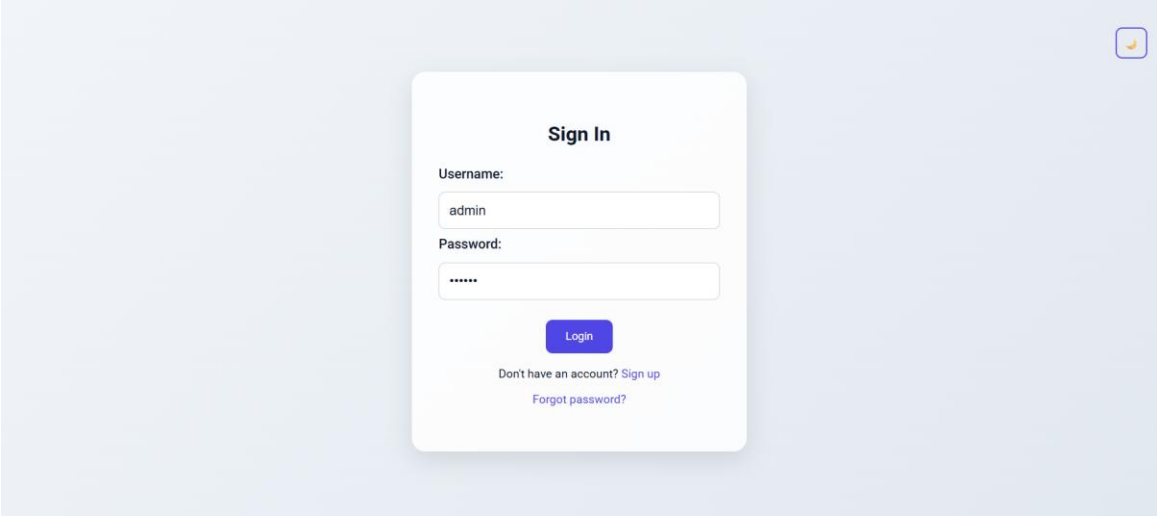
forms.py

```
from django import forms
from django.contrib.auth.forms import AuthenticationForm

class CustomLoginForm(AuthenticationForm):
    username = forms.CharField(
```

```
    widget=forms.TextInput(attrs={
        'class': 'input',
        'placeholder': 'Username'
    })
  )
  password = forms.CharField(
    widget=forms.PasswordInput(attrs={
        'class': 'input',
        'placeholder': 'Password'
    })
  )
```

Output



Program No: 28	**Date:** 30-01-2026

Program Title: Protect views using @login_required decorator.

```
<!--
Protected Django Views
@ Ashbi Saju
Roll no: 16
Date: 30-01-2026
-->
```

urls.py

```
from django.urls import path
from .views import *

urlpatterns = [
    path('', index, name='index'),
]
```

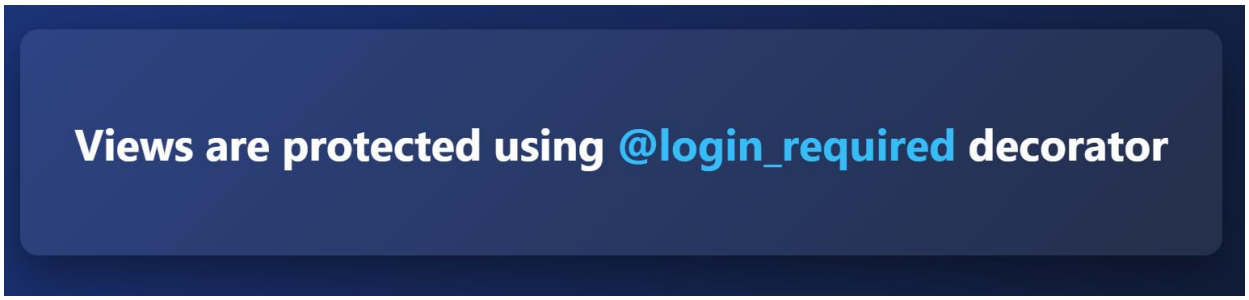
views.py

```
from django.shortcuts import render

def login(request):
    return render(request, 'login.html')

@login_required        #decorator used
def index(request):
    return render(request, 'index.html')
```

Output



Program No: 29	**Date:** 06-02-2026

Program Title: Create and publish a Python library with functions including docstrings and unit tests. Install it using pip, integrate it into a Flask web application with a textarea input and response display, and optionally enhance it with Bootstrap styling, query history, and proper error handling.

```
<!--
Custom Python Library
@ Ashbi Saju
Roll no: 16
Date: 06-02-2026
-->
```

core.py

```
"""
ai_helpers_ashbi.core
AI helper functions with structured responses.
"""

import re

def detect_category(text: str) -> str:
    """
    Simple keyword-based category detection.
    """
    text = text.lower()

    if text.startswith("what is") or "definition" in text:
        return "definition"
    elif text.startswith("how to") or text.startswith("how do"):
        return "how-to"
    elif text.endswith("?"):
        return "question"
    elif "," in text:
        return "list"
    else:
        return "general"

def get_response(prompt: str) -> dict:
    """
    Generate AI-style response with metadata.
    Returns dictionary required by Flask app.
    """
    response_text = f"This is a generated response for: {prompt}"
```

```
    category = detect_category(prompt)
    word_count = len(response_text.split())

    return {
        "prompt": prompt,
        "response": response_text,
        "category": category,
        "word_count": word_count
    }

def summarize_text(text: str, max_sentences: int = 2) -> str:
    """
    Very basic sentence-based summarizer.
    """
    sentences = re.split(r'(?<=[.!?]) +', text)
    summary = " ".join(sentences[:max_sentences])
    return summary.strip()

def format_response(text: str) -> str:
    """
    Clean and format response text.
    """
    return text.strip().capitalize()
```

app.py

```
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))

from flask import Flask, render_template, request, jsonify, session
from ai_helpers_ashbi import get_response, summarize_text

app = Flask(__name__)
app.secret_key = 'ai-helpers-ashbi-flask-secret-2026'
MAX_HISTORY = 10

@app.route('/')
def index():
    if 'history' not in session:
        session['history'] = []
    return render_template('index.html', history=session['history'])

@app.route('/ask', methods=['POST'])
def ask():
    try:
        data   = request.get_json()
```

```
        prompt = data.get('prompt', '').strip()
        action = data.get('action', 'respond')

        if not prompt:
            return jsonify({'error': 'Please enter a prompt.'}), 400

        if action == 'summarize':
            result = {
                'prompt':     prompt,
                'response':   summarize_text(prompt, max_sentences=2),
                'category':   'summary',
                'word_count': len(summarize_text(prompt).split()),
            }
        else:
            result = get_response(prompt)

        history = session.get('history', [])
        history.insert(0, {
            'prompt':     result['prompt'][:80] + ('...' if len(result['prompt']) > 80 else ''),
            'category':   result['category'],
            'word_count': result['word_count'],
        })
        session['history']  = history[:MAX_HISTORY]
        session.modified    = True

        return jsonify({
            'response':   result['response'],
            'category':   result['category'],
            'word_count': result['word_count'],
        })

    except ValueError as e:
        return jsonify({'error': str(e)}), 400
    except Exception as e:
        return jsonify({'error': f'Unexpected error: {str(e)}'}), 500

@app.route('/clear_history', methods=['POST'])
def clear_history():
    session['history'] = []
    session.modified = True
    return jsonify({'ok': True})

if __name__ == '__main__':
    app.run(debug=True)
```

test_core.py

```
import unittest
from ai_helpers import get_response

class TestAIHelpers(unittest.TestCase):

    def test_response(self):
        self.assertIn("AI Response", get_response("Hello"))

if __name__ == "__main__":
    unittest.main()
```

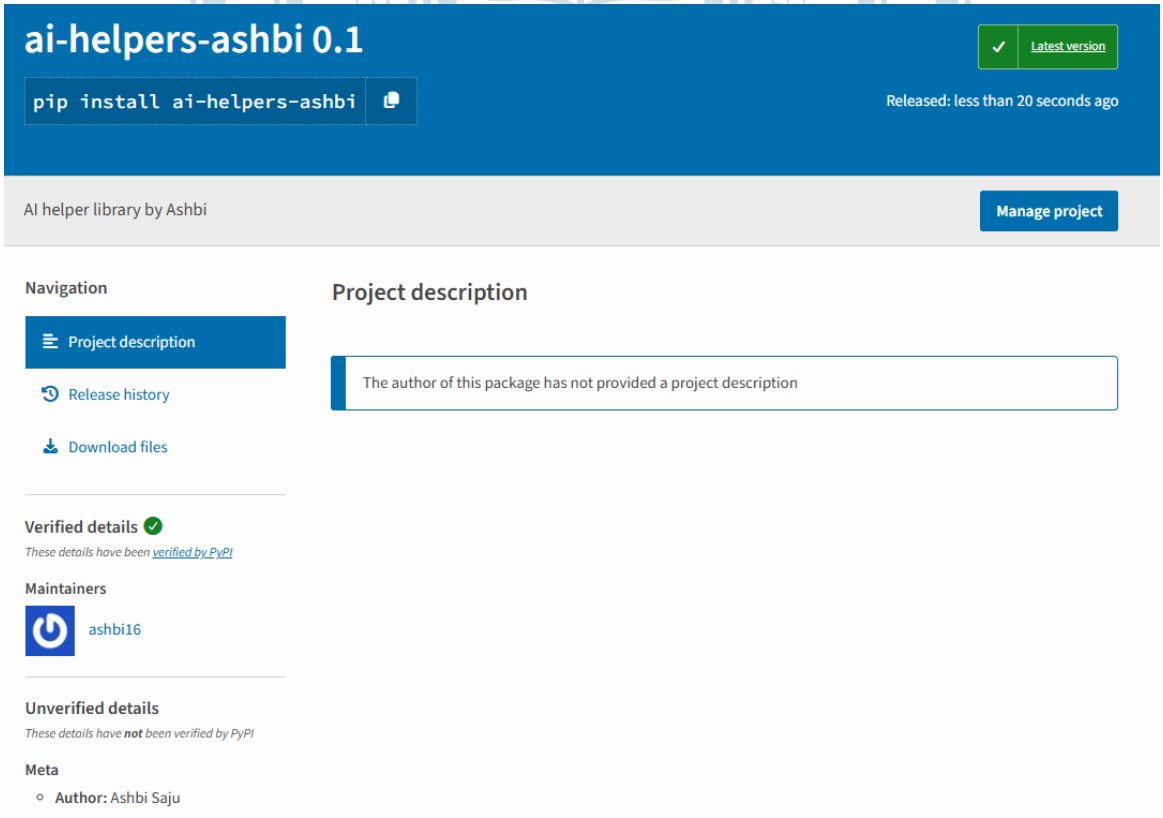
Output

Unit Testing

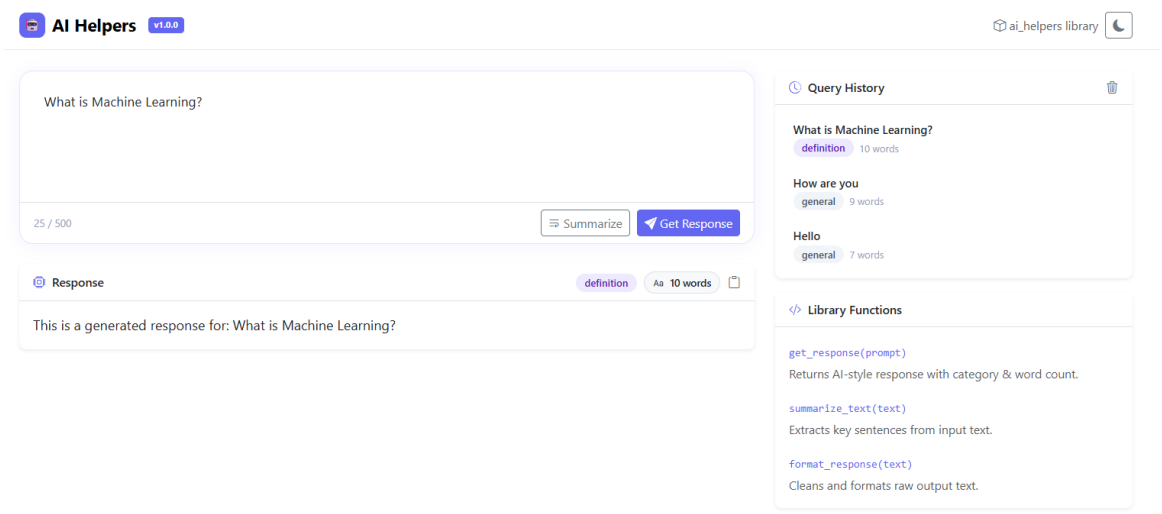
```
PS C:\Users\ASUS\Documents\00 MCA\SEM 2\Django Lab\ai_helpers_project> python -m unittest discover -s tests
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

Python Library



Flask App



LEARN • SERVE • EXCEL
RAJAGIRI

Program No: 30	**Date:** 06-02-2026

Program Title: Online Resume / Portfolio Website

Create a personal online resume or portfolio website using Flask and deploy it on any free platform.

The website must include:

- About Me section
- Education & Skills
- Projects / Achievements
- Contact Information

```
<!--
Flask Portfolio Website
@ Ashbi Saju
Roll no: 16
Date: 06-02-2026
-->

"""
Ashbi Saju - Personal Portfolio
Flask Application Entry Point

Run Instructions:
1. Install dependencies: pip install flask
2. Run: python app.py
3. Visit: http://localhost:5000
"""

from flask import Flask, render_template, request, jsonify, send_from_directory
import os

app = Flask(__name__)
app.secret_key = "ashbi_portfolio_2025"

# ── Portfolio Data ──────────────────────────────────────────────

PROFILE = {
    "name": "Ashbi Saju",
    "title": "MCA Student | Software Developer",
    "email": "ashbi404@gmail.com",
    "location": "Kerala, India",
    "github": "https://github.com/ashbisaju",
    "linkedin": "https://linkedin.com/in/ashbisaju",
    "taglines": [
        "Software Developer",
        "MCA Student",
        "Python Enthusiast",
```

```
        "AI & ML Explorer",
        "Full Stack Builder",
    ],
    "about": (
        "I am a passionate Master of Computer Applications student with a strong interest "
        "in software development, artificial intelligence, and modern web technologies. "
        "I enjoy building efficient, scalable, and user-friendly applications that solve "
        "real-world problems. My technical expertise includes Python, Flask, Django, and "
        "database systems, along with experience in integrating hardware and AI solutions."
    ),
}

SKILLS = [
    {"name": "Python",     "level": 90, "icon": "fab fa-python",     "category": "language"},
    {"name": "Flask",      "level": 85, "icon": "fas fa-flask",      "category": "framework"},
    {"name": "Django",     "level": 75, "icon": "fab fa-python",     "category": "framework"},
    {"name": "Java",       "level": 70, "icon": "fab fa-java",       "category": "language"},
    {"name": "C++",        "level": 65, "icon": "fas fa-code",       "category": "language"},
    {"name": "C",          "level": 65, "icon": "fas fa-code",       "category": "language"},
    {"name": "HTML",       "level": 90, "icon": "fab fa-html5",      "category": "web"},
    {"name": "CSS",        "level": 85, "icon": "fab fa-css3-alt",   "category": "web"},
    {"name": "JavaScript", "level": 75, "icon": "fab fa-js",         "category": "web"},
    {"name": "MySQL",      "level": 80, "icon": "fas fa-database",   "category": "database"},
]

PROJECTS = [
    {
        "id": 1,
        "title": "Bookify – Online Trip Booking Hub",
        "short": "Centralized travel booking platform with hotels, flights & trains.",
        "description": (
            "Bookify is a centralized travel booking web application that allows users to "
            "book hotels, flights, and trains from one platform. It provides secure login, "
            "booking management, payment integration, and an admin dashboard."
        ),
        "tech": ["PHP", "MySQL"],
        "icon": "fas fa-plane-departure",
        "color": "#00d4ff",
        "github": "#",
        "demo": "#",
        "featured": True,
    },
    {
        "id": 2,
        "title": "AI-Based Smart Traffic Light Control System",
        "short": "Intelligent traffic management with YOLOv5 & real-time detection.",
        "description": (
```

```
            "An intelligent traffic management system using YOLOv5, Arduino UNO, and Android "
            "app to dynamically control traffic lights based on real-time vehicle detection."
        ),
        "tech": ["Python", "Arduino", "YOLOv5", "Java"],
        "icon": "fas fa-traffic-light",
        "color": "#ff6b35",
        "github": "#",
        "demo": "#",
        "featured": True,
    },
]

EDUCATION = [
    {
        "degree": "Master of Computer Applications (MCA)",
        "institution": "Rajagiri College of Social Sciences",
        "period": "2025 – 2027",
        "description": "Pursuing advanced studies in computer applications, AI, and software
engineering.",
        "icon": "fas fa-graduation-cap",
        "color": "#00d4ff",
        "current": True,
    },
    {
        "degree": "Bachelor of Computer Applications (BCA)",
        "institution": "De Paul Institute of Science & Technology",
        "period": "2022 – 2025",
        "description": "Completed undergraduate studies with focus on programming, databases,
and web technologies.",
        "icon": "fas fa-university",
        "color": "#7c3aed",
        "current": False,
    },
]

ACHIEVEMENTS = [
    {
        "title": "AI Hardware Integration",
        "description": "Successfully integrated YOLOv5 computer vision with Arduino hardware for
real-time traffic control.",
        "icon": "fas fa-robot",
        "color": "#00d4ff",
    },
    {
        "title": "Full-Stack Development",
        "description": "Built a complete multi-module travel booking platform handling hotels,
flights, and trains.",
```

```
        "icon": "fas fa-layer-group",
        "color": "#ff6b35",
    },
    {
        "title": "Cross-Platform Development",
        "description": "Developed Android mobile app that communicates with IoT hardware for
traffic management.",
        "icon": "fab fa-android",
        "color": "#22c55e",
    },
    {
        "title": "Database Design",
        "description": "Designed and optimized relational database schemas for large-scale booking
and management systems.",
        "icon": "fas fa-database",
        "color": "#f59e0b",
    },
]

# ── Routes ──────────────────────────────────────────────────────────────

@app.route("/")
def home():
    return render_template("home.html", profile=PROFILE, active="home")

@app.route("/about")
def about():
    return render_template("about.html", profile=PROFILE, active="about")

@app.route("/skills")
def skills():
    return render_template("skills.html", profile=PROFILE, skills=SKILLS, active="skills")

@app.route("/projects")
def projects():
    return render_template("projects.html", profile=PROFILE, projects=PROJECTS,
active="projects")

@app.route("/education")
def education():
    return render_template("education.html", profile=PROFILE, education=EDUCATION,
active="education")

@app.route("/achievements")
def achievements():
    return render_template("achievements.html", profile=PROFILE,
achievements=ACHIEVEMENTS, active="achievements")
```

```
@app.route("/contact")
def contact():
    return render_template("contact.html", profile=PROFILE, active="contact")

@app.route("/send-message", methods=["POST"])
def send_message():
    """Handle contact form submission."""
    data = request.get_json()
    name    = data.get("name", "").strip()
    email   = data.get("email", "").strip()
    subject = data.get("subject", "").strip()
    message = data.get("message", "").strip()

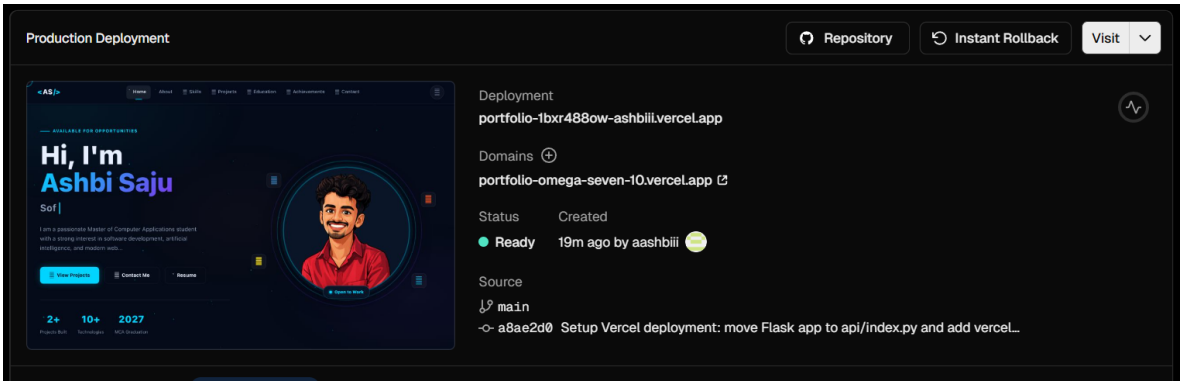
    if not all([name, email, message]):
        return jsonify({"success": False, "message": "Please fill all required fields."}), 400

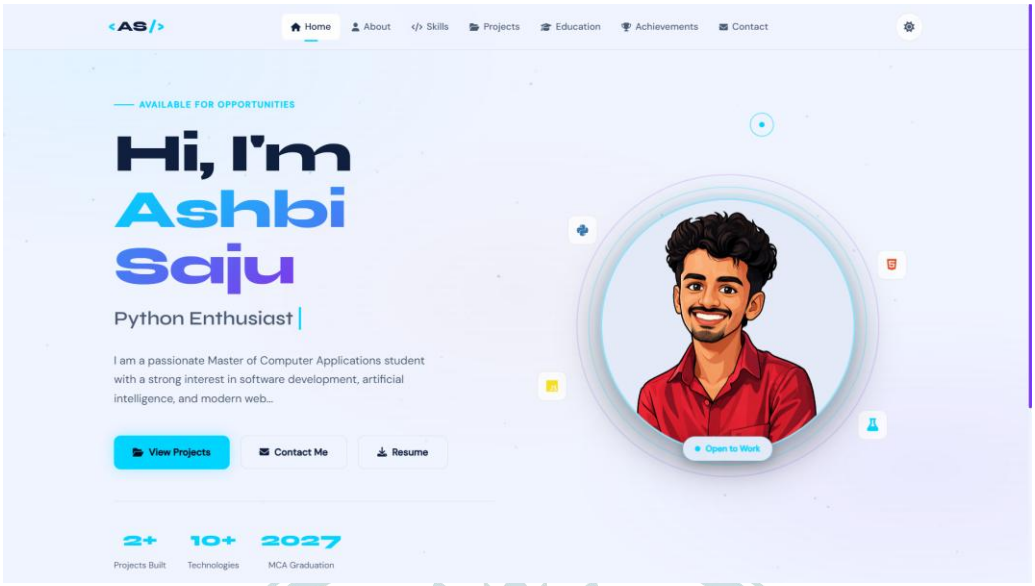
    # In production, integrate an SMTP / email service here.
    print(f"[Contact] From: {name} <{email}> | Subject: {subject}\n{message}")
    return jsonify({"success": True, "message": "Message sent successfully! I'll get back to you soon."})

@app.route("/download-resume")
def download_resume():
    """Serve resume PDF if available, else return 404 message."""
    resume_path = os.path.join(app.static_folder, "files")
    if os.path.exists(os.path.join(resume_path, "resume.pdf")):
        return send_from_directory(resume_path, "resume.pdf", as_attachment=True)
    return jsonify({"message": "Resume will be available soon!"}), 404

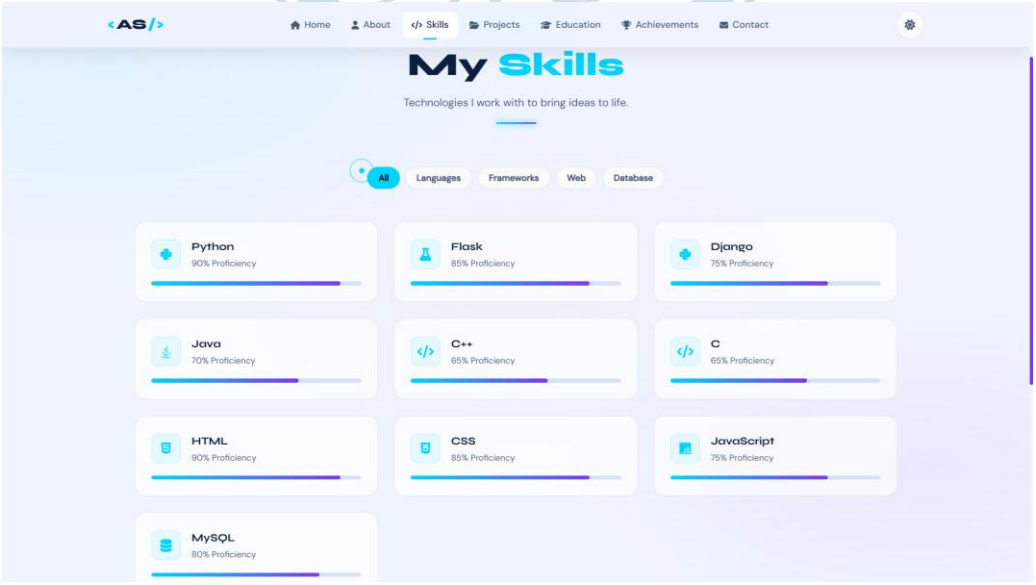
if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)
```

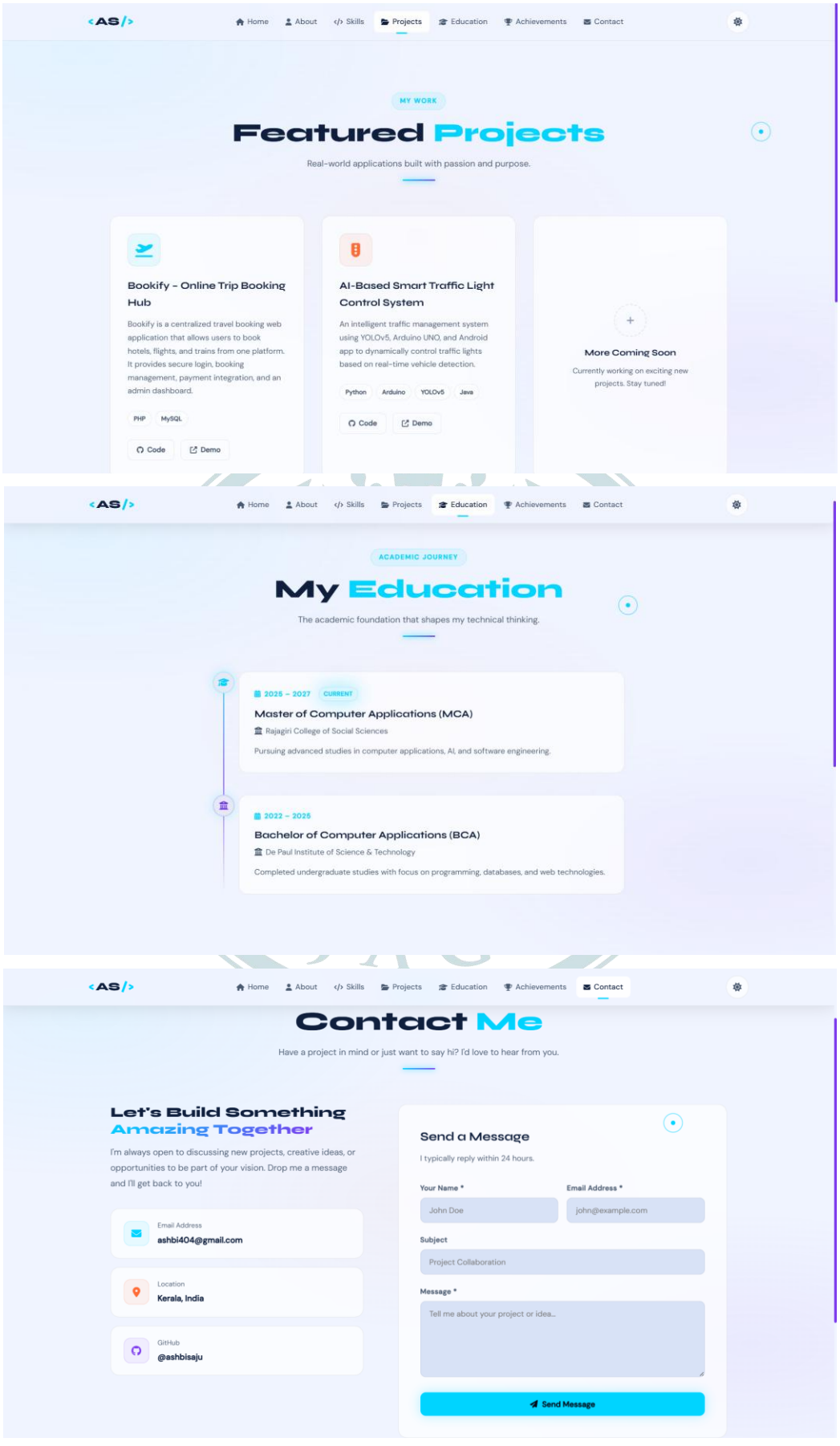
Output




<AS/>
Home
About
Skills
Projects
Education
Achievements
Contact
AVAILABLE FOR OPPORTUNITIES
Hi, I'm
Ashbi
Saju
Python Enthusiast
I am a passionate Master of Computer Applications student with a strong interest in software development, artificial intelligence, and modern web...
View Projects
Contact Me
Resume
Open to Work
2+
Projects Built
10+
Technologies
2027
MCA Graduation


<AS/>
Home
About
Skills
Projects
Education
Achievements
Contact
About Me
MY STORY
Passionate Developer &
Lifelong Learner
I am a passionate Master of Computer Applications student with a strong interest in software development, artificial intelligence, and modern web technologies. I enjoy building efficient, scalable, and user-friendly applications that solve real-world problems. My technical expertise includes Python, Flask, Django, and database systems, along with experience in integrating hardware and AI solutions.
Ashbi Saju
MCA Student | Software Developer
Degree
MCA (Pursuing)
Location
Kerala, India
Email
ashbi404@gmail.com
Languages
English, Malayalam
Get In Touch
View Skills


<AS/>
Home
About
Skills
Projects
Education
Achievements
Contact
My Skills
Technologies I work with to bring ideas to life.
All
Languages
Frameworks
Web
Database
Python
90% Proficiency
Flask
85% Proficiency
Django
75% Proficiency
Java
70% Proficiency
C++
65% Proficiency
C
65% Proficiency
HTML
90% Proficiency
CSS
85% Proficiency
JavaScript
75% Proficiency
MySQL
80% Proficiency


<AS/>
Home
About
Skills
Projects
Education
Achievements
Contact
MY WORK
Featured Projects
Real-world applications built with passion and purpose.
Bookify - Online Trip Booking Hub
Bookify is a centralized travel booking web application that allows users to book hotels, flights, and trains from one platform. It provides secure login, booking management, payment integration, and an admin dashboard.
PHP
MySQL
Code
Demo
AI-Based Smart Traffic Light Control System
An intelligent traffic management system using YOLOv5, Arduino UNO, and Android app to dynamically control traffic lights based on real-time vehicle detection.
Python
Arduino
YOLOv5
Java
Code
Demo
More Coming Soon
Currently working on exciting new projects. Stay tuned!
<AS/>
Home
About
Skills
Projects
Education
Achievements
Contact
ACADEMIC JOURNEY
My Education
The academic foundation that shapes my technical thinking.
2025 – 2027
CURRENT
Master of Computer Applications (MCA)
Rajagiri College of Social Sciences
Pursuing advanced studies in computer applications, AI, and software engineering.
2022 – 2025
Bachelor of Computer Applications (BCA)
De Paul Institute of Science & Technology
Completed undergraduate studies with focus on programming, databases, and web technologies.
<AS/>
Home
About
Skills
Projects
Education
Achievements
Contact
Contact Me
Have a project in mind or just want to say hi? I'd love to hear from you.
Let's Build Something Amazing Together
I'm always open to discussing new projects, creative ideas, or opportunities to be part of your vision. Drop me a message and I'll get back to you!
Email Address
ashbi404@gmail.com
Location
Kerala, India
GitHub
@ashbisaju
Send a Message
I typically reply within 24 hours.
Your Name *
John Doe
Email Address *
john@example.com
Subject
Project Collaboration
Message *
Tell me about your project or idea...
Send Message

Program No: 31	**Date:** 13-02-2026

Program Title: Create a role-based authentication system using Django.

```
<!--
Role-Based Authentication
@ Ashbi Saju
Roll no: 16
Date: 13-02-2026
-->
```

urls.py

```
from django.urls import path
from django.contrib.auth import views as auth_views
from .views import dashboard

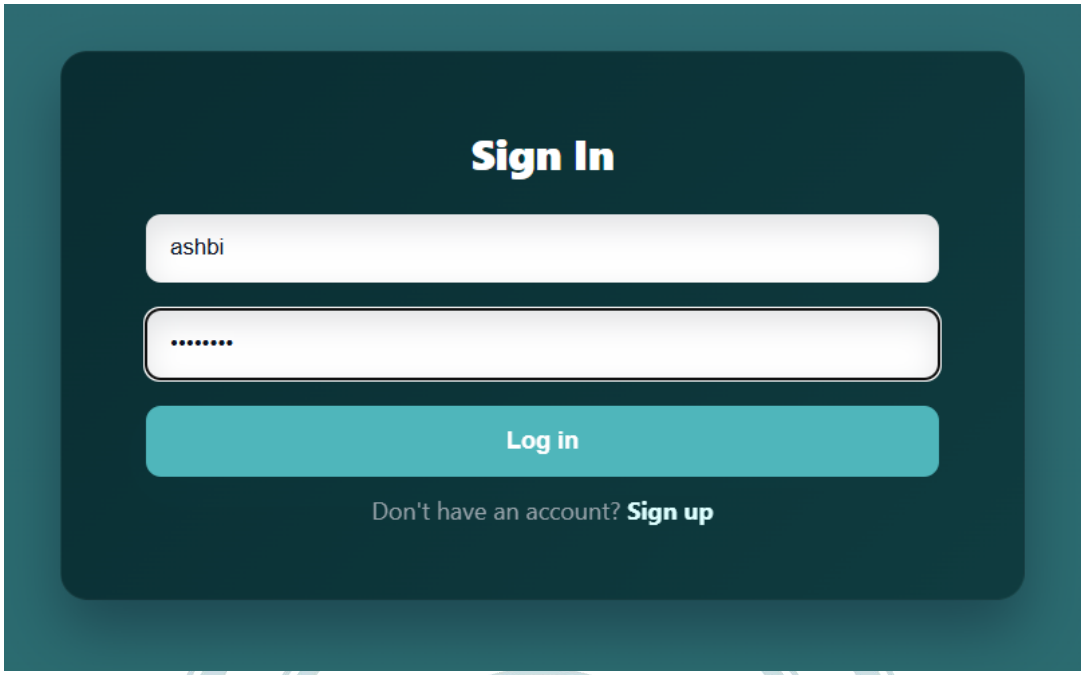
urlpatterns = [
   path('login/', auth_views.LoginView.as_view(template_name='accounts/login.html'),
name='login'),
   path('logout/', auth_views.LogoutView.as_view(), name='logout'),
   path('', dashboard, name='dashboard'),
]
```

views.py

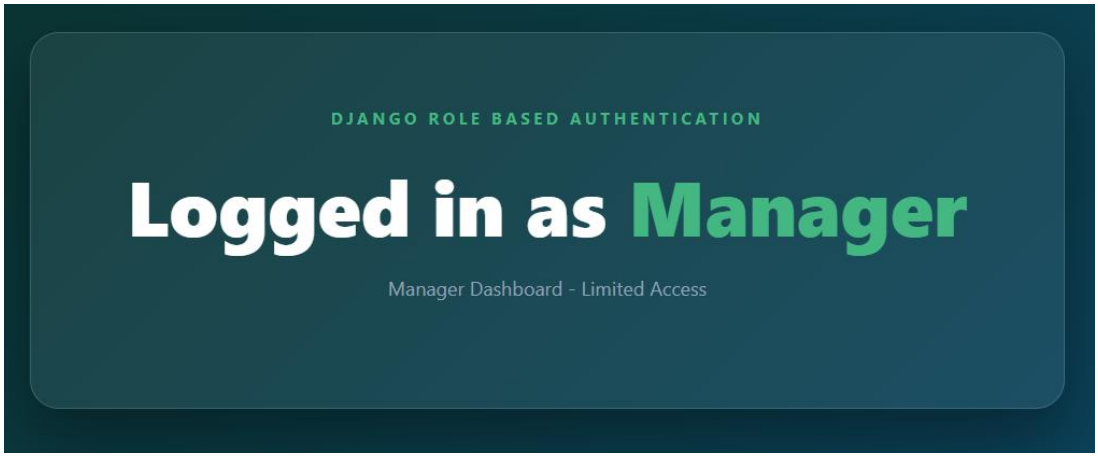
```
from django.shortcuts import render
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
   if request.user.groups.filter(name='Admin').exists():
      return render(request, 'accounts/admin_dashboard.html')
   elif request.user.groups.filter(name='Manager').exists():
      return render(request, 'accounts/manager_dashboard.html')
   elif request.user.groups.filter(name='Employee').exists():
      return render(request, 'accounts/employee_dashboard.html')
   else:
      return render(request, 'accounts/no_role.html')
```

Output


Sign In
ashbi
Log in
Don't have an account? Sign up


DJANGO ROLE BASED AUTHENTICATION
Logged in as Admin
Admin Dashboard - Full Access


DJANGO ROLE BASED AUTHENTICATION
Logged in as Manager
Manager Dashboard - Limited Access

Program No: 32	**Date:** 13-02-2026

Program Title: Develop a student management system with CRUD operations.

```
<!--
Student Management System
@ Ashbi Saju
Roll no: 16
Date: 13-02-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_student_list, name='home'),
    path('student/', views.student_student_list,   name='student_list'),
    path('student/add/', views.student_student_create, name='student_create'),
    path('student/<int:pk>/', views.student_student_detail, name='student_detail'),
    path('student/<int:pk>/edit/', views.student_student_edit,   name='student_edit'),
    path('student/<int:pk>/delete/', views.student_student_delete, name='student_delete'),
]
```

views.py

```
from django.shortcuts import render, get_object_or_404, redirect
from .models import Student

def student_student_list(request):
    students = Student.objects.all()
    return render(request, 'students/student_list.html', {'students': students})

def student_student_detail(request, pk):
    student = get_object_or_404(Student, pk=pk)
    return render(request, 'students/student_detail.html', {'student': student})

def student_student_create(request):
    if request.method == 'POST':
        Student.objects.create(
            name    = request.POST['name'],
            roll_no = request.POST['roll_no'],
            branch  = request.POST['branch'],
            year    = request.POST['year'],
            email   = request.POST.get('email', ''),
```

```
            phone   = request.POST.get('phone', ''),
        )
        return redirect('student_list')
    return render(request, 'students/student_form.html',
    {'action': 'Add', 'view_type': 'STUDENT'})

def student_student_edit(request, pk):
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        student.name    = request.POST['name']
        student.roll_no = request.POST['roll_no']
        student.branch  = request.POST['branch']
        student.year    = request.POST['year']
        student.email   = request.POST.get('email', '')
        student.phone   = request.POST.get('phone', '')
        student.save()
        return redirect('student_list')
    return render(request, 'students/student_form.html',
    {'student': student, 'action': 'Edit', 'view_type': 'STUDENT'})

def student_student_delete(request, pk):
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        student.delete()
        return redirect('student_list')
    return render(request, 'students/student_confirm_delete.html',
    {'student': student, 'view_type': 'STUDENT'})
```

admins.py

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display   = ('roll_no', 'name', 'branch', 'year', 'email', 'phone')
    list_filter    = ('branch', 'year')
    search_fields  = ('name', 'roll_no', 'email')
    ordering       = ('roll_no',)
```

models.py

```
from django.db import models

BRANCH_CHOICES = [
    ('CSE', 'Computer Science'),
```

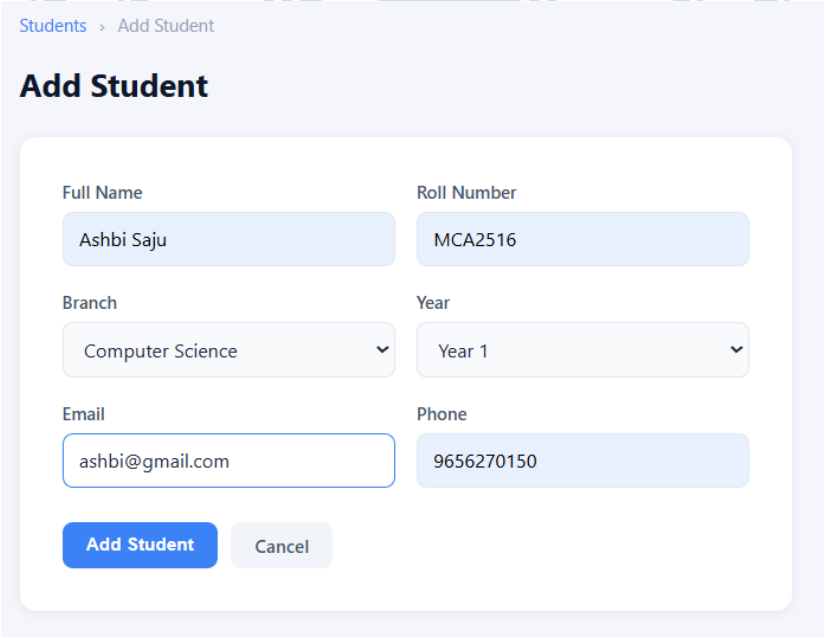
```
    ('ECE', 'Electronics'),
    ('ME',  'Mechanical'),
    ('CE',  'Civil'),
]

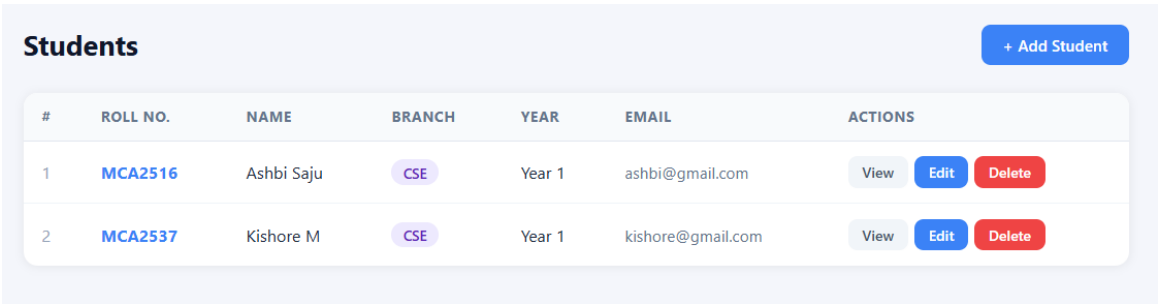
class Student(models.Model):
    name     = models.CharField(max_length=100)
    roll_no  = models.CharField(max_length=20, unique=True)
    branch   = models.CharField(max_length=10, choices=BRANCH_CHOICES)
    year     = models.IntegerField()
    email    = models.EmailField(blank=True)
    phone    = models.CharField(max_length=15, blank=True)

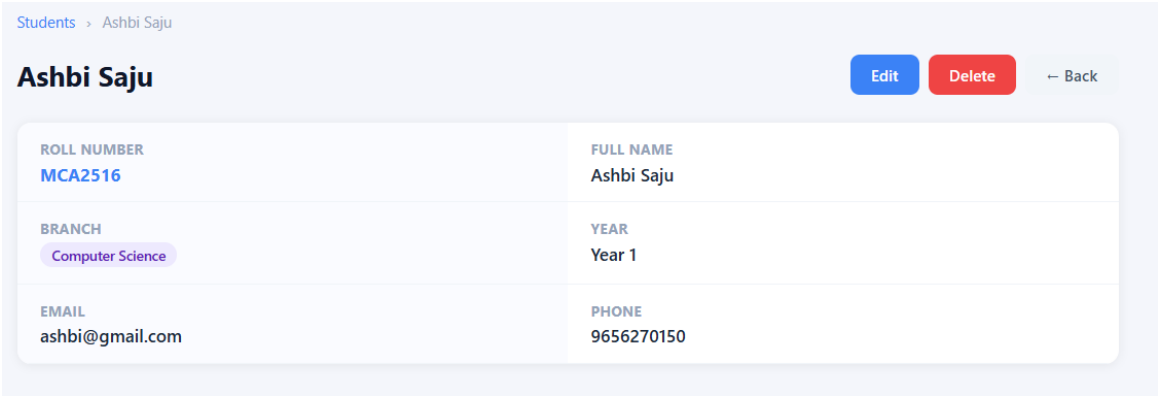
    def __str__(self):
        return f"{self.name} ({self.roll_no})"

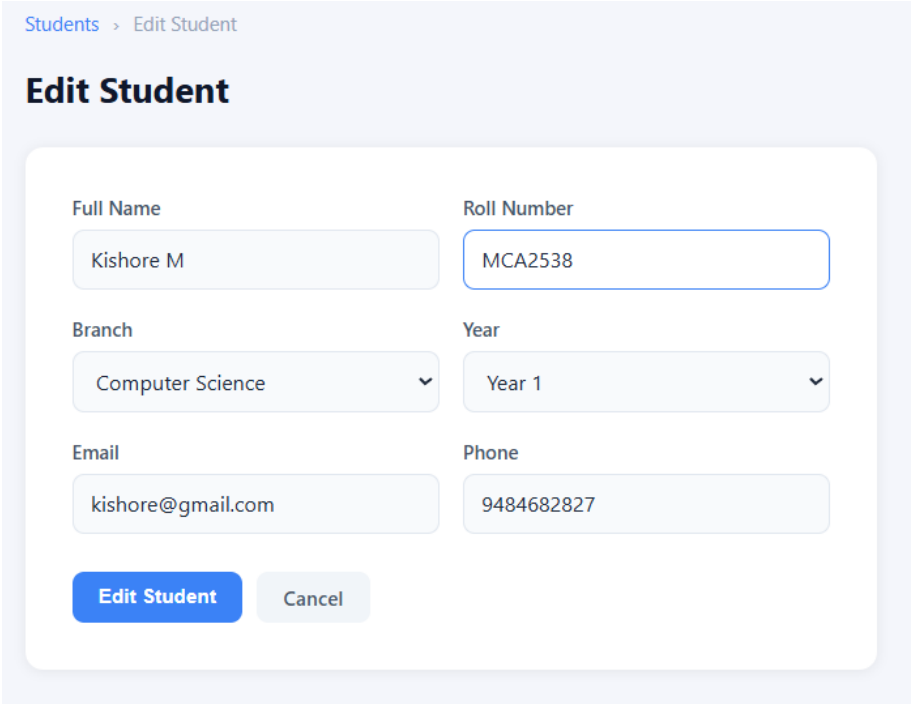
    class Meta:
        ordering = ['roll_no']
```

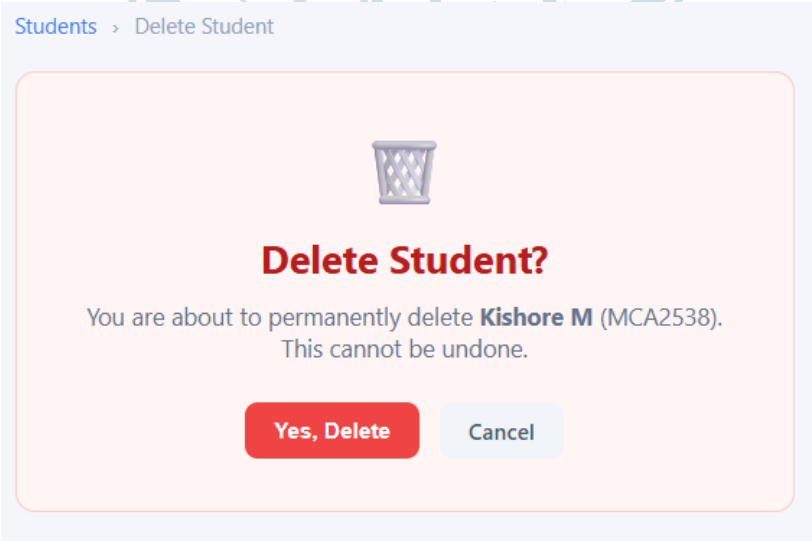
Output






Students › Ashbi Saju
Ashbi Saju
Edit
Delete
← Back
ROLL NUMBER
MCA2516
FULL NAME
Ashbi Saju
BRANCH
Computer Science
YEAR
Year 1
EMAIL
ashbi@gmail.com
PHONE
9656270150


Students › Edit Student
Edit Student
Full Name
Kishore M
Roll Number
MCA2538
Branch
Computer Science
Year
Year 1
Email
kishore@gmail.com
Phone
9484682827
Edit Student
Cancel


Students › Delete Student
Delete Student?
You are about to permanently delete Kishore M (MCA2538).
This cannot be undone.
Yes, Delete
Cancel

Program No: 33	**Date:** 13-02-2026

Program Title: Create simple online feedback system with database integration.

```
<!--
Online Feedback System
@ Ashbi Saju
Roll no: 16
Date: 13-02-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.submit_feedback, name='submit_feedback'),
    path('success/', views.success, name='success'),
    path('feedback/', views.all_feedback, name='all_feedback'),
]
```

views.py

```
from django.shortcuts import render, redirect
from django.db.models import Avg, Count
from .forms import FeedbackForm
from .models import Feedback

def submit_feedback(request):
    if request.method == 'POST':
        form = FeedbackForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('success')
    else:
        form = FeedbackForm()
    return render(request, 'feedback/submit.html', {'form': form})

def success(request):
    return render(request, 'feedback/success.html')

def all_feedback(request):
    feedbacks = Feedback.objects.all()
    stats = Feedback.objects.aggregate(
```

```
        avg_rating=Avg('rating'),
        total=Count('id'),
    )
    category_counts = Feedback.objects.values('category').annotate(count=Count('id'))
    return render(request, 'feedback/list.html', {
        'feedbacks': feedbacks,
        'stats': stats,
        'category_counts': category_counts,
    })
```

admins.py

```
from django.contrib import admin
from .models import Feedback

@admin.register(Feedback)
class FeedbackAdmin(admin.ModelAdmin):
    list_display = ['name', 'email', 'category', 'rating', 'subject', 'submitted_at']
    list_filter  = ['category', 'rating', 'submitted_at']
    search_fields = ['name', 'email', 'subject', 'message']
    readonly_fields = ['submitted_at']
```

models.py

```
from django.db import models

RATING_CHOICES = [
    (1, '⭐ Poor'),
    (2, '⭐ ⭐ Fair'),
    (3, '⭐ ⭐ ⭐ Good'),
    (4, '⭐ ⭐ ⭐ ⭐ Very Good'),
    (5, '⭐ ⭐ ⭐ ⭐ ⭐ Excellent'),
]

CATEGORY_CHOICES = [
    ('general', 'General'),
    ('service', 'Service'),
    ('product', 'Product'),
    ('support', 'Support'),
    ('other', 'Other'),
]

class Feedback(models.Model):
    name       = models.CharField(max_length=100)
    email      = models.EmailField()
```

LEARN • SERVE • EXCEL RAJAGIRI

```
    category    = models.CharField(max_length=20, choices=CATEGORY_CHOICES,
default='general')
    rating      = models.IntegerField(choices=RATING_CHOICES)
    subject     = models.CharField(max_length=200)
    message     = models.TextField()
    submitted_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-submitted_at']

    def __str__(self):
        return f"{self.name} — {self.subject} ({self.get_rating_display()})"
```

forms.py

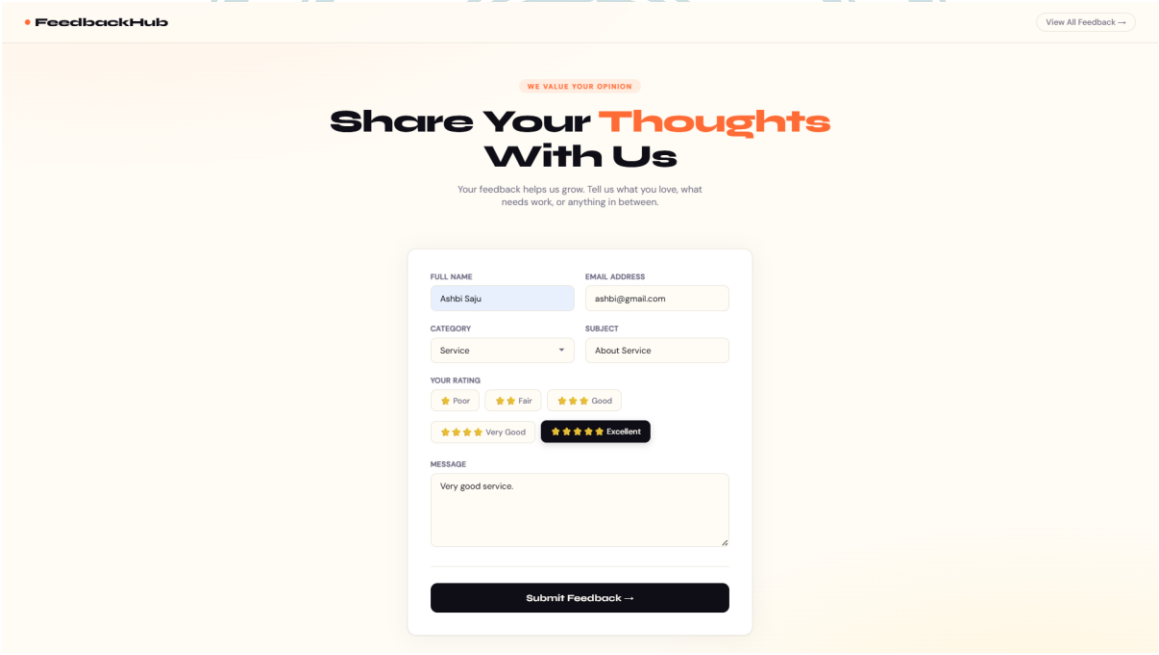
```
from django import forms
from .models import Feedback, RATING_CHOICES, CATEGORY_CHOICES

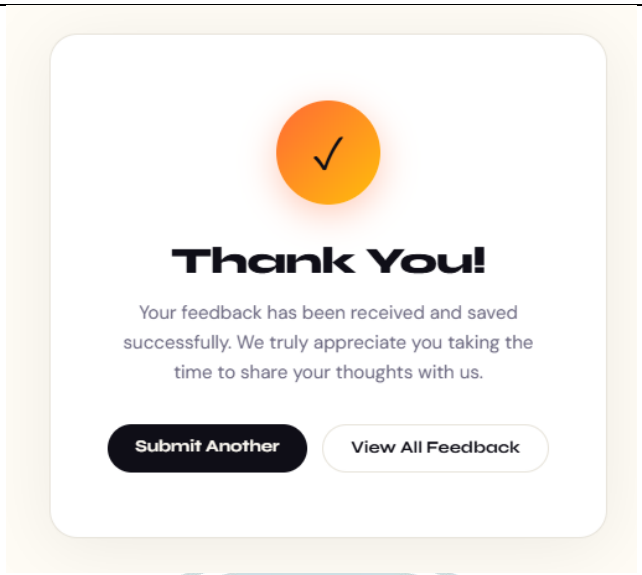
class FeedbackForm(forms.ModelForm):
    name = forms.CharField(
        max_length=100,
        widget=forms.TextInput(attrs={
            'placeholder': 'Your full name',
            'class': 'form-input',
        })
    )
    email = forms.EmailField(
        widget=forms.EmailInput(attrs={
            'placeholder': 'your@email.com',
            'class': 'form-input',
        })
    )
    category = forms.ChoiceField(
        choices=CATEGORY_CHOICES,
        widget=forms.Select(attrs={
            'class': 'form-input',
        })
    )
    rating = forms.ChoiceField(
        choices=RATING_CHOICES,
        widget=forms.RadioSelect(attrs={
            'class': 'rating-radio',
        })
    )
    subject = forms.CharField(
        max_length=200,
```

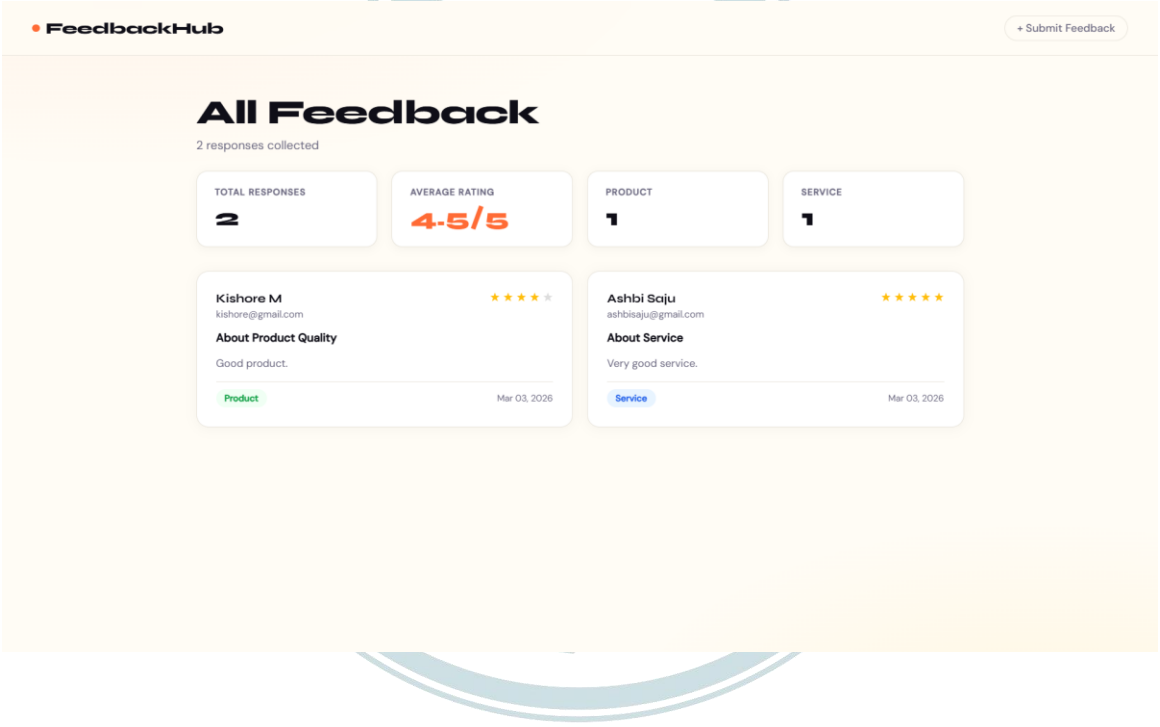
```
        widget=forms.TextInput(attrs={
            'placeholder': 'Brief subject of your feedback',
            'class': 'form-input',
        })
    )
    message = forms.CharField(
        widget=forms.Textarea(attrs={
            'placeholder': 'Tell us what you think...',
            'rows': 5,
            'class': 'form-input',
        })
    )

    class Meta:
        model = Feedback
        fields = ['name', 'email', 'category', 'rating', 'subject', 'message']
```

Output




Thank You!
Your feedback has been received and saved successfully. We truly appreciate you taking the time to share your thoughts with us.
Submit Another
View All Feedback


FeedbackHub
+ Submit Feedback
All Feedback
2 responses collected
TOTAL RESPONSES
2
AVERAGE RATING
4.5/5
PRODUCT
1
SERVICE
1
Kishore M
kishore@gmail.com
About Product Quality
Good product.
Product
Mar 03, 2026
Ashbi Saju
ashbisaju@gmail.com
About Service
Very good service.
Service
Mar 03, 2026

Program No: 34	**Date:** 13-02-2026

Program Title: Create a blog application.

```
<!--
Django Blog Application
@ Ashbi Saju
Roll no: 16
Date: 13-02-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('about/', views.about, name='about'),
    path('post/<slug:slug>/', views.post_detail, name='post_detail'),
]
```

views.py

```
from django.shortcuts import render, get_object_or_404
from django.http import Http404

def home(request):
    category = request.GET.get("category", "")
    search = request.GET.get("search", "").strip()

    posts = POSTS
    if category:
        posts = [p for p in posts if p["category"] == category]
    if search:
        posts = [p for p in posts if search.lower() in p["title"].lower()
                 or search.lower() in p["excerpt"].lower()]

    return render(request, "blog/home.html", {
        "posts": posts,
        "categories": CATEGORIES,
        "active_category": category,
        "search": search,
        "total": len(POSTS),
    })
```

```
def post_detail(request, slug):
    post = get_post_by_slug(slug)
    if not post:
        raise Http404("Post not found")

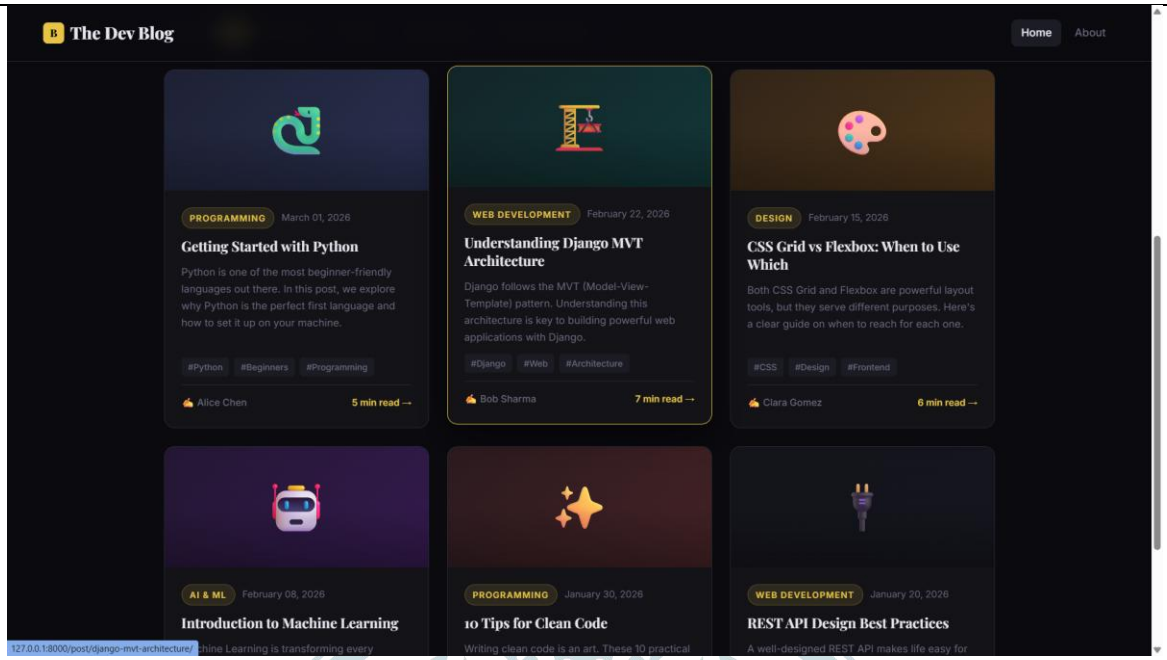
    # Related posts: same category, exclude current
    related = [p for p in POSTS if p["category"] == post["category"] and p["slug"] != slug][:2]

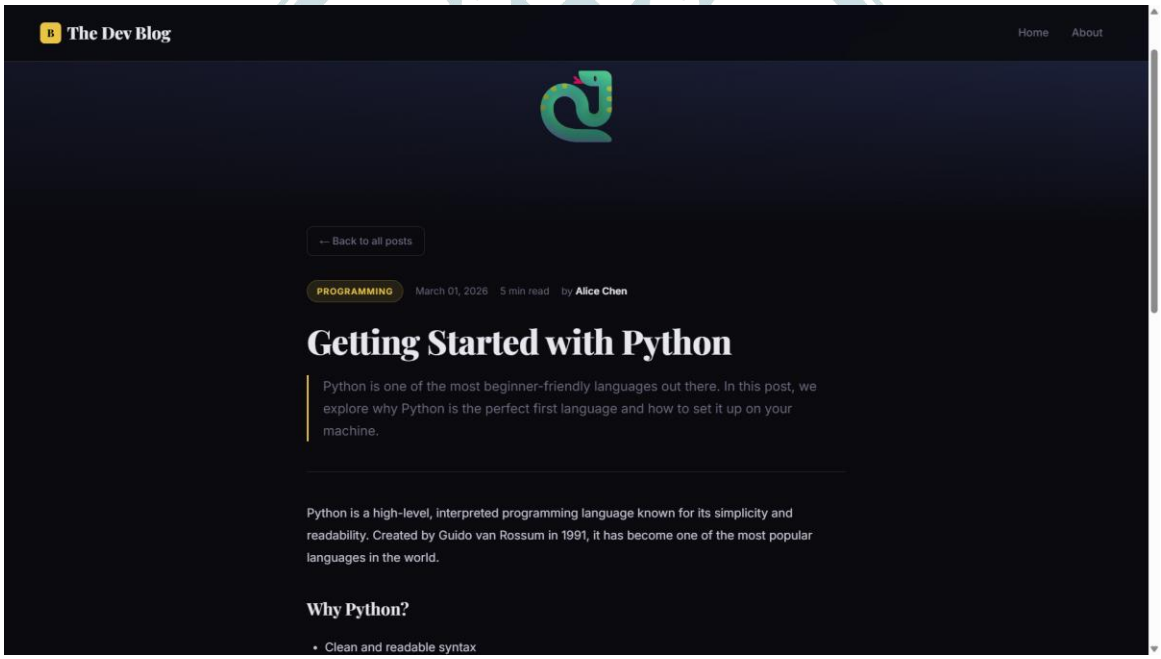
    return render(request, "blog/detail.html", {
        "post": post,
        "related": related,
    })

def about(request):
    authors = {}
    for p in POSTS:
        authors[p["author"]] = authors.get(p["author"], 0) + 1
    return render(request, "blog/about.html", {
        "authors": authors,
        "total_posts": len(POSTS),
        "categories": CATEGORIES,
    })
```

Output




The Dev Blog
Home
About
PROGRAMMING
March 01, 2026
Getting Started with Python
Python is one of the most beginner-friendly languages out there. In this post, we explore why Python is the perfect first language and how to set it up on your machine.
#Python
#Beginners
#Programming
Alice Chen
5 min read →
WEB DEVELOPMENT
February 22, 2026
Understanding Django MVT Architecture
Django follows the MVT (Model-View-Template) pattern. Understanding this architecture is key to building powerful web applications with Django.
#Django
#Web
#Architecture
Bob Sharma
7 min read →
DESIGN
February 15, 2026
CSS Grid vs Flexbox: When to Use Which
Both CSS Grid and Flexbox are powerful layout tools, but they serve different purposes. Here's a clear guide on when to reach for each one.
#CSS
#Design
#Frontend
Clara Gomez
6 min read →
AI & ML
February 08, 2026
Introduction to Machine Learning
PROGRAMMING
January 30, 2026
10 Tips for Clean Code
Writing clean code is an art. These 10 practical
WEB DEVELOPMENT
January 20, 2026
REST API Design Best Practices
A well-designed REST API makes life easy for
127.0.0.1:8000/post/django-mvt-architecture/


The Dev Blog
Home
About
← Back to all posts
PROGRAMMING
March 01, 2026
5 min read
by Alice Chen
Getting Started with Python
Python is one of the most beginner-friendly languages out there. In this post, we explore why Python is the perfect first language and how to set it up on your machine.
Python is a high-level, interpreted programming language known for its simplicity and readability. Created by Guido van Rossum in 1991, it has become one of the most popular languages in the world.
Why Python?
Clean and readable syntax

Program No: 35	**Date:** 20-02-2026

Program Title: Implement session management in Django.

```
<!--
Session Management Django
@ Ashbi Saju
Roll no: 16
Date: 20-02-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('set-theme/', views.set_theme, name='set_theme'),
    path('add-message/', views.add_message, name='add_message'),
    path('clear-messages/', views.clear_messages, name='clear_messages'),
    path('set-expiry/', views.set_expiry, name='set_expiry'),
]
```

views.py

```
from django.shortcuts import render, redirect

# Home — shows session info
def home(request):
    visit_count = request.session.get('visit_count', 0) + 1
    request.session['visit_count'] = visit_count

    username = request.session.get('username', None)
    theme    = request.session.get('theme', 'light')
    messages = request.session.get('messages', [])

    return render(request, 'home.html', {
        'visit_count': visit_count,
        'username':    username,
        'theme':       theme,
        'messages':    messages,
        'session_key': request.session.session_key,
    })
```

```
# Login — store username in session
def login_view(request):
    error = None
    if request.method == 'POST':
        username = request.POST.get('username', '').strip()
        password = request.POST.get('password', '').strip()
        if username and password == 'pass123':
            request.session['username'] = username
            request.session['messages'] = request.session.get('messages', [])
            return redirect('home')
        else:
            error = 'Invalid credentials. Use any username with password: pass123'
    return render(request, 'login.html', {'error': error})

# Logout — clear session
def logout_view(request):
    request.session.flush()
    return redirect('home')

# Set theme preference in session
def set_theme(request):
    if request.method == 'POST':
        theme = request.POST.get('theme', 'light')
        request.session['theme'] = theme
    return redirect('home')

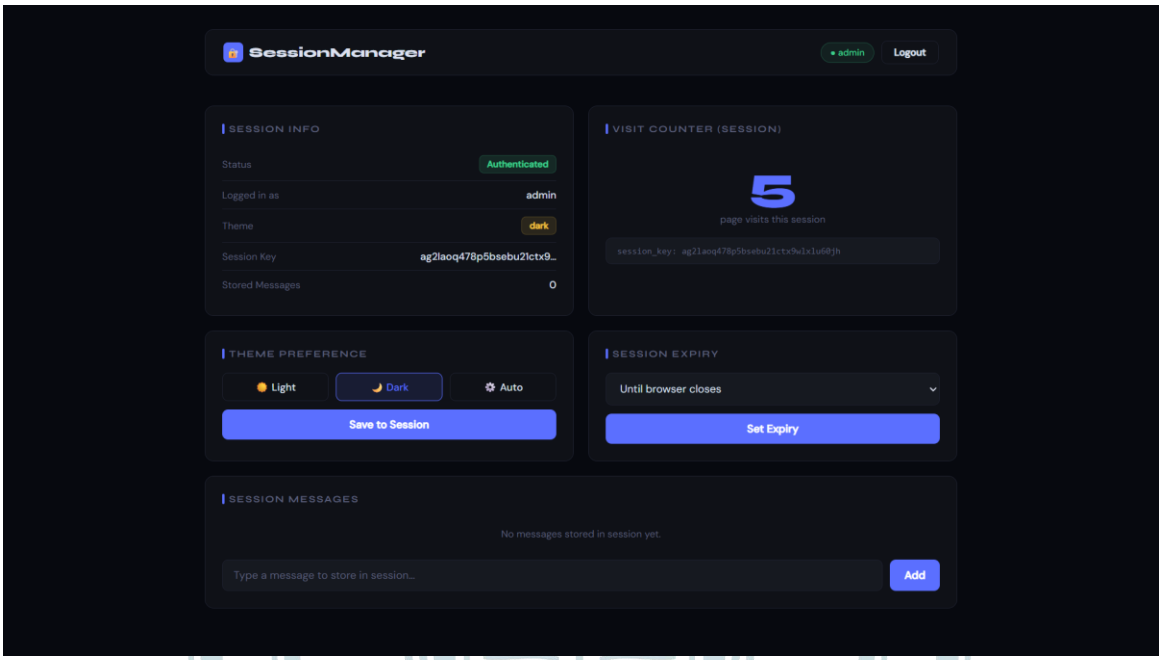
# Add a message to session
def add_message(request):
    if request.method == 'POST':
        msg = request.POST.get('message', '').strip()
        if msg:
            msgs = request.session.get('messages', [])
            msgs.append(msg)
            request.session['messages'] = msgs
            request.session.modified = True
    return redirect('home')

# Clear messages from session
def clear_messages(request):
    request.session['messages'] = []
    request.session.modified = True
    return redirect('home')

# Set session expiry
def set_expiry(request):
    if request.method == 'POST':
```

```
    expiry = int(request.POST.get('expiry', 0))
    request.session.set_expiry(expiry)
return redirect('home')
```

Output



LEARN • SERVE • EXCEL
RAJAGIRI

Program No: 36	**Date:** 20-02-2026

Program Title: Implement pagination and search in a Django application.

```
<!--
Pagination & Search
@ Ashbi Saju
Roll no: 16
Date: 20-02-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.book_list, name='book_list'),
]
```

views.py

```
from django.shortcuts import render
from django.core.paginator import Paginator, EmptyPage, PageNotAnInteger
from .models import Book

def book_list(request):
    query  = request.GET.get('q', '').strip()
    genre  = request.GET.get('genre', '')
    sort   = request.GET.get('sort', 'title')
    per_page = int(request.GET.get('per_page', 6))

    books = Book.objects.all()

    # --- Search ---
    if query:
        books = books.filter(title__icontains=query) | books.filter(author__icontains=query)

    # --- Genre filter ---
    if genre:
        books = books.filter(genre=genre)

    # --- Sort ---
    sort_map = {
        'title':   'title',
```

```
    '-title':  '-title',
    'author':  'author',
    'year':    '-year',
    'rating':  '-rating',
}
books = books.order_by(sort_map.get(sort, 'title'))

total = books.count()

# --- Pagination ---
paginator = Paginator(books, per_page)
page_number = request.GET.get('page', 1)

try:
    page_obj = paginator.page(page_number)
except PageNotAnInteger:
    page_obj = paginator.page(1)
except EmptyPage:
    page_obj = paginator.page(paginator.num_pages)

# Build page range (show max 5 pages around current)
current = page_obj.number
total_pages = paginator.num_pages
start = max(current - 2, 1)
end   = min(current + 2, total_pages)
page_range = range(start, end + 1)

# Preserve GET params (except page) for pagination links
get_params = request.GET.copy()
get_params.pop('page', None)
query_string = get_params.urlencode()

from .models import GENRE_CHOICES
return render(request, 'books/book_list.html', {
    'page_obj':     page_obj,
    'query':        query,
    'genre':        genre,
    'sort':         sort,
    'per_page':     per_page,
    'total':        total,
    'page_range':   page_range,
    'query_string': query_string,
    'genre_choices': GENRE_CHOICES,
})
```

admins.py

```
from django.contrib import admin
from .models import Book

@admin.register(Book)
class BookAdmin(admin.ModelAdmin):
   list_display  = ['title', 'author', 'genre', 'year', 'rating']
   list_filter   = ['genre', 'year']
   search_fields = ['title', 'author']
```

models.py

```
from django.db import models

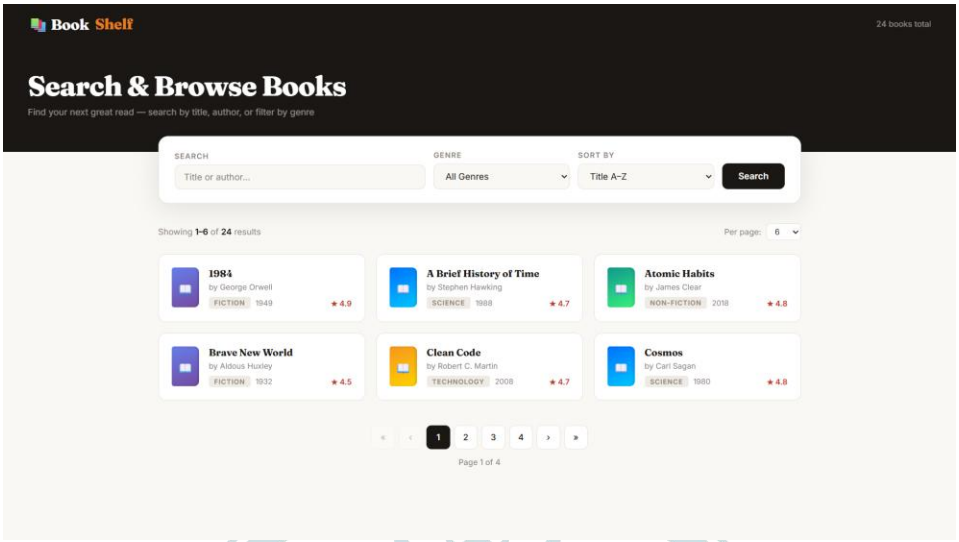
GENRE_CHOICES = [
   ('fiction',    'Fiction'),
   ('non-fiction', 'Non-Fiction'),
   ('science',    'Science'),
   ('technology',  'Technology'),
   ('history',    'History'),
   ('biography',   'Biography'),
]

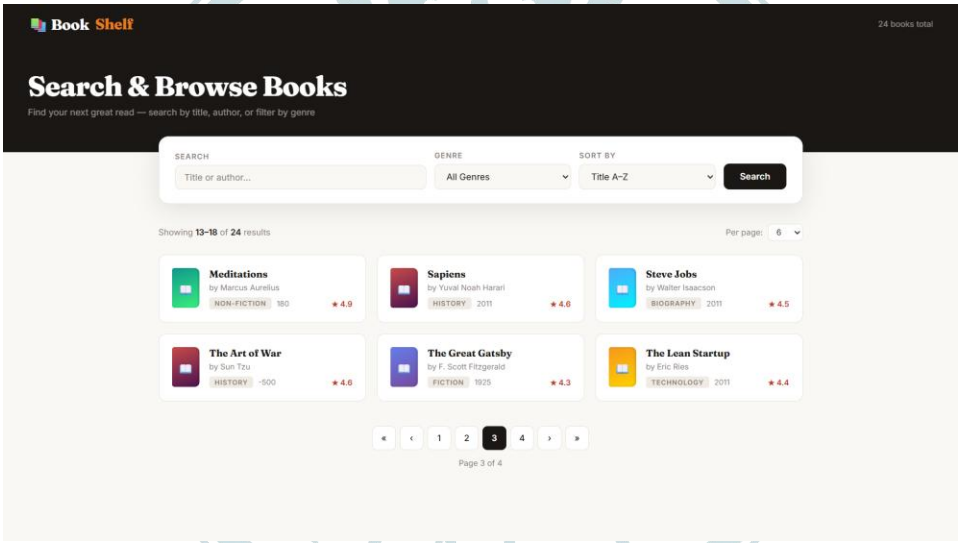
class Book(models.Model):
   title     = models.CharField(max_length=200)
   author    = models.CharField(max_length=100)
   genre     = models.CharField(max_length=20, choices=GENRE_CHOICES)
   year      = models.IntegerField()
   rating    = models.DecimalField(max_digits=3, decimal_places=1)
   description = models.TextField(blank=True)

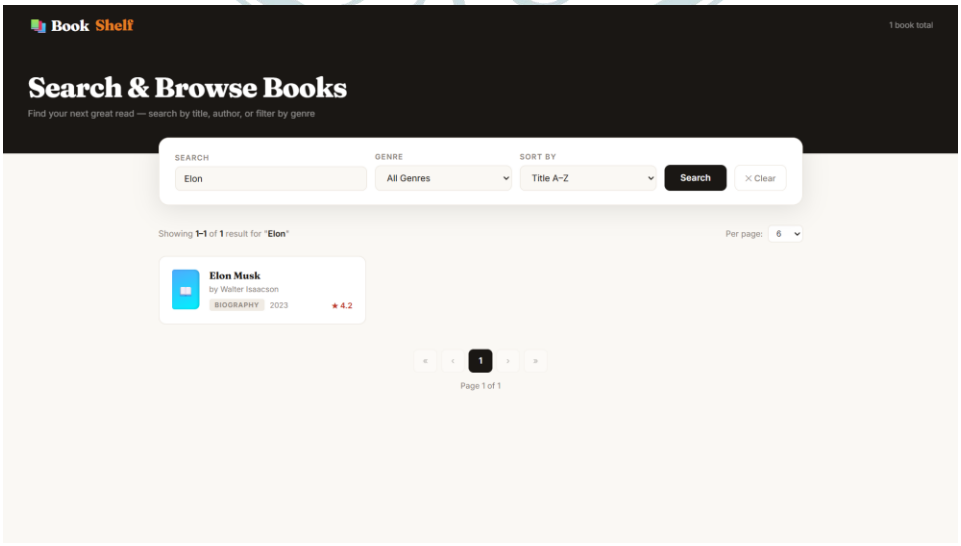
   class Meta:
      ordering = ['title']

   def __str__(self):
      return self.title
```

Output


Book Shelf
24 books total
Search & Browse Books
Find your next great read — search by title, author, or filter by genre
SEARCH
Title or author...
GENRE
All Genres
SORT BY
Title A–Z
Search
Showing 1–6 of 24 results
Per page: 6
1984
by George Orwell
FICTION 1949 ★ 4.9
A Brief History of Time
by Stephen Hawking
SCIENCE 1988 ★ 4.7
Atomic Habits
by James Clear
NON-FICTION 2018 ★ 4.8
Brave New World
by Aldous Huxley
FICTION 1932 ★ 4.5
Clean Code
by Robert C. Martin
TECHNOLOGY 2008 ★ 4.7
Cosmos
by Carl Sagan
SCIENCE 1980 ★ 4.8
« ‹ 1 2 3 4 › »
Page 1 of 4


Book Shelf
24 books total
Search & Browse Books
Find your next great read — search by title, author, or filter by genre
SEARCH
Title or author...
GENRE
All Genres
SORT BY
Title A–Z
Search
Showing 13–18 of 24 results
Per page: 6
Meditations
by Marcus Aurelius
NON-FICTION 180 ★ 4.9
Sapiens
by Yuval Noah Harari
HISTORY 2011 ★ 4.6
Steve Jobs
by Walter Isaacson
BIOGRAPHY 2011 ★ 4.5
The Art of War
by Sun Tzu
HISTORY -500 ★ 4.6
The Great Gatsby
by F. Scott Fitzgerald
FICTION 1925 ★ 4.3
The Lean Startup
by Eric Ries
TECHNOLOGY 2011 ★ 4.4
« ‹ 1 2 3 4 › »
Page 3 of 4


Book Shelf
1 book total
Search & Browse Books
Find your next great read — search by title, author, or filter by genre
SEARCH
Elon
GENRE
All Genres
SORT BY
Title A–Z
Search
× Clear
Showing 1–1 of 1 result for "Elon"
Per page: 6
Elon Musk
by Walter Isaacson
BIOGRAPHY 2023 ★ 4.2
1
Page 1 of 1

Program No: 37	**Date:** 20-02-2026

Program Title: Implement form validation and error handling in Django.

```
<!--
Form Validation Django
@ Ashbi Saju
Roll no: 16
Date: 20-02-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('',       views.register, name='register'),
    path('success/', views.success,  name='success'),
]
```

views.py

```
from django.shortcuts import render, redirect
from django.contrib import messages
from .forms import RegistrationForm
from .models import Registration

def register(request):
    if request.method == 'POST':
        form = RegistrationForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, f"Account created for @{form.cleaned_data['username']}!")
            return redirect('success')
        else:
            messages.error(request, "Please fix the errors below and try again.")
    else:
        form = RegistrationForm()

    return render(request, 'validator/register.html', {'form': form})

def success(request):
    registrations = Registration.objects.order_by('-created_at')
    return render(request, 'validator/success.html', {'registrations': registrations})
```

admins.py

```
from django.contrib import admin
from .models import Registration

@admin.register(Registration)
class RegistrationAdmin(admin.ModelAdmin):
   list_display  = ['username', 'first_name', 'last_name', 'email', 'age', 'created_at']
   search_fields = ['username', 'email']
```

models.py

```
from django.db import models

class Registration(models.Model):
   first_name = models.CharField(max_length=50)
   last_name  = models.CharField(max_length=50)
   email      = models.EmailField(unique=True)
   username   = models.CharField(max_length=30, unique=True)
   phone      = models.CharField(max_length=15)
   age        = models.IntegerField()
   website    = models.URLField(blank=True)
   bio        = models.TextField(max_length=300, blank=True)
   created_at = models.DateTimeField(auto_now_add=True)

   def __str__(self):
      return f"{self.username} ({self.email})"
```

forms.py

```
import re
from django import forms
from django.core.exceptions import ValidationError
from .models import Registration

# ── Custom field-level validators ──────────────────────────────────────

def validate_no_numbers(value):
   """Name must not contain digits."""
   if any(char.isdigit() for char in value):
      raise ValidationError("Name must not contain numbers.")

def validate_username(value):
   """Username: 4–30 chars, alphanumeric + underscores only."""
```

```
    if len(value) < 4:
        raise ValidationError("Username must be at least 4 characters long.")
    if not re.match(r'^[a-zA-Z0-9_]+$', value):
        raise ValidationError("Username may only contain letters, numbers, and underscores.")

def validate_phone(value):
    """Phone: digits, spaces, +, -, () allowed."""
    cleaned = re.sub(r'[\s\-\(\)\+]', '', value)
    if not cleaned.isdigit():
        raise ValidationError("Enter a valid phone number (digits, spaces, +, -, () allowed).")
    if not (7 <= len(cleaned) <= 15):
        raise ValidationError("Phone number must be between 7 and 15 digits.")

# ── Registration Form ──────────────────────────────────────────────

class RegistrationForm(forms.ModelForm):

    # Extra field (not in model) for password confirmation
    password  = forms.CharField(
        widget=forms.PasswordInput(attrs={'placeholder': 'Create a password', 'class': 'fi'}),
        min_length=8,
        help_text="Minimum 8 characters, at least one digit.",
    )
    password2 = forms.CharField(
        widget=forms.PasswordInput(attrs={'placeholder': 'Confirm password', 'class': 'fi'}),
        label="Confirm Password",
    )

    class Meta:
        model  = Registration
        fields = ['first_name', 'last_name', 'username', 'email',
                  'phone', 'age', 'website', 'bio']
        widgets = {
            'first_name': forms.TextInput(attrs={'placeholder': 'John',      'class': 'fi'}),
            'last_name':  forms.TextInput(attrs={'placeholder': 'Doe',       'class': 'fi'}),
            'username':   forms.TextInput(attrs={'placeholder': 'john_doe',  'class': 'fi'}),
            'email':      forms.EmailInput(attrs={'placeholder': 'john@example.com', 'class': 'fi'}),
            'phone':      forms.TextInput(attrs={'placeholder': '+91 98765 43210', 'class': 'fi'}),
            'age':        forms.NumberInput(attrs={'placeholder': '18', 'class': 'fi'}),
            'website':    forms.URLInput(attrs={'placeholder': 'https://yoursite.com', 'class': 'fi'}),
            'bio':        forms.Textarea(attrs={'placeholder': 'Tell us about yourself...', 'rows': 3, 'class':
'fi'}),
        }

    # ── Field-level clean methods ──────────────────────────────────

    def clean_first_name(self):
```

```
        value = self.cleaned_data['first_name'].strip()
        validate_no_numbers(value)
        if len(value) < 2:
            raise ValidationError("First name must be at least 2 characters.")
        return value.title()

    def clean_last_name(self):
        value = self.cleaned_data['last_name'].strip()
        validate_no_numbers(value)
        if len(value) < 2:
            raise ValidationError("Last name must be at least 2 characters.")
        return value.title()

    def clean_username(self):
        value = self.cleaned_data['username'].strip().lower()
        validate_username(value)
        if Registration.objects.filter(username__iexact=value).exists():
            raise ValidationError("This username is already taken.")
        return value

    def clean_email(self):
        value = self.cleaned_data['email'].strip().lower()
        if Registration.objects.filter(email__iexact=value).exists():
            raise ValidationError("An account with this email already exists.")
        return value

    def clean_phone(self):
        value = self.cleaned_data['phone'].strip()
        validate_phone(value)
        return value

    def clean_age(self):
        age = self.cleaned_data['age']
        if age < 13:
            raise ValidationError("You must be at least 13 years old to register.")
        if age > 120:
            raise ValidationError("Please enter a valid age.")
        return age

    def clean_password(self):
        password = self.cleaned_data.get('password', '')
        if not any(char.isdigit() for char in password):
            raise ValidationError("Password must contain at least one digit.")
        if password.islower():
            raise ValidationError("Password must contain at least one uppercase letter.")
        return password
```

```
    # ── Cross-field (form-level) clean ──────────────────────────────────────

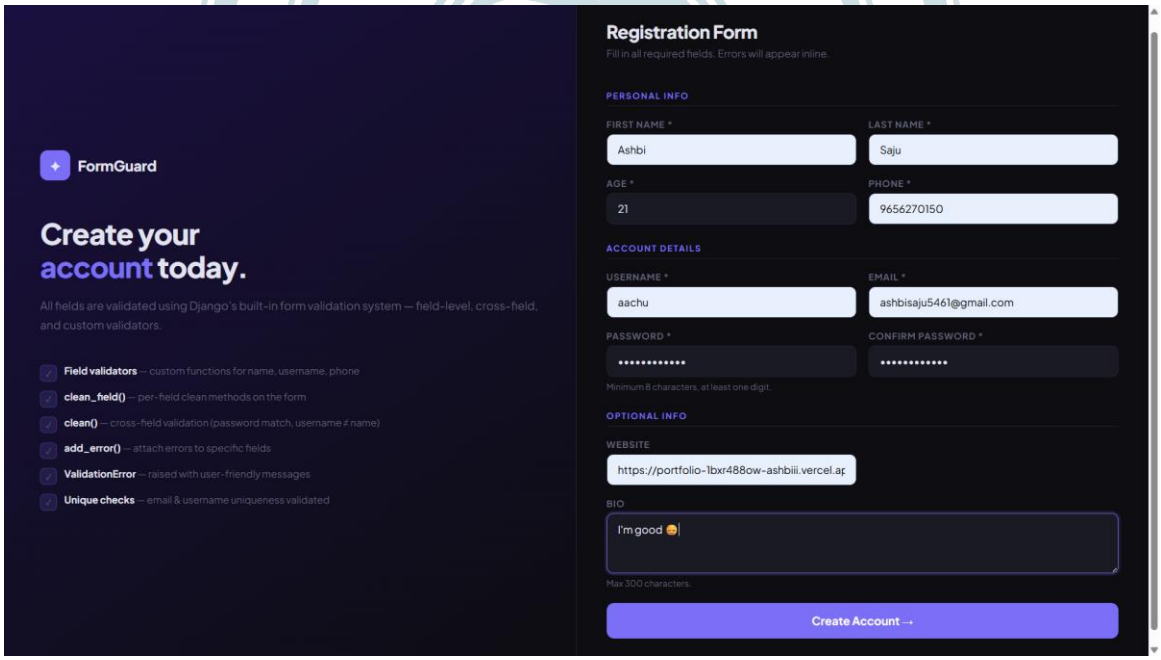
    def clean(self):
        cleaned_data = super().clean()
        password  = cleaned_data.get('password')
        password2 = cleaned_data.get('password2')

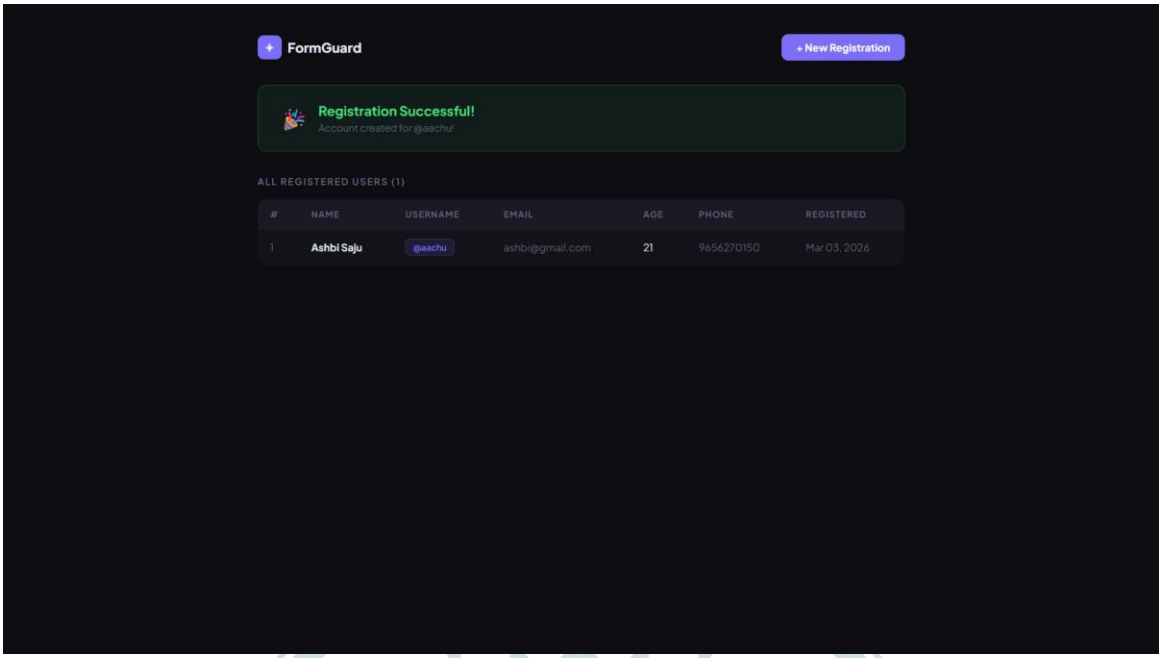
        if password and password2 and password != password2:
            self.add_error('password2', "Passwords do not match.")

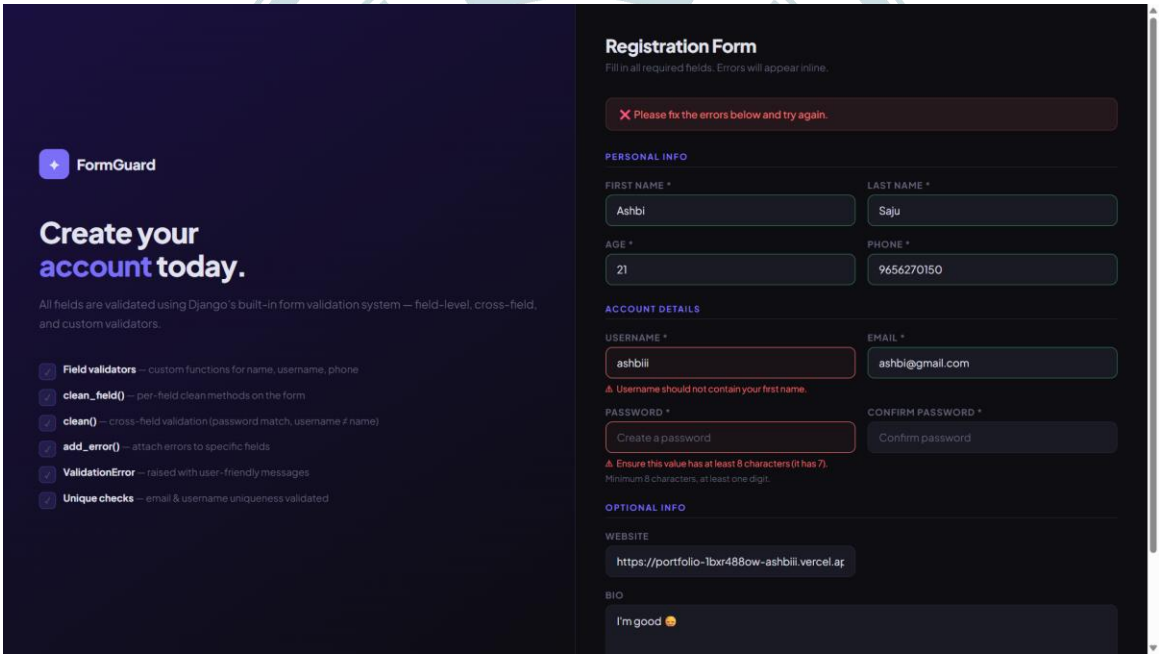
        # Cross-field: username must not contain first name
        username   = cleaned_data.get('username', '')
        first_name = cleaned_data.get('first_name', '').lower()
        if first_name and first_name in username:
            self.add_error('username', "Username should not contain your first name.")

        return cleaned_data
```

Output




FormGuard
+ New Registration
Registration Successful!
Account created for @aachu!
ALL REGISTERED USERS (1)
NAME USERNAME EMAIL AGE PHONE REGISTERED
1 Ashbi Saju @aachu ashbi@gmail.com 21 9656270150 Mar 03, 2026


FormGuard
Create your account today.
All fields are validated using Django's built-in form validation system — field-level, cross-field, and custom validators.
Field validators — custom functions for name, username, phone
clean_field() — per-field clean methods on the form
clean() — cross-field validation (password match, username ≠ name)
add_error() — attach errors to specific fields
ValidationError — raised with user-friendly messages
Unique checks — email & username uniqueness validated
Registration Form
Fill in all required fields. Errors will appear inline.
Please fix the errors below and try again.
PERSONAL INFO
FIRST NAME * Ashbi
LAST NAME * Saju
AGE * 21
PHONE * 9656270150
ACCOUNT DETAILS
USERNAME * ashbiii
Username should not contain your first name.
EMAIL * ashbi@gmail.com
PASSWORD * Create a password
CONFIRM PASSWORD * Confirm password
Ensure this value has at least 8 characters (it has 7).
Minimum 8 characters, at least one digit.
OPTIONAL INFO
WEBSITE https://portfolio-1bxr488ow-ashbiii.vercel.ap
BIO I'm good

Program No: 38	**Date:** 27-02-2026

Program Title: Create a secure authentication system with password hashing.

```
<!--
Secure Authentication System
@ Ashbi Saju
Roll no: 16
Date: 27-02-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('',              views.dashboard,      name='home'),
    path('register/',     views.register_view,  name='register'),
    path('login/',        views.login_view,     name='login'),
    path('logout/',       views.logout_view,    name='logout'),
    path('dashboard/',    views.dashboard,      name='dashboard'),
    path('change-password/', views.change_password, name='change_password'),
]
```

views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout, update_session_auth_hash
from django.contrib.auth.decorators import login_required
from django.contrib.auth.hashers import identify_hasher
from django.contrib import messages

from .forms import RegisterForm, LoginForm, ChangePasswordForm
from .models import UserProfile, LoginLog

def get_client_ip(request):
    x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
    if x_forwarded_for:
        return x_forwarded_for.split(',')[0].strip()
    return request.META.get('REMOTE_ADDR')

# ── Register ──────────────────────────────────────────────────────────
def register_view(request):
    if request.user.is_authenticated:
```

```
        return redirect('dashboard')

    form = RegisterForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        user = form.save()
        # Create profile
        initials = (user.first_name[:1] + user.last_name[:1]).upper() or user.username[:2].upper()
        UserProfile.objects.create(user=user, avatar_initial=initials)
        messages.success(request, "Account created! Please log in.")
        return redirect('login')

    return render(request, 'accounts/register.html', {'form': form})

# ── Login ──────────────────────────────────────────────────────────────
def login_view(request):
    if request.user.is_authenticated:
        return redirect('dashboard')

    form = LoginForm(request, data=request.POST or None)
    if request.method == 'POST':
        ip = get_client_ip(request)
        ua = request.META.get('HTTP_USER_AGENT', '')[:200]

        if form.is_valid():
            user = form.get_user()
            login(request, user)

            # Update profile login stats
            profile, _ = UserProfile.objects.get_or_create(user=user)
            profile.login_count += 1
            profile.last_login_ip = ip
            profile.save()

            # Log success
            LoginLog.objects.create(user=user, ip_address=ip, user_agent=ua, success=True)

            messages.success(request, f"Welcome back, {user.username}!")
            return redirect('dashboard')
        else:
            # Attempt to log failed login
            from django.contrib.auth.models import User
            username = request.POST.get('username', '')
            try:
                failed_user = User.objects.get(username=username)
                LoginLog.objects.create(user=failed_user, ip_address=ip, user_agent=ua,
success=False)
            except User.DoesNotExist:
```

```
            pass

    return render(request, 'accounts/login.html', {'form': form})

# ── Logout ──────────────────────────────────────────────────────────
@login_required
def logout_view(request):
    logout(request)
    messages.info(request, "You have been logged out.")
    return redirect('login')

# ── Dashboard ───────────────────────────────────────────────────────
@login_required
def dashboard(request):
    user    = request.user
    profile = UserProfile.objects.get_or_create(user=user)[0]
    logs    = LoginLog.objects.filter(user=user)[:8]

    # Show the hashed password algorithm info
    hasher = identify_hasher(user.password)
    hash_info = {
        'algorithm': hasher.algorithm,
        'hash_preview': user.password[:60] + '...',
    }

    return render(request, 'accounts/dashboard.html', {
        'profile':   profile,
        'logs':      logs,
        'hash_info': hash_info,
    })

# ── Change Password ─────────────────────────────────────────────────
@login_required
def change_password(request):
    form = ChangePasswordForm(request.user, request.POST or None)
    if request.method == 'POST' and form.is_valid():
        user = form.save()
        # Keep session alive after password change
        update_session_auth_hash(request, user)
        messages.success(request, "Password changed successfully!")
        return redirect('dashboard')

    return render(request, 'accounts/change_password.html', {'form': form})
```

admins.py

```
from django.contrib import admin
from .models import UserProfile, LoginLog

@admin.register(UserProfile)
class UserProfileAdmin(admin.ModelAdmin):
    list_display  = ['user', 'login_count', 'last_login_ip']
    readonly_fields = ['login_count', 'last_login_ip']

@admin.register(LoginLog)
class LoginLogAdmin(admin.ModelAdmin):
    list_display  = ['user', 'timestamp', 'ip_address', 'success']
    list_filter   = ['success']
    readonly_fields = ['user', 'timestamp', 'ip_address', 'success', 'user_agent']
```

models.py

```
from django.db import models
from django.contrib.auth.models import User

class UserProfile(models.Model):
    user      = models.OneToOneField(User, on_delete=models.CASCADE, related_name='profile')
    bio       = models.TextField(max_length=200, blank=True)
    avatar_initial = models.CharField(max_length=2, blank=True)
    login_count    = models.PositiveIntegerField(default=0)
    last_login_ip  = models.GenericIPAddressField(null=True, blank=True)

    def __str__(self):
        return f"Profile of {self.user.username}"

class LoginLog(models.Model):
    user      = models.ForeignKey(User, on_delete=models.CASCADE, related_name='login_logs')
    timestamp  = models.DateTimeField(auto_now_add=True)
    ip_address = models.GenericIPAddressField(null=True, blank=True)
    success    = models.BooleanField(default=True)
    user_agent = models.CharField(max_length=200, blank=True)

    class Meta:
        ordering = ['-timestamp']

    def __str__(self):
        status = "✓" if self.success else "✗"
        return f"{status} {self.user.username} @ {self.timestamp}"
```

forms.py

```
import re
from django import forms
from django.contrib.auth.models import User
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.password_validation import validate_password
from django.core.exceptions import ValidationError

class RegisterForm(forms.ModelForm):
    password1 = forms.CharField(
        label="Password",
        widget=forms.PasswordInput(attrs={'placeholder': 'Create a strong password', 'class': 'fi'}),
    )
    password2 = forms.CharField(
        label="Confirm Password",
        widget=forms.PasswordInput(attrs={'placeholder': 'Repeat your password', 'class': 'fi'}),
    )

    class Meta:
        model  = User
        fields = ['username', 'email', 'first_name', 'last_name']
        widgets = {
            'username':   forms.TextInput(attrs={'placeholder': 'Choose a username', 'class': 'fi'}),
            'email':      forms.EmailInput(attrs={'placeholder': 'your@email.com',   'class': 'fi'}),
            'first_name': forms.TextInput(attrs={'placeholder': 'First name',       'class': 'fi'}),
            'last_name':  forms.TextInput(attrs={'placeholder': 'Last name',        'class': 'fi'}),
        }

    def clean_username(self):
        username = self.cleaned_data['username'].strip()
        if User.objects.filter(username__iexact=username).exists():
            raise ValidationError("This username is already taken.")
        if not re.match(r'^[a-zA-Z0-9_]{3,30}$', username):
            raise ValidationError("3–30 characters. Letters, numbers, underscores only.")
        return username

    def clean_email(self):
        email = self.cleaned_data['email'].strip().lower()
        if User.objects.filter(email__iexact=email).exists():
            raise ValidationError("An account with this email already exists.")
        return email

    def clean_password1(self):
        password = self.cleaned_data.get('password1')
        # Run Django's built-in validators
        validate_password(password)
```

```
        # Extra: must have uppercase, lowercase, digit, special char
        if not re.search(r'[A-Z]', password):
            raise ValidationError("Password must contain at least one uppercase letter.")
        if not re.search(r'[a-z]', password):
            raise ValidationError("Password must contain at least one lowercase letter.")
        if not re.search(r'\d', password):
            raise ValidationError("Password must contain at least one digit.")
        if not re.search(r'[!@#$%^&*(),.?":{}|<>_\-]', password):
            raise ValidationError("Password must contain at least one special character.")
        return password

    def clean(self):
        cleaned = super().clean()
        p1 = cleaned.get('password1')
        p2 = cleaned.get('password2')
        if p1 and p2 and p1 != p2:
            self.add_error('password2', "Passwords do not match.")
        return cleaned

    def save(self, commit=True):
        user = super().save(commit=False)
        # set_password() hashes the password using PBKDF2 (Django default)
        user.set_password(self.cleaned_data['password1'])
        if commit:
            user.save()
        return user

class LoginForm(AuthenticationForm):
    username = forms.CharField(
        widget=forms.TextInput(attrs={'placeholder': 'Username', 'class': 'fi', 'autofocus': True})
    )
    password = forms.CharField(
        widget=forms.PasswordInput(attrs={'placeholder': 'Password', 'class': 'fi'})
    )

class ChangePasswordForm(forms.Form):
    old_password = forms.CharField(
        label="Current Password",
        widget=forms.PasswordInput(attrs={'placeholder': 'Current password', 'class': 'fi'}),
    )
    new_password1 = forms.CharField(
        label="New Password",
        widget=forms.PasswordInput(attrs={'placeholder': 'New password', 'class': 'fi'}),
    )
    new_password2 = forms.CharField(
        label="Confirm New Password",
        widget=forms.PasswordInput(attrs={'placeholder': 'Confirm new password', 'class': 'fi'}),
```

```
)

def __init__(self, user, *args, **kwargs):
    self.user = user
    super().__init__(*args, **kwargs)

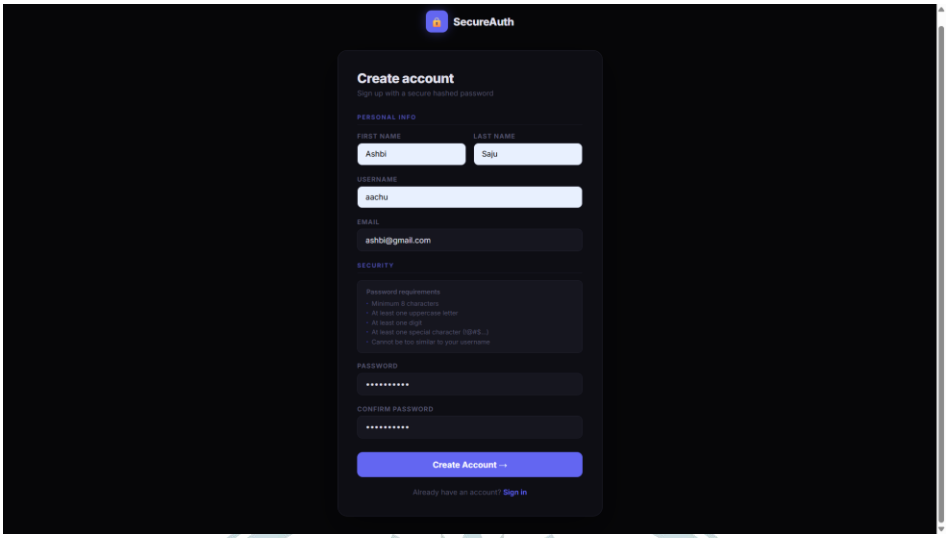
def clean_old_password(self):
    old = self.cleaned_data['old_password']
    if not self.user.check_password(old):
        raise ValidationError("Your current password is incorrect.")
    return old

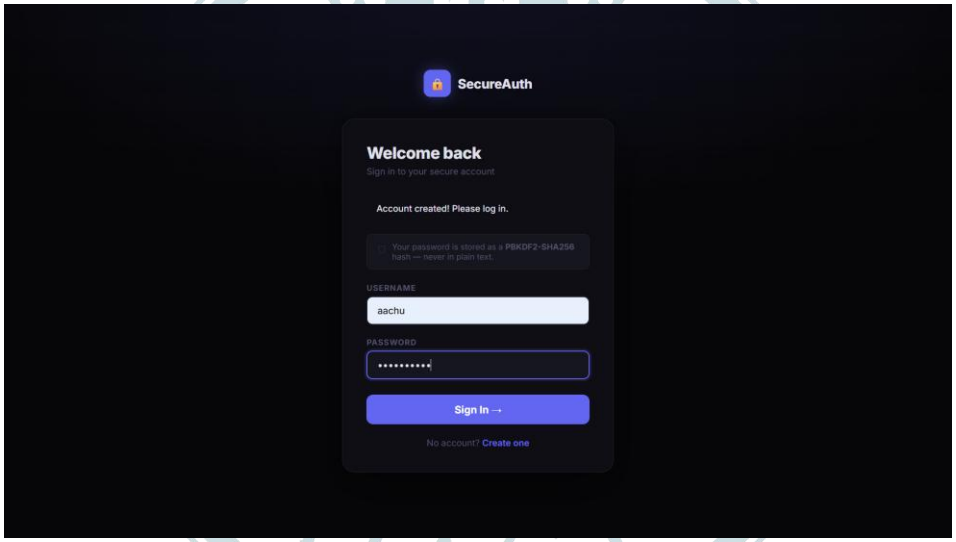
def clean_new_password1(self):
    password = self.cleaned_data.get('new_password1')
    validate_password(password, self.user)
    if not re.search(r'[A-Z]', password):
        raise ValidationError("Must contain at least one uppercase letter.")
    if not re.search(r'\d', password):
        raise ValidationError("Must contain at least one digit.")
    if not re.search(r'[!@#$%^&*(),.?":{}|<>_\-]', password):
        raise ValidationError("Must contain at least one special character.")
    return password

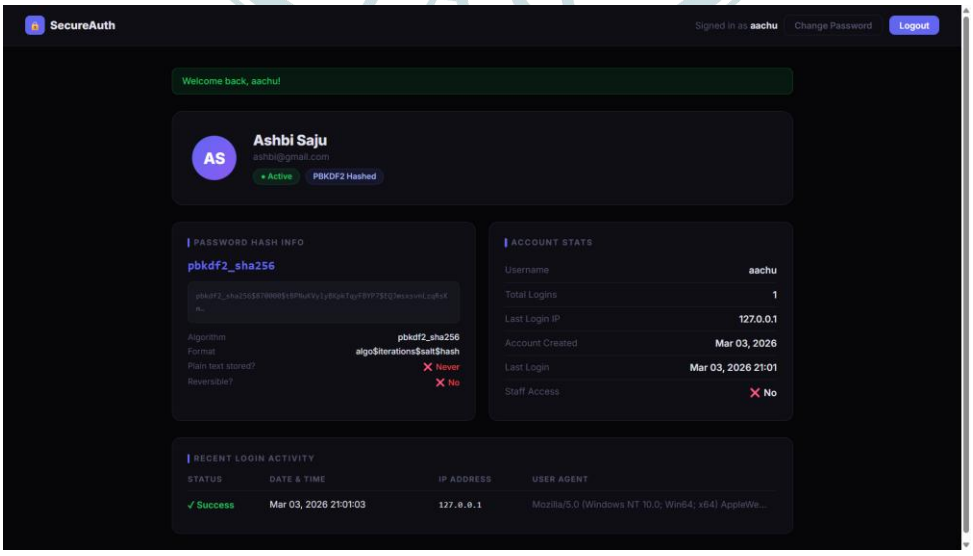
def clean(self):
    cleaned = super().clean()
    p1 = cleaned.get('new_password1')
    p2 = cleaned.get('new_password2')
    if p1 and p2 and p1 != p2:
        self.add_error('new_password2', "Passwords do not match.")
    return cleaned

def save(self):
    self.user.set_password(self.cleaned_data['new_password1'])
    self.user.save()
    return self.user
```

Output


SecureAuth
Create account
Sign up with a secure hashed password
PERSONAL INFO
FIRST NAME
Ashbi
LAST NAME
Saju
USERNAME
aachu
EMAIL
ashbi@gmail.com
SECURITY
PASSWORD
CONFIRM PASSWORD
Create Account →
Already have an account? Sign in


SecureAuth
Welcome back
Sign in to your secure account
Account created! Please log in.
USERNAME
aachu
PASSWORD
Sign In →
No account? Create one


SecureAuth
Signed in as aachu
Change Password
Logout
Welcome back, aachu!
AS
Ashbi Saju
ashbi@gmail.com
Active
PBKDF2 Hashed
PASSWORD HASH INFO
pbkdf2_sha256
Algorithm
pbkdf2_sha256
Format
algo$iterations$salt$hash
Plain text stored?
Never
Reversible?
No
ACCOUNT STATS
Username
aachu
Total Logins
1
Last Login IP
127.0.0.1
Account Created
Mar 03, 2026
Last Login
Mar 03, 2026 21:01
Staff Access
No
RECENT LOGIN ACTIVITY
STATUS
DATE & TIME
IP ADDRESS
USER AGENT
Success
Mar 03, 2026 21:01:03
127.0.0.1

Program No: 39	**Date:** 27-02-2026

Program Title: Deploy a full stack application on a free hosting platform.

```
<!--
Full Stack Deployment
@ Ashbi Saju
Roll no: 16
Date: 27-02-2026
-->
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.dashboard, name='dashboard'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),

    # Mentee CRUD
    path('students/', views.mentee_list, name='mentee_list'),
    path('students/add/', views.mentee_add, name='mentee_add'),
    path('students/<int:pk>/', views.mentee_view, name='mentee_view'),
    path('students/<int:pk>/edit/', views.mentee_edit, name='mentee_edit'),
    path('students/<int:pk>/delete/', views.mentee_delete, name='mentee_delete'),

    # Meetings
    path('meetings/', views.all_meetings, name='all_meetings'),
    path('meetings/request/<int:mentee_pk>/', views.mentor_request, name='mentor_request'),
    path('meetings/request/', views.mentee_request, name='mentee_request'),
    path('meetings/<int:pk>/respond/<str:action>/', views.meeting_respond),
    path('meetings/<int:pk>/complete/', views.meeting_complete, name='meeting_complete'),
    path('meetings/<int:pk>/cancel/', views.meeting_cancel, name='meeting_cancel'),
]
```

views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login, logout
from django.contrib.auth.decorators import login_required
from django.contrib.auth.models import User
from django.contrib import messages
from django.utils import timezone
```

```
from .models import Profile, Meeting
from .forms import LoginForm, MenteeCreateForm, MenteeEditForm, MeetingForm,
MeetingNotesForm

# ── Helpers ──────────────────────────────────────────────────────────────

def get_mentor():
    p = Profile.objects.filter(role='mentor').first()
    return p.user if p else None

def require_mentor(fn):
    from functools import wraps
    @wraps(fn)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('login')
        if not hasattr(request.user, 'profile') or request.user.profile.role != 'mentor':
            messages.error(request, "Access denied.")
            return redirect('dashboard')
        return fn(request, *args, **kwargs)
    return wrapper

def require_mentee(fn):
    from functools import wraps
    @wraps(fn)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('login')
        if not hasattr(request.user, 'profile') or request.user.profile.role != 'mentee':
            messages.error(request, "Access denied.")
            return redirect('dashboard')
        return fn(request, *args, **kwargs)
    return wrapper

# ── Auth ─────────────────────────────────────────────────────────────────

def login_view(request):
    if request.user.is_authenticated:
        return redirect('dashboard')
    form = LoginForm(request, data=request.POST or None)
    if request.method == 'POST' and form.is_valid():
        login(request, form.get_user())
        return redirect('dashboard')
    return render(request, 'core/login.html', {'form': form})
```

```
@login_required
def logout_view(request):
    logout(request)
    return redirect('login')

# ── Dashboard ──────────────────────────────────────────────────────

@login_required
def dashboard(request):
    user    = request.user
    profile = user.profile

    if profile.is_mentor():
        mentees  = Profile.objects.filter(role='mentee').select_related('user')
        # Requests the mentor must respond to = mentee-initiated, still pending
        incoming = Meeting.objects.filter(mentor=user, status='pending', requested_by='mentee')
        # Requests mentor sent that are still pending
        sent_pending = Meeting.objects.filter(mentor=user, status='pending',
requested_by='mentor')
        upcoming = Meeting.objects.filter(
            mentor=user, status='accepted', meeting_date__gt=timezone.now()
        ).order_by('meeting_date')[:6]
        past = Meeting.objects.filter(
            mentor=user, meeting_date__lt=timezone.now()
        ).exclude(status='pending').order_by('-meeting_date')[:6]
        return render(request, 'core/mentor_dashboard.html', {
            'mentees': mentees, 'incoming': incoming,
            'sent_pending': sent_pending,
            'upcoming': upcoming, 'past': past,
        })

    else:
        mentor   = get_mentor()
        # Requests mentee must respond to = mentor-initiated, still pending
        incoming = Meeting.objects.filter(mentee=user, status='pending', requested_by='mentor')
        # Requests mentee sent, still pending
        sent_pending = Meeting.objects.filter(mentee=user, status='pending',
requested_by='mentee')
        upcoming = Meeting.objects.filter(
            mentee=user, status='accepted', meeting_date__gt=timezone.now()
        ).order_by('meeting_date')[:6]
        past = Meeting.objects.filter(
            mentee=user, meeting_date__lt=timezone.now()
        ).exclude(status='pending').order_by('-meeting_date')[:6]
        return render(request, 'core/mentee_dashboard.html', {
            'mentor': mentor, 'incoming': incoming,
```

```
        'sent_pending': sent_pending,
        'upcoming': upcoming, 'past': past,
    })

# ── Mentee CRUD (Mentor only) ──────────────────────────────────────

@require_mentor
def mentee_list(request):
    mentees = Profile.objects.filter(role='mentee').select_related('user')
    return render(request, 'core/mentee_list.html', {'mentees': mentees})

@require_mentor
def mentee_add(request):
    form = MenteeCreateForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        user = form.save()
        messages.success(request, f"Student '{user.get_full_name() or user.username}' added!")
        return redirect('mentee_list')
    return render(request, 'core/mentee_form.html', {'form': form, 'action': 'Add'})

@require_mentor
def mentee_edit(request, pk):
    mentee  = get_object_or_404(User, pk=pk, profile__role='mentee')
    p       = mentee.profile
    initial = {
        'first_name': mentee.first_name, 'last_name': mentee.last_name,
        'email': mentee.email, 'bio': p.bio, 'phone': p.phone, 'subject': p.subject,
    }
    form = MenteeEditForm(request.POST or None, initial=initial)
    if request.method == 'POST' and form.is_valid():
        d = form.cleaned_data
        mentee.first_name = d['first_name']
        mentee.last_name  = d.get('last_name', '')
        mentee.email      = d.get('email', '')
        mentee.save()
        p.bio = d.get('bio', ''); p.phone = d.get('phone', ''); p.subject = d.get('subject', '')
        p.save()
        messages.success(request, "Student updated.")
        return redirect('mentee_list')
    return render(request, 'core/mentee_form.html', {'form': form, 'action': 'Edit', 'mentee':
mentee})

@require_mentor
def mentee_view(request, pk):
    mentee   = get_object_or_404(User, pk=pk, profile__role='mentee')
    meetings = Meeting.objects.filter(mentee=mentee).order_by('-meeting_date')
    return render(request, 'core/mentee_detail.html', {'mentee': mentee, 'meetings': meetings})
```

```
@require_mentor
def mentee_delete(request, pk):
    mentee = get_object_or_404(User, pk=pk, profile__role='mentee')
    if request.method == 'POST':
        name = mentee.get_full_name() or mentee.username
        mentee.delete()
        messages.success(request, f"Student '{name}' deleted.")
        return redirect('mentee_list')
    return render(request, 'core/mentee_confirm_delete.html', {'mentee': mentee})

# ── Meetings ──────────────────────────────────────────────────────────

@require_mentor
def mentor_request_meeting(request, mentee_pk):
    """Mentor sends a meeting request to a mentee."""
    mentee = get_object_or_404(User, pk=mentee_pk, profile__role='mentee')
    form   = MeetingForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        m = form.save(commit=False)
        m.mentor = request.user; m.mentee = mentee
        m.requested_by = 'mentor'; m.status = 'pending'
        m.save()
        messages.success(request, f"Meeting request sent to {mentee.get_full_name() or
mentee.username}.")
        return redirect('mentee_view', pk=mentee_pk)
    return render(request, 'core/meeting_form.html', {'form': form, 'other': mentee, 'action':
'Request Meeting with'})

@require_mentee
def mentee_request_meeting(request):
    """Mentee sends a meeting request to the mentor."""
    mentor = get_mentor()
    if not mentor:
        messages.error(request, "No mentor found in the system.")
        return redirect('dashboard')
    form = MeetingForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        m = form.save(commit=False)
        m.mentor = mentor; m.mentee = request.user
        m.requested_by = 'mentee'; m.status = 'pending'
        m.save()
        messages.success(request, "Meeting request sent!")
        return redirect('dashboard')
    return render(request, 'core/meeting_form.html', {'form': form, 'other': mentor, 'action':
'Request Meeting with'})
```

```
@login_required
def meeting_respond(request, pk, action):
    """Accept or decline a meeting — only the recipient can respond."""
    meeting = get_object_or_404(Meeting, pk=pk)
    user    = request.user

    # Validate: only the person who did NOT request can respond
    if meeting.requested_by == 'mentor' and user != meeting.mentee:
        messages.error(request, "Not authorised."); return redirect('dashboard')
    if meeting.requested_by == 'mentee' and user != meeting.mentor:
        messages.error(request, "Not authorised."); return redirect('dashboard')
    if meeting.status != 'pending':
        messages.info(request, "This request has already been responded to.")
        return redirect('dashboard')

    if action == 'accept':
        meeting.status = 'accepted'
        messages.success(request, f"Meeting '{meeting.title}' accepted! ✅ ")
    elif action == 'decline':
        meeting.status = 'declined'
        messages.info(request, f"Meeting '{meeting.title}' declined.")
    meeting.save()
    return redirect('dashboard')

@login_required
def meeting_complete(request, pk):
    meeting = get_object_or_404(Meeting, pk=pk)
    if request.user not in [meeting.mentor, meeting.mentee]:
        return redirect('dashboard')
    form = MeetingNotesForm(request.POST or None, instance=meeting)
    if request.method == 'POST' and form.is_valid():
        m = form.save(commit=False)
        m.status = 'completed'; m.save()
        messages.success(request, "Meeting marked as completed.")
        return redirect('all_meetings')
    return render(request, 'core/meeting_notes.html', {'form': form, 'meeting': meeting})

@login_required
def meeting_cancel(request, pk):
    meeting = get_object_or_404(Meeting, pk=pk)
    if request.user not in [meeting.mentor, meeting.mentee]:
        return redirect('dashboard')
    if request.method == 'POST':
        meeting.status = 'cancelled'; meeting.save()
        messages.info(request, "Meeting cancelled.")
        return redirect('all_meetings')
```

```
    return render(request, 'core/meeting_cancel_confirm.html', {'meeting': meeting})

@login_required
def all_meetings(request):
    user = request.user
    qs   = (Meeting.objects.filter(mentor=user) if user.profile.is_mentor()
            else Meeting.objects.filter(mentee=user))
    return render(request, 'core/all_meetings.html', {'meetings': qs.order_by('-meeting_date')})
```

admins.py

```
from django.contrib import admin
from .models import Profile, Meeting

@admin.register(Profile)
class ProfileAdmin(admin.ModelAdmin):
    list_display  = ['user', 'role', 'subject', 'phone']
    list_filter   = ['role']

@admin.register(Meeting)
class MeetingAdmin(admin.ModelAdmin):
    list_display  = ['title', 'mentor', 'mentee', 'meeting_date', 'status', 'requested_by']
    list_filter   = ['status', 'requested_by']
    search_fields = ['title', 'mentor__username', 'mentee__username']
```

models.py

```
from django.db import models
from django.contrib.auth.models import User

class Profile(models.Model):
    ROLE_CHOICES = [('mentor', 'Mentor'), ('mentee', 'Mentee')]
    user      = models.OneToOneField(User, on_delete=models.CASCADE, related_name='profile')
    role      = models.CharField(max_length=10, choices=ROLE_CHOICES)
    bio       = models.TextField(max_length=300, blank=True)
    phone     = models.CharField(max_length=20, blank=True)
    subject   = models.CharField(max_length=100, blank=True)
    avatar_color = models.CharField(max_length=20, default='#6366f1')

    def __str__(self):
        return f"{self.user.get_full_name() or self.user.username} ({self.role})"

    def is_mentor(self):
        return self.role == 'mentor'
```

```
    def is_mentee(self):
        return self.role == 'mentee'

class Meeting(models.Model):
    STATUS_CHOICES = [
        ('pending',   'Pending'),
        ('accepted',  'Accepted'),
        ('declined',  'Declined'),
        ('completed', 'Completed'),
        ('cancelled', 'Cancelled'),
    ]
    REQUESTER_CHOICES = [('mentor', 'Mentor'), ('mentee', 'Mentee')]

    mentor = models.ForeignKey(User, on_delete=models.CASCADE)
    mentee  = models.ForeignKey(User, on_delete=models.CASCADE)
    title       = models.CharField(max_length=200)
    description  = models.TextField(blank=True)
    meeting_date = models.DateTimeField()
    status      = models.CharField(max_length=15, choices=STATUS_CHOICES, default='pending')
    requested_by = models.CharField(max_length=10, choices=REQUESTER_CHOICES)
    notes       = models.TextField(blank=True)
    created_at   = models.DateTimeField(auto_now_add=True)
    updated_at   = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.title} ({self.mentor.username} ↔ {self.mentee.username})"

    def is_past(self):
        from django.utils import timezone
        return self.meeting_date < timezone.now()
```

forms.py

```
from django import forms
from django.contrib.auth.models import User
from django.contrib.auth.forms import AuthenticationForm
from .models import Profile, Meeting

class LoginForm(AuthenticationForm):
    username = forms.CharField(
        widget=forms.TextInput(attrs={'placeholder': 'Username', 'class': 'fi', 'autofocus': True})
    )
    password = forms.CharField(
        widget=forms.PasswordInput(attrs={'placeholder': 'Password', 'class': 'fi'})
    )
```

```
class MenteeCreateForm(forms.Form):
    first_name = forms.CharField(max_length=50,  widget=forms.TextInput(attrs={'class': 'fi',
'placeholder': 'First name'}))
    last_name  = forms.CharField(max_length=50,  required=False,
widget=forms.TextInput(attrs={'class': 'fi', 'placeholder': 'Last name'}))
    username   = forms.CharField(max_length=30,  widget=forms.TextInput(attrs={'class': 'fi',
'placeholder': 'username'}))
    email      = forms.EmailField(required=False, widget=forms.EmailInput(attrs={'class': 'fi',
'placeholder': 'email@example.com'}))
    password   = forms.CharField(widget=forms.PasswordInput(attrs={'class': 'fi', 'placeholder':
'Set password'}))
    subject    = forms.CharField(max_length=100, required=False,
widget=forms.TextInput(attrs={'class': 'fi', 'placeholder': 'e.g. Computer Science'}))
    phone      = forms.CharField(max_length=20,  required=False,
widget=forms.TextInput(attrs={'class': 'fi', 'placeholder': '+91 ...'}))
    bio        = forms.CharField(required=False,  widget=forms.Textarea(attrs={'class': 'fi', 'rows': 2,
'placeholder': 'Short bio...'}))

    def clean_username(self):
        uname = self.cleaned_data['username'].strip()
        if User.objects.filter(username__iexact=uname).exists():
            raise forms.ValidationError("Username already taken.")
        return uname

    def save(self):
        import random
        d = self.cleaned_data
        user = User.objects.create_user(
            username=d['username'], password=d['password'],
            first_name=d['first_name'], last_name=d.get('last_name', ''),
            email=d.get('email', ''),
        )
        colors = ['#6366f1','#8b5cf6','#ec4899','#14b8a6','#f59e0b','#10b981','#3b82f6','#f43f5e']
        Profile.objects.create(
            user=user, role='mentee',
            subject=d.get('subject',''), phone=d.get('phone',''),
            bio=d.get('bio',''), avatar_color=random.choice(colors),
        )
        return user

class MenteeEditForm(forms.Form):
    first_name = forms.CharField(max_length=50,  widget=forms.TextInput(attrs={'class': 'fi'}))
    last_name  = forms.CharField(max_length=50,  required=False,
widget=forms.TextInput(attrs={'class': 'fi'}))
    email      = forms.EmailField(required=False, widget=forms.EmailInput(attrs={'class': 'fi'}))
    subject    = forms.CharField(max_length=100, required=False,
widget=forms.TextInput(attrs={'class': 'fi'}))
```

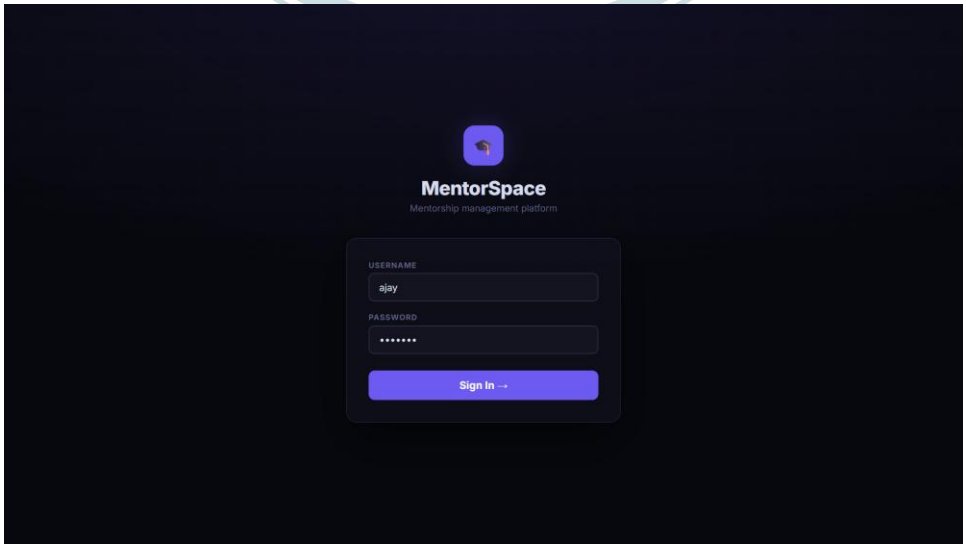
```
    phone      = forms.CharField(max_length=20,  required=False,
widget=forms.TextInput(attrs={'class': 'fi'}))
    bio        = forms.CharField(required=False,  widget=forms.Textarea(attrs={'class': 'fi', 'rows':
2}))

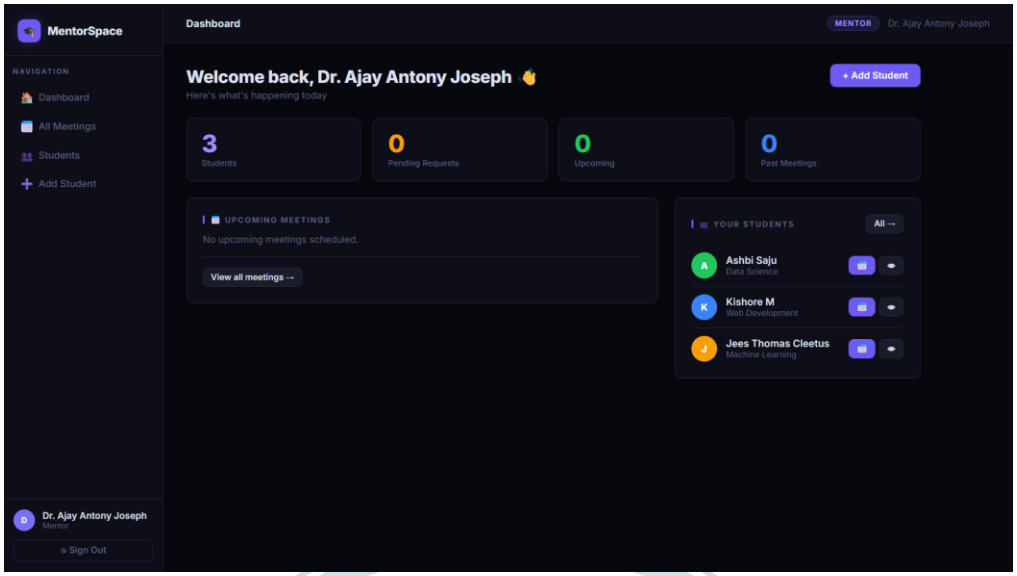
class MeetingForm(forms.ModelForm):
    meeting_date = forms.DateTimeField(
        widget=forms.DateTimeInput(attrs={'type': 'datetime-local', 'class': 'fi'}),
        input_formats=['%Y-%m-%dT%H:%M'],
    )

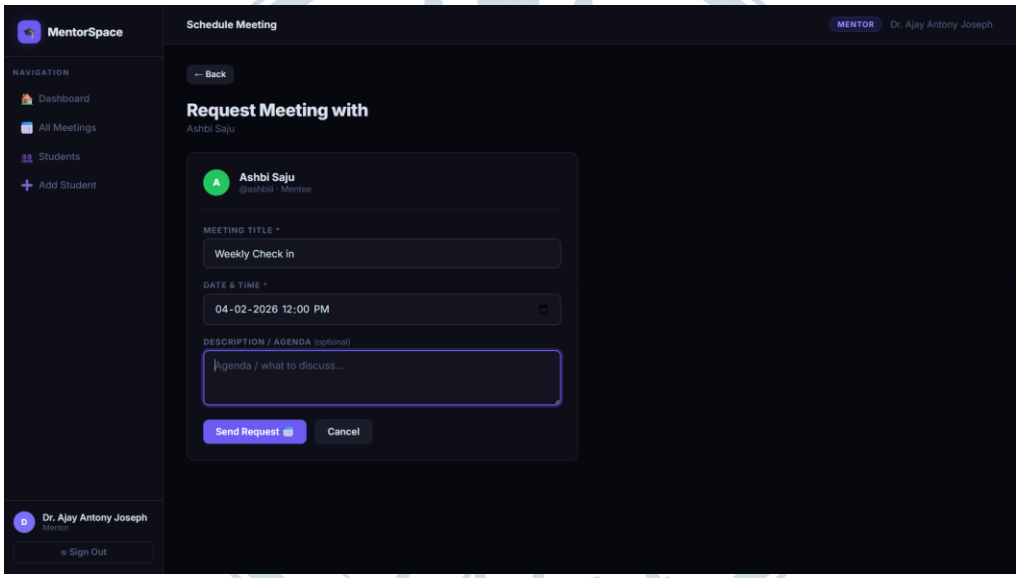
    class Meta:
        model  = Meeting
        fields = ['title', 'description', 'meeting_date']
        widgets = {
            'title':       forms.TextInput(attrs={'class': 'fi', 'placeholder': 'e.g. Weekly Check-in'}),
            'description': forms.Textarea(attrs={'class': 'fi', 'rows': 3, 'placeholder': 'Agenda / what to
discuss...'}),
        }

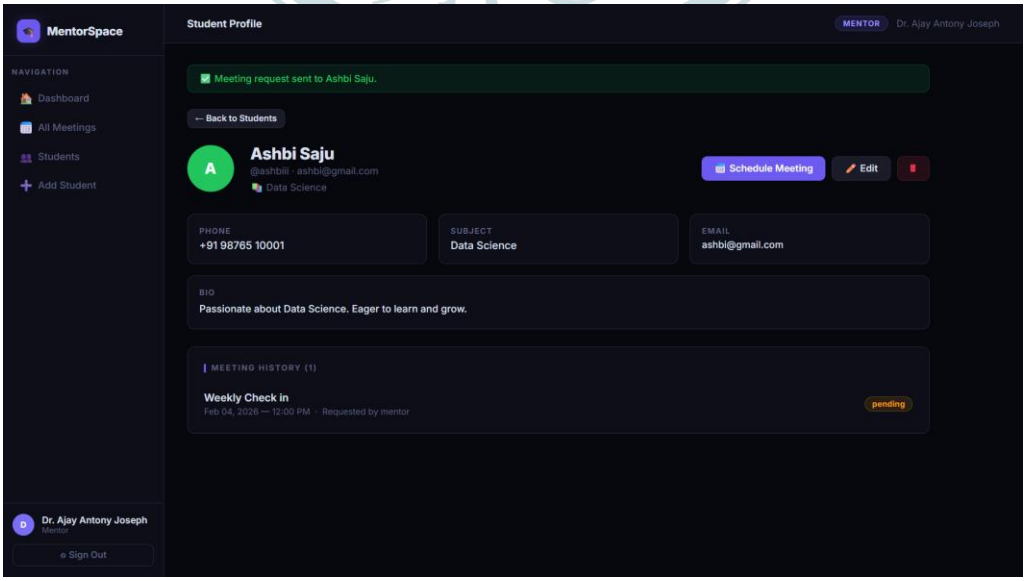
class MeetingNotesForm(forms.ModelForm):
    class Meta:
        model  = Meeting
        fields = ['notes']
        widgets = {
            'notes': forms.Textarea(attrs={'class': 'fi', 'rows': 4, 'placeholder': 'Post-meeting notes,
action items...'})
        }
```

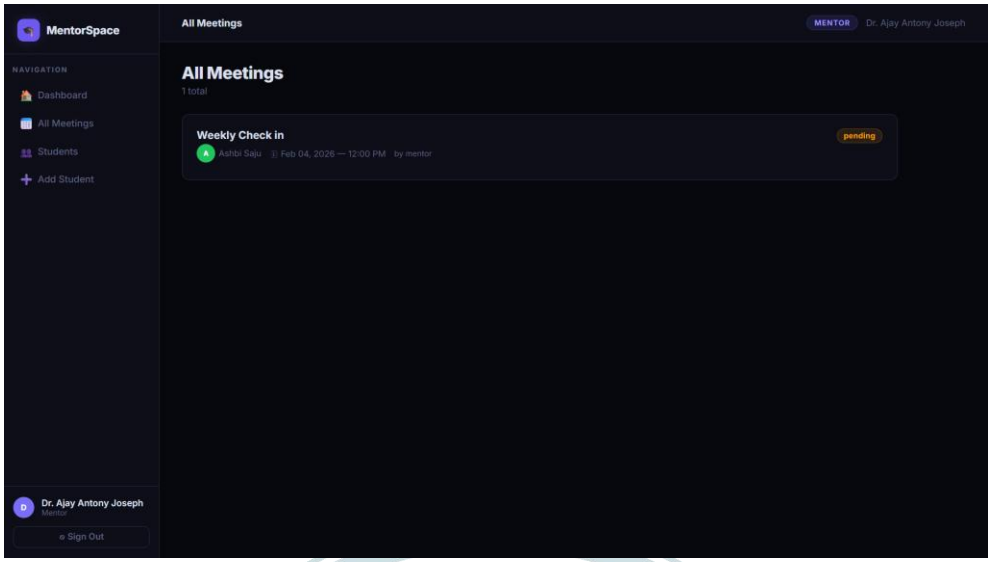
Output

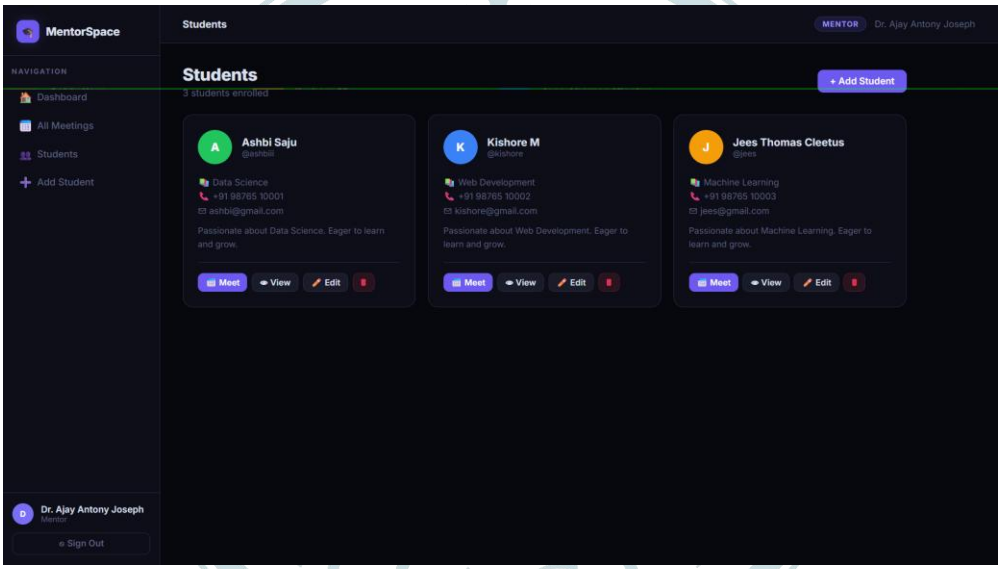


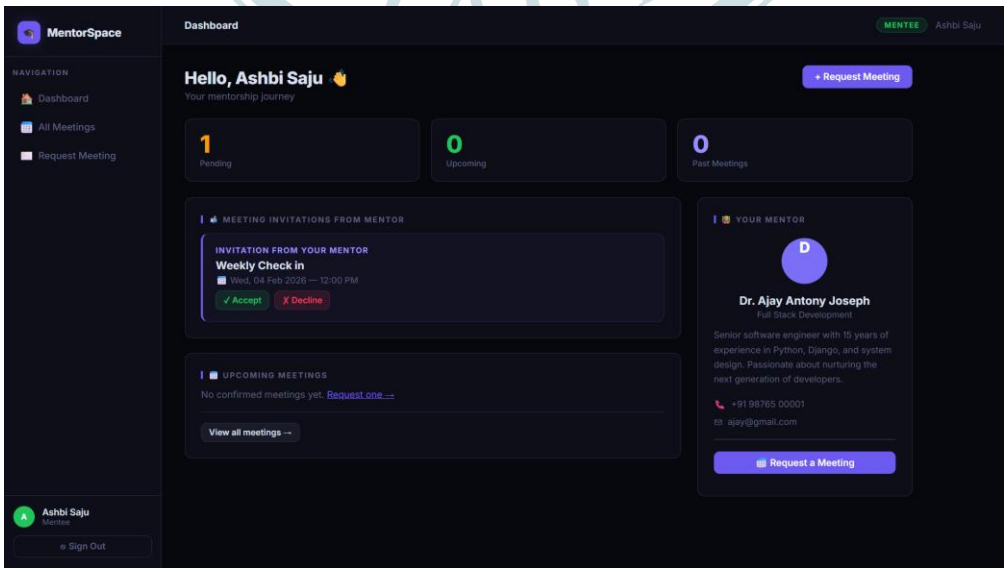

MentorSpace
Dashboard
MENTOR
Dr. Ajay Antony Joseph
NAVIGATION
Dashboard
All Meetings
Students
Add Student
Welcome back, Dr. Ajay Antony Joseph
Here's what's happening today
+ Add Student
3
Students
0
Pending Requests
0
Upcoming
0
Past Meetings
UPCOMING MEETINGS
No upcoming meetings scheduled.
View all meetings →
YOUR STUDENTS
All →
Ashbi Saju
Data Science
Kishore M
Web Development
Jees Thomas Cleetus
Machine Learning
Dr. Ajay Antony Joseph
Mentor
Sign Out


MentorSpace
Schedule Meeting
MENTOR
Dr. Ajay Antony Joseph
NAVIGATION
Dashboard
All Meetings
Students
Add Student
← Back
Request Meeting with
Ashbi Saju
Ashbi Saju
@ashbiii · Mentee
MEETING TITLE *
Weekly Check in
DATE & TIME *
04-02-2026 12:00 PM
DESCRIPTION / AGENDA (optional)
Agenda / what to discuss...
Send Request
Cancel
Dr. Ajay Antony Joseph
Mentor
Sign Out

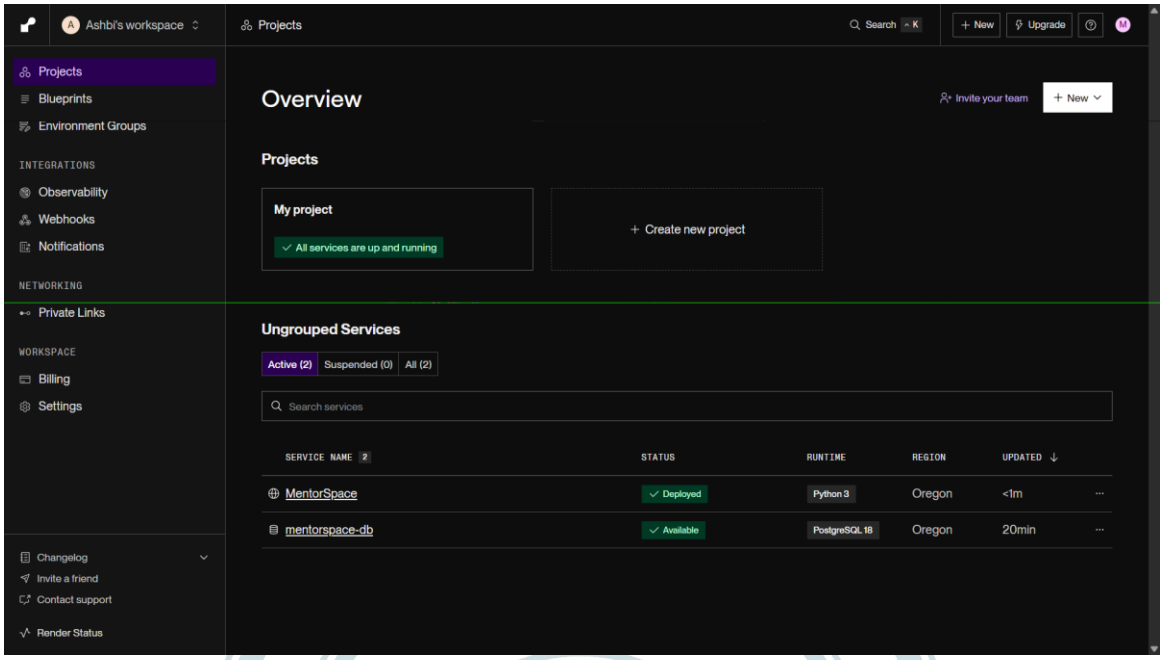

MentorSpace
Student Profile
MENTOR
Dr. Ajay Antony Joseph
NAVIGATION
Dashboard
All Meetings
Students
Add Student
Meeting request sent to Ashbi Saju.
← Back to Students
Ashbi Saju
@ashbiii · ashbi@gmail.com
Data Science
Schedule Meeting
Edit
PHONE
+91 98765 10001
SUBJECT
Data Science
EMAIL
ashbi@gmail.com
BIO
Passionate about Data Science. Eager to learn and grow.
MEETING HISTORY (1)
Weekly Check in
Feb 04, 2026 — 12:00 PM · Requested by mentor
pending
Dr. Ajay Antony Joseph
Mentor
Sign Out

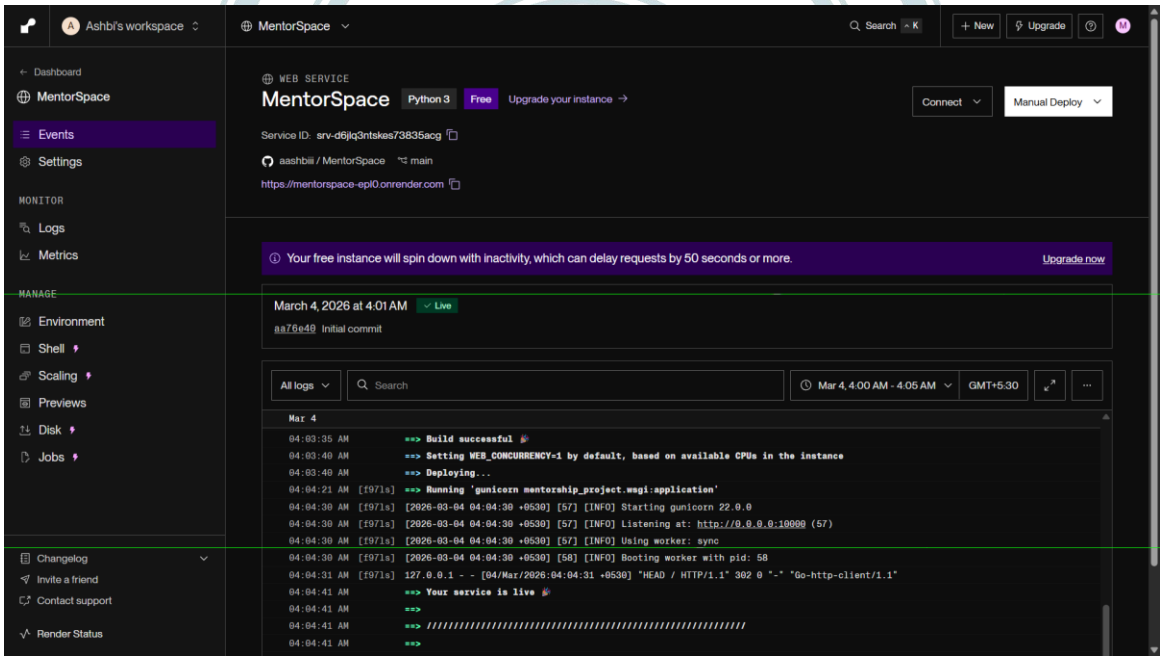

MentorSpace
All Meetings
MENTOR
Dr. Ajay Antony Joseph
NAVIGATION
Dashboard
All Meetings
Students
Add Student
All Meetings
1 total
Weekly Check in
Ashbi Saju
Feb 04, 2026 — 12:00 PM
by mentor
pending
Dr. Ajay Antony Joseph
Mentor
Sign Out


MentorSpace
Students
MENTOR
Dr. Ajay Antony Joseph
NAVIGATION
Dashboard
All Meetings
Students
Add Student
Students
3 students enrolled
+ Add Student
Ashbi Saju
Data Science
+91 98765 10001
ashbi@gmail.com
Passionate about Data Science. Eager to learn and grow.
Meet
View
Edit
Kishore M
Web Development
+91 98765 10002
kishore@gmail.com
Passionate about Web Development. Eager to learn and grow.
Meet
View
Edit
Jees Thomas Cleetus
Machine Learning
+91 98765 10003
jees@gmail.com
Passionate about Machine Learning. Eager to learn and grow.
Meet
View
Edit
Dr. Ajay Antony Joseph
Mentor
Sign Out


MentorSpace
Dashboard
MENTEE
Ashbi Saju
NAVIGATION
Dashboard
All Meetings
Request Meeting
Hello, Ashbi Saju
Your mentorship journey
+ Request Meeting
1
Pending
0
Upcoming
0
Past Meetings
MEETING INVITATIONS FROM MENTOR
INVITATION FROM YOUR MENTOR
Weekly Check in
Wed, 04 Feb 2026 — 12:00 PM
✓ Accept
✗ Decline
UPCOMING MEETINGS
No confirmed meetings yet. Request one →
View all meetings →
YOUR MENTOR
D
Dr. Ajay Antony Joseph
Full Stack Development
Senior software engineer with 15 years of experience in Python, Django, and system design. Passionate about nurturing the next generation of developers.
+91 98765 00001
ajay@gmail.com
Request a Meeting
Ashbi Saju
Mentee
Sign Out

Deployed on Render





Program No: 40	**Date:** 04-03-2026

Program Title: Develop a socially relevant mini project aligned with UN SDGs.

```
<!--
UN SDG Mini Project
@ Ashbi Saju
Roll no: 16
Date: 06-03-2026
-->
```

urls.py

```
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static

import MentrixApp.views

from . import views

urlpatterns = [
    path('', MentrixApp.views.home, name='home'),
    path('logout', views.logout, name='logout'),

    path('faculty/', views.faculty_dashboard, name='faculty_dashboard'),
    path('students/', views.faculty_students, name='faculty_students'),
    path('student/<int:student_id>/', views.student_profile, name='student_profile'),
    path('student/<int:student_id>/mobile/', views.student_mobile, name='student_mobile'),
    path('student/<int:id>/sessions/', views.student_session, name="student_session"),
    path('session/<int:session_id>/generate-ai/', views.generate_ai, name='generate _ai'),
    path('student/<int:id>/upload-application/', views.upload_app, name='upload_app'),
    path('student/<int:id>/upload-assessment/', views.upload_assess, name='upload_assess'),
    path('student/<int:id>/new-session/', views.new_session, name='new_session'),
    path('session/<int:id>/details/', views.session_details, name='session_details'),
    path('profile/', views.faculty_profile, name='faculty_profile'),
    path('filter-students/', views.filter_students, name='filter_students'),
    path('session/<int:session_id>/', views.view_session, name='view_session'),
    path('student/<int:student_id>/ai-summary/', views.ai_summary, name='ai_summary'),
    path('faculty/session-requests/', views.session_requests, name='session_requests'),
    path('faculty/requests/accept/<int:request_id>/',views.accept_session,),
    path('faculty/requests/reject/<int:request_id>/',views.reject _request,),
    path('faculty/students/',views.faculty_ViewStudents,name='students-view'),
    path('faculty/view-schedule/', views.view_schedule, name='view_schedule'),
    path('faculty/schedule-status/<int:schedule_id>/', views.get_schedule,),
```

```
    path("faculty/get_sessions/", views.faculty_api_get_sessions, name="faculty_get_sessions"),
    path("faculty/get_day_sessions/<str:day>/", views.faculty_api_get_day_sessions),

    path("api/get_sessions/", views.api_get_sessions),
    path("api/get_day_sessions/<date_str>", views.api_get_day_sessions),
    path("api/get_suggestions/", views.api_get_suggestions, name='api_get_suggestions'),

] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_POST
from MentrixApp.models import *
import json
from datetime import date

def faculty_dashboard(request):
    login_id = request.session.get('login_id')
    if not login_id:
        return redirect('home')

    faculty = get_object_or_404(Faculty, login_id=login_id)
    dept = faculty.department
    today = date.today()

    active_sessions = Schedule.objects.filter(
        batch__course__department=dept,
        start_date__lte=today,
        end_date__gte=today
    )

    pending_sessions = Schedule.objects.filter(
        batch__course__department=dept,
        start_date__gt=today
    )

    missed_sessions = Schedule.objects.filter(
        batch__course__department=dept,
        end_date__lt=today
    )

    # Fetch pending session requests for this faculty
    session_requests = SessionRequest.objects.filter(
        faculty=faculty,
```

```
    status='Pending'
).order_by('-request_date')

# Fetch accepted session requests (for Calendar)
accepted_requests = SessionRequest.objects.filter(
    faculty=faculty,
    status='Accepted'
).order_by('session_date')

# Fetch completed sessions (Schedules that are past or marked completed)
completed_schedules = Schedule.objects.filter(
    batch__course__department=dept
).filter(
    Q(status='completed') | Q(end_date__lt=today)
)

# Fetch completed requests (Individual sessions driven by requests)
completed_requests = SessionRequest.objects.filter(
    faculty=faculty,
    status='Completed'
)

# --- Build Calendar Events JSON ---
calendar_events = []

# Active Sessions
for s in active_sessions:
    calendar_events.append({
        "name": s.batch.batch_name,
        "start": f"{s.start_date.isoformat()}T00:00:00",
        "end": f"{s.end_date.isoformat()}T23:59:59",
        "status": s.status.lower(),
        "show_time": False
    })

# Pending Sessions (Upcoming)
for s in pending_sessions:
    calendar_events.append({
        "name": s.batch.batch_name,
        "start": f"{s.start_date.isoformat()}T00:00:00",
        "end": f"{s.end_date.isoformat()}T23:59:59",
        "status": "pending",
        "show_time": False
    })

# Accepted Requests
for r in accepted_requests:
```

```
        start_dt = f"{r.session_date.isoformat()}T{r.session_time.strftime('%H:%M')}" if
r.session_date and r.session_time else None
        if start_dt:
            calendar_events.append({
                "name": f"{r.student.name} (Requested)",
                "start": start_dt,
                "status": "accepted",
                "show_time": True
            })

    # Completed Schedules
    for s in completed_schedules:
        calendar_events.append({
            "name": s.batch.batch_name,
            "start": f"{s.start_date.isoformat()}T00:00:00",
            "status": "completed",
            "show_time": False
        })

    # Completed Requests
    for r in completed_requests:
        start_dt = f"{r.session_date.isoformat()}T{r.session_time.strftime('%H:%M')}" if
r.session_date and r.session_time else None
        if start_dt:
            calendar_events.append({
                "name": f"{r.student.name} (Completed)",
                "start": start_dt,
                "status": "completed",
                "show_time": True
            })

    return render(request, 'Faculty/faculty_dashboard.html', {
        'faculty': faculty,
        'active_sessions': active_sessions,
        'pending_sessions': pending_sessions,
        'missed_sessions': missed_sessions,
        'session_requests': session_requests,
        'accepted_requests': accepted_requests,
        'completed_schedules': completed_schedules,
        'completed_requests': completed_requests,
        'calendar_events_json': json.dumps(calendar_events),
    })

#Logout
def logout(request):
    request.session.flush()   # clears session completely
    return redirect('/')
```

```
def student_profile(request, student_id):
    login_id = request.session.get('login_id')
    if not login_id:
        return redirect('home')

    faculty = get_object_or_404(Faculty, login_id=login_id)
    student = get_object_or_404(Student, student_id=student_id)

    context = {
        'faculty': faculty,   # logged-in faculty
        'student': student,
    }

    return render(request, 'Faculty/student_profile.html', context)

def student_session(request, id):
    faculty_login_id = request.session.get('login_id')
    student = Student.objects.get(student_id=id)
    sessions = MentoringSession.objects.filter(student__student_id=id).order_by('-date')
    student_uploads = StudentUploads.objects.filter(student=student)
    faculty = Faculty.objects.get(login_id=faculty_login_id)
    return render(request, 'Faculty/student_sessions.html', {
        'student': student,
        'faculty': faculty,
        'sessions': sessions,
        'student_uploads': student_uploads,
    })

def session_details(request, id):
    session = get_object_or_404(MentoringSession, id=id)

    return render(request, 'session_details.html', {
        'session': session,
        'student': session.student,
    })

def view_session(request, session_id):
    session = MentoringSession.objects.get(session_id=session_id)
    login_id = request.session.get('login_id')
    if not login_id:
        return redirect('home')

    faculty = get_object_or_404(Faculty, login_id=login_id)
    return render(request, 'Faculty/view_session.html', {'session': session, 'faculty': faculty,})
```

models.py

```
from django.db import models
from django.utils import timezone

class Login (models.Model):
    login_id=models.AutoField(primary_key=True)
    username=models.CharField(max_length=25)
    password=models.CharField(max_length=128)
    userType=models.CharField(max_length=20, default='faculty') # 'student', 'faculty', 'hod',
'admin', 'superadmin'

class Department(models.Model):
    dept_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    code = models.CharField(max_length=20, unique=True)
    hod = models.ForeignKey('Faculty', on_delete=models.SET_NULL, null=True, blank=True,
related_name='department_led')
    email = models.EmailField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)

class Courses(models.Model):
    course_id = models.AutoField(primary_key=True)
    course_name = models.CharField(max_length=20)
    department = models.ForeignKey(Department, on_delete=models.CASCADE)

class Batches(models.Model):
    batch_id = models.AutoField(primary_key=True)
    batch_name = models.CharField(max_length=30,blank=True)
    course = models.ForeignKey(Courses,on_delete=models.CASCADE)
    batch_mail = models.EmailField(max_length=150,db_index=False,blank=True,null=True)

class Faculty(models.Model):
    faculty_id = models.AutoField(primary_key=True)
    login=models.ForeignKey(Login,on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    email = models.EmailField(max_length=150,db_index=False)
    phone = models.CharField(max_length=15, blank=True, null=True)
    department = models.ForeignKey(Department, on_delete=models.CASCADE,
related_name="faculties")
    designation = models.CharField(max_length=100, blank=True, null=True)
    faculty_image = models.ImageField(upload_to='faculty_images/', blank=True, null=True)
    is_hod = models.BooleanField(default=False)  # HOD
    is_admin = models.BooleanField(default=False)       # Admin

class Student(models.Model):
    login = models.ForeignKey(Login, on_delete=models.CASCADE, null=True, blank=True)
```

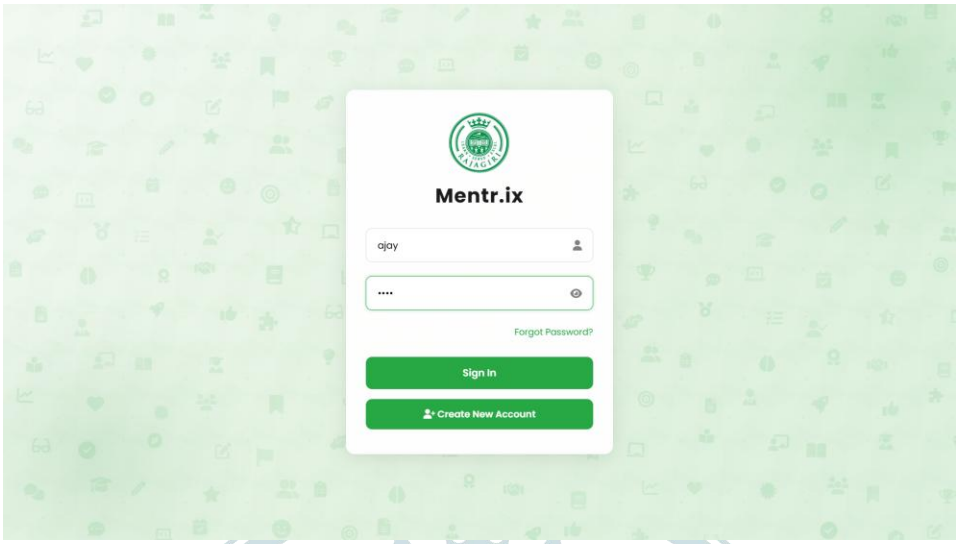
```
    student_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    email = models.EmailField(max_length=150,db_index=False, blank=True, null=True)
    reg_no = models.CharField(max_length=10, unique=True)
    phone = models.CharField(max_length=15, blank=True, null=True)
    department = models.ForeignKey(Department, on_delete=models.CASCADE,
related_name="students")
    student_image = models.FileField(blank=True,null=True,upload_to='student_images/')
    course = models.ForeignKey(Courses, on_delete=models.CASCADE)
    batch = models.ForeignKey(Batches,on_delete=models.CASCADE)
    year = models.IntegerField()
    faculty = models.ForeignKey(Faculty, on_delete=models.SET_NULL, null=True,
related_name="students")
    dob = models.DateField(null=True, blank=True)
    application_form = models.FileField(upload_to='application_forms/', null=True, blank=True)
    assessment_file = models.FileField(upload_to='assessments/', null=True, blank=True)

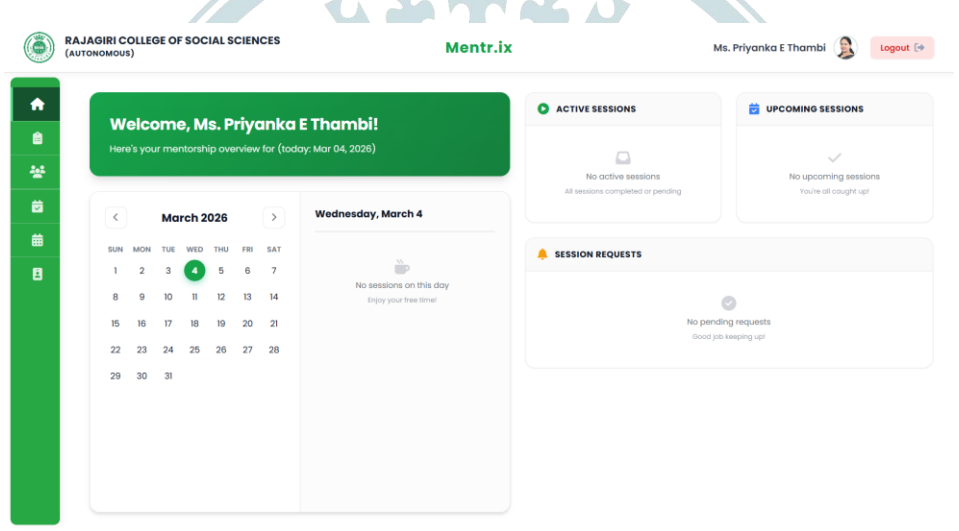
class SessionRequest(models.Model):
    request_id = models.AutoField(primary_key=True)
    student = models.ForeignKey(Student,on_delete=models.CASCADE)
    faculty = models.ForeignKey(Faculty,on_delete=models.CASCADE)
    request_date = models.DateField(default=timezone.now)
    status = models.CharField(max_length=25,default="Pending",choices=[
        ("pending","Pending"),
        ("accepted","Accepted"),
        ("completed","Completed"),
        ("rejected","Rejected"),
        ("cancelled","Cancelled")
        ])

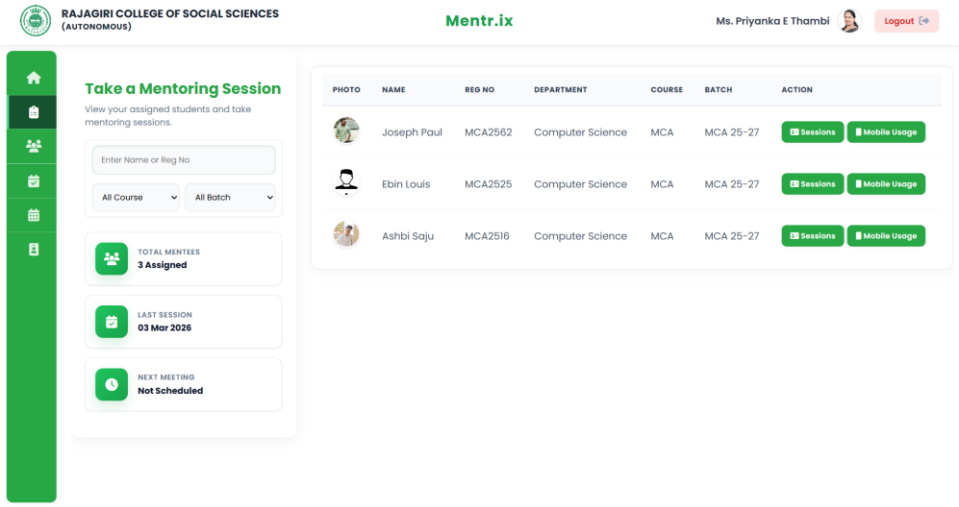
    session_date = models.DateField(null=True,blank=True)
    session_time = models.TimeField(null=True,blank=True)
    comments = models.CharField(max_length=200,blank=True,null=True)

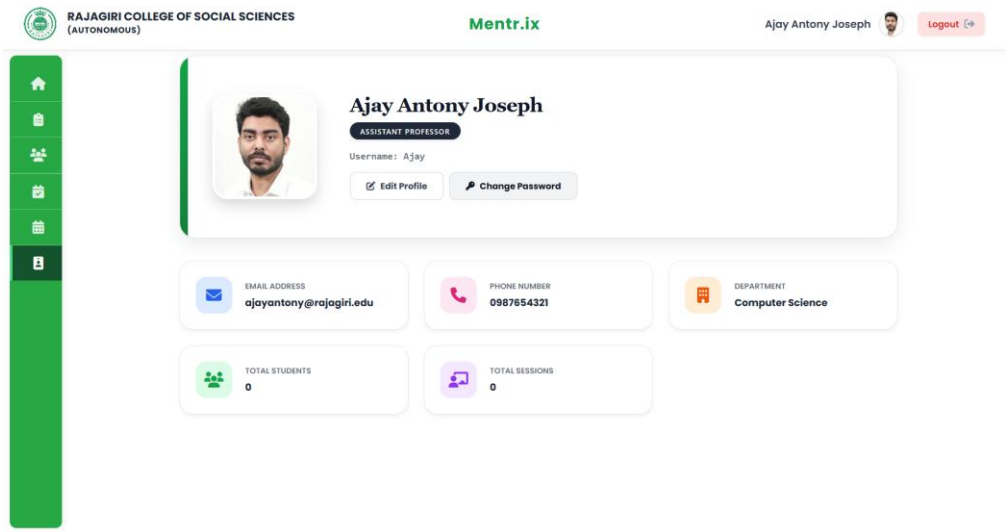
class MobileConfiscation(models.Model):
    confiscation_id = models.AutoField(primary_key=True)
    student = models.ForeignKey(Student, on_delete=models.CASCADE,
related_name="mobile_confiscations")
    faculty = models.ForeignKey(Faculty, on_delete=models.SET_NULL, null=True, blank=True)
    confiscation_date = models.DateField()
    description = models.TextField(blank=True, null=True)
    previous_confiscations = models.IntegerField(default=0)
    sswf_deposit = models.CharField(max_length=10, blank=True, null=True, choices=[('Yes',
'Yes'), ('No', 'No')])
    pdf_file = models.FileField(upload_to='mobile_usage_forms/', blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

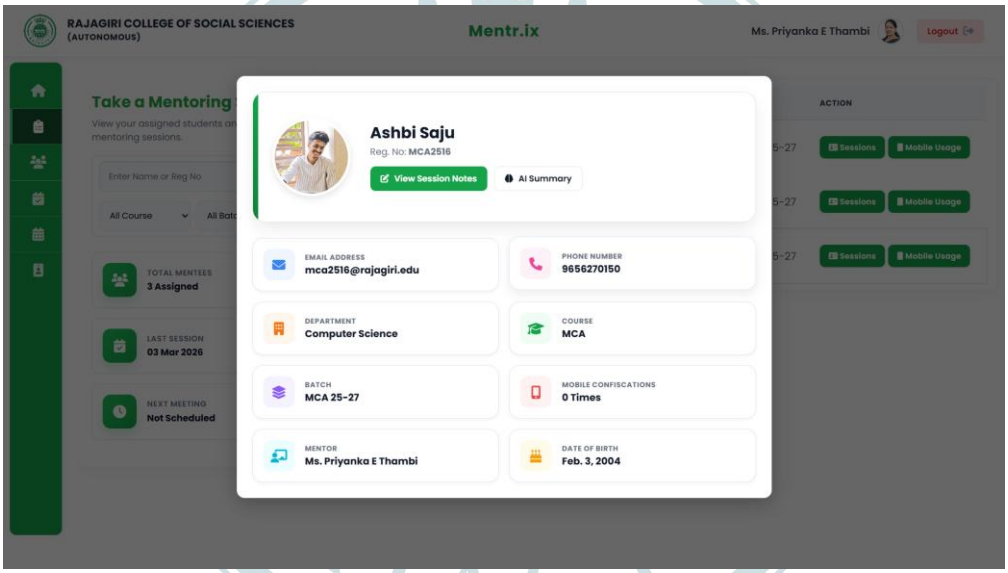
Output

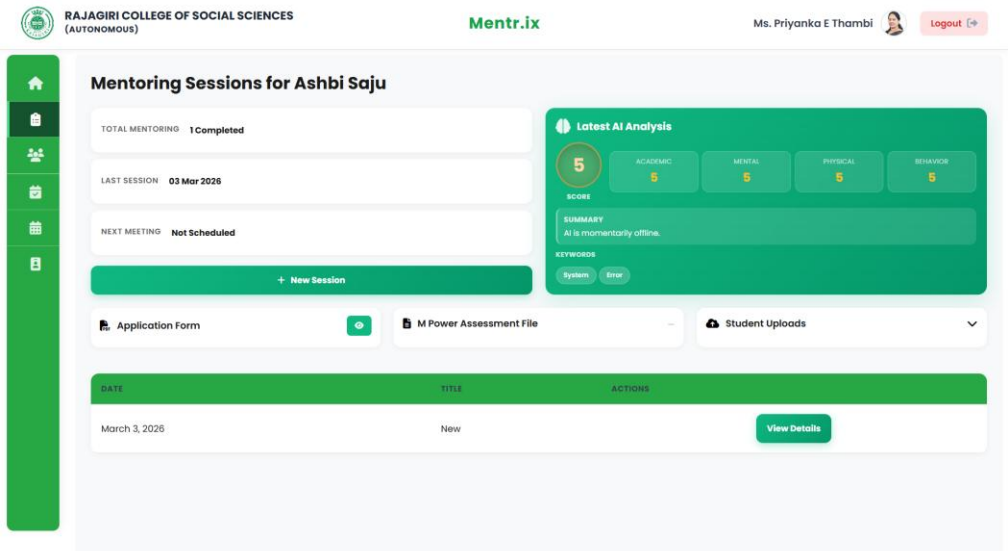

Mentr.ix
ajay
Forgot Password?
Sign in
Create New Account


RAJAGIRI COLLEGE OF SOCIAL SCIENCES
(AUTONOMOUS)
Mentr.ix
Ms. Priyanka E Thambi
Logout
Welcome, Ms. Priyanka E Thambi!
Here's your mentorship overview for (today: Mar 04, 2026)
March 2026
Wednesday, March 4
No sessions on this day
ACTIVE SESSIONS
No active sessions
UPCOMING SESSIONS
No upcoming sessions
SESSION REQUESTS
No pending requests


RAJAGIRI COLLEGE OF SOCIAL SCIENCES
(AUTONOMOUS)
Mentr.ix
Ms. Priyanka E Thambi
Logout
Take a Mentoring Session
View your assigned students and take mentoring sessions.
Enter Name or Reg No
All Course
All Batch
TOTAL MENTEES
3 Assigned
LAST SESSION
03 Mar 2026
NEXT MEETING
Not Scheduled
PHOTO
NAME
REG NO
DEPARTMENT
COURSE
BATCH
ACTION
Joseph Paul
MCA2562
Computer Science
MCA
MCA 25-27
Sessions
Mobile Usage
Ebin Louis
MCA2525
Computer Science
MCA
MCA 25-27
Sessions
Mobile Usage
Ashbi Saju
MCA2516
Computer Science
MCA
MCA 25-27
Sessions
Mobile Usage


RAJAGIRI COLLEGE OF SOCIAL SCIENCES
(AUTONOMOUS)
Mentr.ix
Ajay Antony Joseph
Logout
Ajay Antony Joseph
ASSISTANT PROFESSOR
Username: Ajay
Edit Profile
Change Password
EMAIL ADDRESS
ajayantony@rajagiri.edu
PHONE NUMBER
0987654321
DEPARTMENT
Computer Science
TOTAL STUDENTS
0
TOTAL SESSIONS
0


RAJAGIRI COLLEGE OF SOCIAL SCIENCES
(AUTONOMOUS)
Mentr.ix
Ms. Priyanka E Thambi
Logout
Take a Mentoring
ACTION
Ashbi Saju
Reg. No: MCA2516
View Session Notes
AI Summary
EMAIL ADDRESS
mca2516@rajagiri.edu
PHONE NUMBER
9656270150
DEPARTMENT
Computer Science
COURSE
MCA
BATCH
MCA 25-27
MOBILE CONFISCATIONS
0 Times
MENTOR
Ms. Priyanka E Thambi
DATE OF BIRTH
Feb. 3, 2004
TOTAL MENTEES
3 Assigned
LAST SESSION
03 Mar 2026
NEXT MEETING
Not Scheduled


RAJAGIRI COLLEGE OF SOCIAL SCIENCES
(AUTONOMOUS)
Mentr.ix
Ms. Priyanka E Thambi
Logout
Mentoring Sessions for Ashbi Saju
TOTAL MENTORING 1 Completed
LAST SESSION 03 Mar 2026
NEXT MEETING Not Scheduled
+ New Session
Latest AI Analysis
5
SCORE
ACADEMIC 5
MENTAL 5
PHYSICAL 5
BEHAVIOR 5
SUMMARY
AI is momentarily offline.
KEYWORDS
System
Error
Application Form
M Power Assessment File
Student Uploads
DATE
TITLE
ACTIONS
March 3, 2026
New
View Details

LEARN • SERVE • EXCEL
RAJAGIRI